

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



**BÁO CÁO BÀI TẬP 3 PHÁT TRIỂN CÁC HỆ
THỐNG THÔNG MINH**

Giảng viên hướng dẫn	: PGS.TS TRẦN ĐÌNH QUẾ
Họ và tên sinh viên	: TRẦN MẠNH DŨNG
Mã sinh viên	: B23DCCN214
Lớp	: 05

Hà Nội – 2026

MỤC LỤC

1. Dự đoán bệnh tiểu đường	5
1.1. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU	6
1.2. CƠ SỞ LÝ THUYẾT	6
1.2.1. Deep Learning là gì?	6
1.2.2. So sánh Machine Learning truyền thống và Deep Learning	7
1.2.3. Kiến trúc mạng nơ-ron sử dụng	7
1.3. DỮ LIỆU VÀ TIỀN XỬ LÝ	8
1.3.1. Chia tập dữ liệu	8
1.3.2. Chuẩn hóa dữ liệu	8
1.4. PHƯƠNG PHÁP XÂY DỰNG MÔ HÌNH	8
1.4.1. Lan truyền tiến (Forward Propagation)	8
1.4.2. Hàm mất mát (Binary Cross-Entropy)	9
1.4.3. Lan truyền ngược (Backpropagation)	9
1.4.4. Cập nhật trọng số (Gradient Descent)	9
1.5. CÀI ĐẶT VÀ HUẤN LUYỆN MÔ HÌNH	9
1.6. PHÂN TÍCH CHI TIẾT TỪNG CELL CODE TRONG JUPYTER NOTEBOOK	10
1.7. GIẢI THÍCH CHI TIẾT CÁC HÀM TRONG CHƯƠNG TRÌNH	18
1.8. KẾT QUẢ THỰC NGHIỆM	23
1.9. NHẬN XÉT VÀ THẢO LUẬN	24
1.10. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	25
2. Dự đoán giá nhà Việt Nam	26
2.1. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU	26
2.2. CƠ SỞ LÝ THUYẾT	27
2.2.1. Bài toán hồi quy (Regression) trong Deep Learning	27
2.2.2. Sự khác biệt so với bài toán phân loại nhị phân	27
2.2.3. Kiến trúc mạng nơ-ron sâu 5 lớp sử dụng	28
2.3. DỮ LIỆU VÀ TIỀN XỬ LÝ	28
2.3.1. Làm sạch dữ liệu	29
2.3.2. Chia tập dữ liệu	29
2.3.3. Chuẩn hóa dữ liệu (cả đặc trưng lẫn nhãn)	29
2.4. PHƯƠNG PHÁP XÂY DỰNG MÔ HÌNH	29
2.4.1. Lan truyền tiến (Forward Propagation)	29
2.4.2. Hàm mất mát MSE (Mean Squared Error)	30
2.4.3. Lan truyền ngược (Backpropagation)	30
2.4.4. Cập nhật trọng số (Gradient Descent)	30
2.5. CÀI ĐẶT VÀ HUẤN LUYỆN MÔ HÌNH	30
2.6. PHÂN TÍCH CHI TIẾT TỪNG CELL CODE TRONG JUPYTER NOTEBOOK	31
2.7. GIẢI THÍCH CHI TIẾT CÁC HÀM TRONG CHƯƠNG TRÌNH	45
2.8. KẾT QUẢ THỰC NGHIỆM	49

2.9. NHẬN XÉT VÀ THẢO LUẬN	51
2.10. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	52
3. Dự đoán chi tiêu khách hàng E-commerce	53
3.1. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU	54
3.2. CƠ SỞ LÝ THUYẾT	55
3.2.1. Bài toán dự đoán chi tiêu khách hàng trong phân tích khách hàng	55
3.2.2. Ranh giới giữa Scikit-learn (tiền xử lý) và NumPy tự cài đặt (mô hình)	55
3.2.3. Mã hóa biến phân loại (Label Encoding) và hạn chế của nó	56
3.2.4. Kiến trúc mạng nơ-ron sâu 5 lớp sử dụng	56
3.3. DỮ LIỆU VÀ TIỀN XỬ LÝ	57
3.3.1. Xử lý giá trị thiếu (Missing Values)	58
3.3.2. Mã hóa biến phân loại	58
3.3.3. Lựa chọn đặc trưng (Feature Selection)	58
3.3.4. Chia tập dữ liệu và chuẩn hóa	58
3.4. PHƯƠNG PHÁP XÂY DỰNG MÔ HÌNH	59
3.4.1. Lan truyền tiến (Forward Propagation)	59
3.4.2. Hàm mất mát MSE (Mean Squared Error)	59
3.4.3. Lan truyền ngược (Backpropagation)	59
3.4.4. Cập nhật trọng số (Gradient Descent)	59
3.5. CÀI ĐẶT VÀ HUẤN LUYỆN MÔ HÌNH	60
3.6. PHÂN TÍCH CHI TIẾT TỪNG CELL CODE TRONG JUPYTER NOTEBOOK	60
3.7. GIẢI THÍCH CHI TIẾT CÁC HÀM VÀ CÔNG CỤ TIỀN XỬ LÝ	78
3.8. KẾT QUẢ THỰC NGHIỆM	80
3.9. NHẬN XÉT VÀ THẢO LUẬN	82
3.10. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	84

1. Dự đoán bệnh tiểu đường

XÂY DỰNG MẠNG NƠ-RON NHÂN TẠO TỪ ĐẦU

BẰNG NUMPY CHO BÀI TOÁN DỰ ĐOÁN BỆNH TIỂU ĐƯỜNG

1.1. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU

Học sâu (Deep Learning) hiện là nền tảng của phần lớn các hệ thống trí tuệ nhân tạo hiện đại, từ nhận dạng hình ảnh, xử lý ngôn ngữ tự nhiên đến hỗ trợ chẩn đoán y tế. Tuy nhiên, khi sử dụng trực tiếp các framework như TensorFlow, PyTorch hay Keras, người học dễ coi mạng nơ-ron như một "hộp đen", chỉ gọi hàm mà không hiểu bản chất toán học bên trong.

Báo cáo này trình bày quá trình xây dựng lại một mạng nơ-ron nhân tạo hoàn toàn từ đầu, chỉ sử dụng thư viện NumPy, nhằm giải quyết bài toán dự đoán bệnh tiểu đường trên tập dữ liệu Pima Indians Diabetes. Mục tiêu cụ thể của báo cáo bao gồm:

1. Trình bày ý tưởng cốt lõi của Deep Learning dưới dạng hợp thành các hàm số (function composition).
2. Cài đặt thủ công lan truyền tiến, hàm mất mát, lan truyền ngược và gradient descent bằng NumPy.
3. Huấn luyện mạng nơ-ron 3 lớp với kiến trúc $8 \rightarrow 16 \rightarrow 8 \rightarrow 1$ trên dữ liệu thực tế.
4. Đánh giá mô hình bằng các độ đo Accuracy, Precision, Recall, F1-score và phân tích kết quả thu được.

1.2. CƠ SỞ LÝ THUYẾT

1.2.1. Deep Learning là gì?

Ý tưởng trung tâm của Deep Learning là: thay vì học trực tiếp một ánh xạ duy nhất từ đầu vào đến đầu ra, mô hình học một chuỗi các phép biến đổi (representation) được ghép nối tiếp nhau. Với đầu vào X , ta xây dựng:

$$X \rightarrow f_1(X) \rightarrow f_2(f_1(X)) \rightarrow f_3(f_2(f_1(X)))$$

Do đó, đầu ra dự đoán $\hat{y}$ được biểu diễn dưới dạng hợp thành hàm số:

$$\hat{y} = f_3(f_2(f_1(X))) = (f_3 \circ f_2 \circ f_1)(X)$$

Mỗi hàm f_i tương ứng với một lớp (layer) của mạng nơ-ron, có tham số riêng được học từ dữ liệu thay vì được thiết kế thủ công.

1.2.2. So sánh Machine Learning truyền thống và Deep Learning

Với bài toán dự đoán tiểu đường, dữ liệu đầu vào của mỗi bệnh nhân gồm 8 đặc trưng: số lần mang thai, nồng độ glucose, huyết áp, độ dày nếp da, insulin, chỉ số khối cơ thể (BMI), chỉ số di truyền tiểu đường và tuổi.

Tiêu chí	Machine Learning truyền thống	Deep Learning
Quy trình	Dữ liệu → Trích chọn đặc trưng thủ công → Mô hình → Dự đoán	Dữ liệu → Biểu diễn được học tự động → Dự đoán
Vai trò con người	Thiết kế đặc trưng (feature engineering)	Thiết kế kiến trúc, mô hình tự học biểu diễn
Bản chất	Học một hàm dự đoán trên đặc trưng có sẵn	Học đồng thời biểu diễn và hàm dự đoán

1.2.3. Kiến trúc mạng nơ-ron sử dụng

Mạng nơ-ron được xây dựng trong bài toán này có kiến trúc 8 → 16 → 8 → 1, gồm ba lớp có tham số huấn luyện:

Lớp	Phép biến đổi	Kích thước	Hàm kích hoạt
Lớp 1 (ẩn)	$Z_1 = XW_1 + b_1$	8 → 16	ReLU
Lớp 2 (ẩn)	$Z_2 = H_1W_2 + b_2$	16 → 8	ReLU
Lớp 3 (đầu ra)	$Z_3 = H_2W_3 + b_3$	8 → 1	Sigmoid

Lớp 1 học các biểu diễn cấp thấp trực tiếp từ 8 đặc trưng gốc; Lớp 2 học biểu diễn cấp cao hơn từ chính các biểu diễn của Lớp 1 (representation learning); Lớp 3 ánh xạ biểu diễn cuối cùng sang một xác suất duy nhất trong khoảng (0, 1), thể hiện khả năng bệnh nhân mắc tiểu đường.

Hàm ReLU được sử dụng ở hai lớp ẩn để đưa tính phi tuyến vào mô hình:

$$ReLU(x) = \max(0, x)$$

Nếu không có hàm kích hoạt phi tuyến, việc chồng nhiều phép biến đổi tuyến tính liên tiếp về bản chất vẫn chỉ tương đương với một phép biến đổi tuyến tính duy nhất, khiến mạng không thể học được các quan hệ phức tạp trong dữ liệu.

Ở lớp đầu ra, hàm sigmoid được dùng để chuyển giá trị Z_3 về xác suất:

$$\sigma(z) = 1 / (1 + e^{-z}), \quad 0 < \sigma(z) < 1$$

1.3. DỮ LIỆU VÀ TIỀN XỬ LÝ

Bộ dữ liệu sử dụng là Pima Indians Diabetes Dataset, gồm 768 mẫu bệnh nhân, mỗi mẫu có 8 đặc trưng đầu vào và 1 nhãn nhị phân (0: không mắc tiểu đường, 1: mắc tiểu đường).

STT	Đặc trưng	Ý nghĩa
0	Pregnancies	Số lần mang thai
1	Glucose	Nồng độ glucose trong huyết tương
2	BloodPressure	Huyết áp
3	SkinThickness	Độ dày nếp gấp da
4	Insulin	Nồng độ insulin huyết thanh
5	BMI	Chỉ số khối cơ thể
6	DiabetesPedigreeFunction	Chỉ số di truyền tiểu đường
7	Age	Tuổi

1.3.1. Chia tập dữ liệu

Dữ liệu được chia ngẫu nhiên (np.random.seed(42)) theo tỉ lệ 80% huấn luyện – 20% kiểm thử, cho kết quả thực tế thu được là 614 mẫu huấn luyện và 154 mẫu kiểm thử.

1.3.2. Chuẩn hóa dữ liệu

Vì các đặc trưng có thang đo rất khác nhau (ví dụ glucose và tuổi), dữ liệu được chuẩn hóa theo công thức:

$$x' = (x - \mu) / \sigma$$

trong đó μ và σ được tính từ tập huấn luyện, sau đó áp dụng cho cả tập kiểm thử để tránh hiện tượng rò rỉ thông tin (data leakage) từ tập test sang quá trình huấn luyện.

1.4. PHƯƠNG PHÁP XÂY DỰNG MÔ HÌNH

1.4.1. Lan truyền tiến (Forward Propagation)

Lan truyền tiến là quá trình tính toán đầu ra dự đoán từ dữ liệu đầu vào, đi qua lần lượt các lớp của mạng:

$$Z_1 = XW_1 + b_1 ; H_1 = ReLU(Z_1)$$

$$Z_2 = H_1W_2 + b_2 ; H_2 = ReLU(Z_2)$$

$$Z_3 = H_2W_3 + b_3 ; \hat{y} = \sigma(Z_3)$$

Toàn bộ mô hình được viết gọn lại thành một phương trình hợp thành hàm số duy nhất:

$$\hat{y} = \sigma(ReLU(ReLU(XW_1 + b_1) \cdot W_2 + b_2) \cdot W_3 + b_3)$$

1.4.2. Hàm mất mát (Binary Cross-Entropy)

Vì đây là bài toán phân loại nhị phân, hàm mất mát Binary Cross-Entropy được sử dụng để đo mức độ sai lệch giữa dự đoán và nhãn thật:

$$L = -(1/N) \sum_i [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

Giá trị loss càng nhỏ nghĩa là mô hình dự đoán càng gần với nhãn thực tế. Một hằng số nhỏ $\epsilon = 1e-8$ được thêm vào để tránh phép toán $\log(0)$ gây lỗi số học.

1.4.3. Lan truyền ngược (Backpropagation)

Backpropagation áp dụng quy tắc chuỗi (chain rule) để tính đạo hàm của hàm mất mát theo từng tham số, cho biết mỗi trọng số cần thay đổi theo hướng nào để giảm loss. Nhờ tính chất thuận lợi khi kết hợp sigmoid với binary cross-entropy, đạo hàm tại lớp ra có dạng đơn giản:

$$\partial L / \partial Z_3 = \hat{y} - y$$

Từ đó, gradient được lan truyền ngược qua các lớp:

$$dW_3 = H_2^T \cdot dZ_3 ; dH_2 = dZ_3 \cdot W_3^T ; dZ_2 = dH_2 \odot ReLU'(Z_2)$$

$$dW_2 = H_1^T \cdot dZ_2 ; dH_1 = dZ_2 \cdot W_2^T ; dZ_1 = dH_1 \odot ReLU'(Z_1) ; dW_1 = X^T \cdot dZ_1$$

Có thể thấy backpropagation không phải là một kỹ thuật huyền bí, mà chỉ là việc áp dụng có hệ thống quy tắc chuỗi lên một chuỗi các hàm số hợp thành.

1.4.4. Cập nhật trọng số (Gradient Descent)

Sau khi có gradient, các tham số được cập nhật theo hướng ngược với gradient, với tốc độ học (learning rate) η :

$$W \leftarrow W - \eta \cdot (\partial L / \partial W)$$

Chu trình huấn luyện hoàn chỉnh được lặp lại nhiều lần theo trình tự:

$$Forward \rightarrow Loss \rightarrow Backward \rightarrow Update$$

1.5. CÀI ĐẶT VÀ HUẤN LUYỆN MÔ HÌNH

Mô hình được cài đặt bằng Python thuần với NumPy, không sử dụng bất kỳ framework Deep Learning nào (TensorFlow, PyTorch, Keras, scikit-learn). Các thành phần chính đã cài đặt bao gồm:

- Khởi tạo trọng số ngẫu nhiên theo phương pháp He initialization: $W \sim N(0,1) \times \sqrt{(2/n_{in})}$.
- Hàm kích hoạt ReLU, sigmoid và đạo hàm tương ứng.
- Hàm forward(X) thực hiện lan truyền tiến qua 3 lớp và lưu lại cache ($Z_1, H_1, Z_2, H_2, Z_3, \hat{y}$) phục vụ cho backpropagation.
- Hàm binary_cross_entropy(y, y_hat) tính giá trị hàm mất mát.
- Hàm backward(y, cache) tính gradient cho toàn bộ 6 tham số $\{W_1, b_1, W_2, b_2, W_3, b_3\}$.

- Vòng lặp huấn luyện (training loop) với learning rate $\eta = 0.01$ và 1000 epoch, cập nhật tham số bằng gradient descent sau mỗi epoch.

Toàn bộ quá trình huấn luyện tuân theo đúng bốn bước: Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update, được lặp lại 1000 lần liên tiếp trên tập huấn luyện (614 mẫu).

1.6. PHÂN TÍCH CHI TIẾT TỪNG CELL CODE TRONG JUPYTER NOTEBOOK

Chương này trình bày lại toàn bộ notebook `Deeplearning_numpy_diabets.ipynb` theo đúng thứ tự các cell đã chạy, kèm theo giải thích ý nghĩa của từng dòng lệnh và kết quả (output) thực tế thu được khi thực thi. Mục đích là giúp người đọc theo dõi được mạch tư duy lập trình từ bước nạp dữ liệu cho đến bước đánh giá mô hình cuối cùng.

1.6.1. Cell 1 — Nạp thư viện NumPy

```
import numpy as np
```

Đây là thư viện duy nhất được sử dụng trong toàn bộ notebook. NumPy cung cấp cấu trúc mảng đa chiều (ndarray) và các phép toán đại số tuyến tính (nhân ma trận, cộng, broadcasting...) với tốc độ cao, đủ để tự cài đặt toàn bộ một mạng nơ-ron mà không cần đến TensorFlow/PyTorch/Keras.

1.6.2. Cell 2 — Đọc dữ liệu từ file CSV

```
# Đọc dữ liệu từ file CSV (bỏ qua dòng header)
data = np.loadtxt("C:/Users/Admin/Downloads/Test/diabetes.csv", delimiter=";", skiprows=1)

# Tách features (8 cột đầu) và labels (cột cuối)
X = data[:, :8]
y = data[:, 8].reshape(-1, 1)

print("X shape:", X.shape) # Kỳ vọng: (768, 8)
print("y shape:", y.shape) # Kỳ vọng: (768, 1)
```

Output:

```
X shape: (768, 8)
y shape: (768, 1)
```

`np.loadtxt()` đọc toàn bộ file CSV thành một mảng NumPy 2 chiều, tham số `delimiter=";"` cho biết các giá trị cách nhau bởi dấu phẩy, còn `skiprows=1` bỏ qua dòng tiêu đề (header) của file. Biến `data` khi đó có 768 hàng (768 bệnh nhân) và 9 cột (8 đặc trưng + 1 nhãn).

Lệnh `data[:, :8]` dùng kỹ thuật slicing của NumPy để lấy tất cả các hàng và 8 cột đầu tiên (chỉ số 0 đến 7), tương ứng với 8 đặc trưng đầu vào `X`. Lệnh `data[:, 8]` lấy cột thứ 9 (chỉ số 8) là nhãn `y`; hàm

.reshape(-1, 1) chuyển vector 1 chiều (768,) thành ma trận cột (768, 1) — hình dạng bắt buộc để thực hiện đúng các phép nhân ma trận và tính loss ở các bước sau.

1.6.3. Cell 3 — Chia tập huấn luyện và kiểm thử

```
np.random.seed(42) # Đặt seed để kết quả tái lập

# Tạo chỉ số ngẫu nhiên
indices = np.random.permutation(len(X))

# Tỷ lệ train:test = 80:20
split = int(0.8 * len(X))

train_idx = indices[:split]
test_idx = indices[split:]

X_train = X[train_idx]
y_train = y[train_idx]
X_test = X[test_idx]
y_test = y[test_idx]

print("Training samples:", len(X_train)) # ~614
print("Testing samples:", len(X_test)) # ~154
```

Output:

```
Training samples: 614
Testing samples: 154
```

np.random.seed(42) cố định bộ sinh số ngẫu nhiên để mỗi lần chạy lại notebook đều cho ra cùng một cách chia dữ liệu và cùng một bộ trọng số khởi tạo — điều kiện cần thiết để kết quả có thể tái lập (reproducible).

np.random.permutation(len(X)) tạo ra một hoán vị ngẫu nhiên của các chỉ số từ 0 đến 767, giúp trộn ngẫu nhiên thứ tự bệnh nhân trước khi chia tập, tránh trường hợp dữ liệu vốn được sắp xếp theo một trật tự nào đó (ví dụ theo thời gian thu thập) làm sai lệch kết quả huấn luyện. split = int(0.8*768) = 614 xác định điểm cắt 80%. 614 chỉ số đầu tiên trong mảng đã xáo trộn được dùng làm tập huấn luyện (train_idx), 154 chỉ số còn lại làm tập kiểm thử (test_idx). Cuối cùng, kỹ thuật fancy indexing X[train_idx] trích các hàng tương ứng ra thành X_train, y_train, X_test, y_test.

1.6.4. Cell 4 — Chuẩn hóa dữ liệu (Standardization)

```
# Tính mean và std từ TRAIN set
mean = X_train.mean(axis=0)
std = X_train.std(axis=0) + 1e-8 # +1e-8 để tránh chia cho 0

# Chuẩn hóa
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std # Dùng mean/std của TRAIN
```

`X_train.mean(axis=0)` tính giá trị trung bình theo từng cột (`axis=0` nghĩa là tính dọc theo các hàng, cho ra 1 giá trị trung bình cho mỗi đặc trưng trong 8 đặc trưng). Tương tự, `.std(axis=0)` tính độ lệch chuẩn theo cột. Hằng số nhỏ $1e-8$ được cộng thêm vào `std` để đảm bảo không xảy ra phép chia cho 0 nếu một đặc trưng nào đó có độ lệch chuẩn bằng 0.

Phép $(X_train - \text{mean}) / \text{std}$ tận dụng cơ chế broadcasting của NumPy: một mảng (614, 8) trừ/chia cho một vector (8,) sẽ tự động được áp dụng cho từng cột tương ứng, cho ra kết quả mỗi đặc trưng có trung bình xấp xỉ 0 và độ lệch chuẩn xấp xỉ 1. Điểm quan trọng cần lưu ý: `X_test` cũng được chuẩn hóa bằng đúng `mean` và `std` đã tính từ `X_train` (không tính lại từ `X_test`), nhằm mô phỏng đúng tình huống thực tế là mô hình chỉ được biết thống kê của dữ liệu huấn luyện, tránh rò rỉ thông tin (data leakage) từ tập kiểm thử.

1.6.5. Cell 5 — Định nghĩa hàm kích hoạt ReLU và Sigmoid

```
# ReLU và đạo hàm
def relu(x):
    return np.maximum(0, x)

def relu_derivative(x):
    return (x > 0).astype(float)

# Sigmoid và đạo hàm
def sigmoid(x):
    x = np.clip(x, -50, 50) # Tránh overflow
    return 1.0 / (1.0 + np.exp(-x))

def sigmoid_derivative(a):
    return a * (1.0 - a)
```

Bốn hàm này là nền tảng toán học của mạng nơ-ron; ý nghĩa chi tiết của từng hàm được trình bày đầy đủ trong Chương 7 — Giải thích chi tiết các hàm.

1.6.6. Cell 6 — Khởi tạo trọng số và bias của mạng

```
# Architecture: 8 -> 16 -> 8 -> 1
W1 = np.random.randn(8, 16) * np.sqrt(2.0 / 8)
b1 = np.zeros((1, 16))

W2 = np.random.randn(16, 8) * np.sqrt(2.0 / 16)
b2 = np.zeros((1, 8))

W3 = np.random.randn(8, 1) * np.sqrt(2.0 / 8)
b3 = np.zeros((1, 1))
```

`np.random.randn(m, n)` sinh ra một ma trận kích thước (m, n) với các phần tử tuân theo phân phối chuẩn $N(0, 1)$. Nhân thêm hệ số $\sqrt{2/n_{in}}$ (với `n_in` là số nơ-ron đầu vào của lớp đó) chính là kỹ thuật khởi tạo He initialization — phương pháp khởi tạo phổ biến khi dùng hàm kích hoạt ReLU, giúp giữ phương sai của tín hiệu ổn định khi đi qua nhiều lớp, tránh hiện tượng gradient bị triệt tiêu (vanishing) hoặc bùng nổ (exploding) ngay từ đầu quá trình huấn luyện.

Các ma trận trọng số có kích thước tương ứng với kiến trúc $8 \rightarrow 16 \rightarrow 8 \rightarrow 1$: W_1 (8×16) biến đổi 8 đặc trưng thành 16 nơ-ron ẩn; W_2 (16×8) biến đổi 16 nơ-ron thành 8 nơ-ron; W_3 (8×1) biến đổi 8 nơ-ron thành 1 giá trị đầu ra. Các vector bias (b_1, b_2, b_3) được khởi tạo bằng 0 — đây là quy ước phổ biến vì việc phá vỡ tính đối xứng giữa các nơ-ron đã được đảm nhiệm bởi trọng số W được khởi tạo ngẫu nhiên.

1.6.7. Cell 7 — Hàm lan truyền tiến forward(X)

```
def forward(X):
    # Layer 1
    z1 = X @ W1 + b1
    h1 = relu(z1)

    # Layer 2
    z2 = h1 @ W2 + b2
    h2 = relu(z2)

    # Layer 3 (output)
    z3 = h2 @ W3 + b3
    y_hat = sigmoid(z3)

    cache = {
        "x": X, "z1": z1, "h1": h1,
        "z2": z2, "h2": h2,
        "z3": z3, "y_hat": y_hat
    }
    return y_hat, cache
```

Toán tử `@` trong NumPy thực hiện phép nhân ma trận. Hàm `forward` thực hiện tuần tự ba phép biến đổi tuyến tính xen kẽ với hàm kích hoạt, đúng như công thức toán học ở Chương 4.1. Giải thích chi tiết dòng lệnh và ý nghĩa của biến `cache` được trình bày trong Chương 7.

1.6.8. Cell 8 — Hàm mất mát `binary_cross_entropy(y, y_hat)`

```
def binary_cross_entropy(y, y_hat):
    eps = 1e-8
    y_hat = np.clip(y_hat, eps, 1 - eps) # Tránh log(0)

    loss = -np.mean(
        y * np.log(y_hat) +
        (1 - y) * np.log(1 - y_hat)
    )
    return loss
```

Hàm này tính giá trị trung bình của hàm mất mát binary cross-entropy trên toàn bộ tập dữ liệu truyền vào. Giải thích chi tiết ở Chương 7.

1.6.9. Cell 9 — Hàm lan truyền ngược `backward(y, cache)`

```

def backward(y, cache):
    X = cache["x"]
    z1, h1 = cache["z1"], cache["h1"]
    z2, h2 = cache["z2"], cache["h2"]
    yhat = cache["y_hat"]
    n = len(X)

    # === LAYER 3 ===
    dz3 = (yhat - y) / n
    dW3 = h2.T @ dz3
    db3 = np.sum(dz3, axis=0, keepdims=True)

    # === LAYER 2 ===
    dh2 = dz3 @ W3.T
    dz2 = dh2 * relu_derivative(z2)
    dW2 = h1.T @ dz2
    db2 = np.sum(dz2, axis=0, keepdims=True)

    # === LAYER 1 ===
    dh1 = dz2 @ W2.T
    dz1 = dh1 * relu_derivative(z1)
    dW1 = X.T @ dz1
    db1 = np.sum(dz1, axis=0, keepdims=True)

    gradients = {
        "dW1": dW1, "db1": db1,
        "dW2": dW2, "db2": db2,
        "dW3": dW3, "db3": db3
    }
    return gradients

```

Hàm backward thực hiện đúng các công thức đạo hàm ngược đã trình bày ở Chương 4.3, lần lượt tính gradient cho Lớp 3, Lớp 2 rồi Lớp 1 — đi ngược chiều với lan truyền tiến. Giải thích chi tiết từng dòng được trình bày ở Chương 7.

1.6.10. Cell 10 — Vòng lặp huấn luyện (Training loop)

```

learning_rate = 0.01
epochs = 1000
loss_history = []

print("🚀 Bắt đầu training...")
print("-" * 50)

for epoch in range(epochs):
    # Forward
    y_hat, cache = forward(X_train)

    # Loss
    loss = binary_cross_entropy(y_train, y_hat)
    loss_history.append(loss)

    # Backward - gradients được tạo ở đây
    gradients = backward(y_train, cache)

    # Update - gradients tồn tại ngay sau backward
    W1 -= learning_rate * gradients["dW1"]
    b1 -= learning_rate * gradients["db1"]
    W2 -= learning_rate * gradients["dW2"]
    b2 -= learning_rate * gradients["db2"]
    W3 -= learning_rate * gradients["dW3"]
    b3 -= learning_rate * gradients["db3"]

    # Print progress
    if epoch % 100 == 0:
        print(f"Epoch {epoch:4d} | Loss = {loss:.4f}")

print("-" * 50)
print(f"✅ Training hoàn tất! Final Loss: {loss_history[-1]:.4f}")

```

Output:

```

🚀 Bắt đầu training...
-----
Epoch   0 | Loss = 0.7753
Epoch 100 | Loss = 0.6353
Epoch 200 | Loss = 0.6002
Epoch 300 | Loss = 0.5780
Epoch 400 | Loss = 0.5599
Epoch 500 | Loss = 0.5446
Epoch 600 | Loss = 0.5308
Epoch 700 | Loss = 0.5187
Epoch 800 | Loss = 0.5085
Epoch 900 | Loss = 0.5000
-----
✅ Training hoàn tất! Final Loss: 0.4929

```

Đây là cell quan trọng nhất, hiện thực hóa đúng chu trình bốn bước Forward → Loss → Backward → Update lặp lại 1000 lần (epochs = 1000) trên toàn bộ tập huấn luyện cùng lúc (không chia batch

nhỏ — đây gọi là Batch Gradient Descent). Biến `loss_history` là một list rỗng ban đầu, được dùng để lưu lại giá trị loss của từng epoch nhằm phục vụ việc vẽ biểu đồ ở cell tiếp theo.

Trong mỗi vòng lặp: (1) `forward(X_train)` tính dự đoán và lưu cache; (2) `binary_cross_entropy` tính loss hiện tại và loss này được append vào `loss_history`; (3) `backward(y_train, cache)` tính toàn bộ 6 gradient dựa trên cache vừa lưu; (4) sáu dòng `W -= learning_rate * gradients[...]` chính là công thức gradient descent $W \leftarrow W - \eta \cdot \partial L / \partial W$, được áp dụng cho cả 3 ma trận trọng số và 3 vector bias. Điều kiện `if epoch % 100 == 0` chỉ in log tiến trình huấn luyện sau mỗi 100 epoch để theo dõi mà không làm ngợp màn hình output.

1.6.11. Cell 11 — Nạp thư viện Matplotlib

```
import matplotlib.pyplot as plt
```

Matplotlib chỉ được dùng cho mục đích trực quan hóa (vẽ biểu đồ), hoàn toàn tách biệt khỏi phần tính toán của mạng nơ-ron — mạng nơ-ron vẫn chỉ được cài đặt bằng NumPy.

1.6.12. Cell 12 — Vẽ biểu đồ đường cong Loss

```
plt.figure(figsize=(10, 6))
plt.plot(loss_history)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss Curve")
plt.grid(True)
plt.show()
```

Cell này vẽ đồ thị thể hiện sự thay đổi của giá trị loss (trục tung) theo từng epoch huấn luyện (trục hoành), dựa trên toàn bộ 1000 giá trị đã lưu trong `loss_history`. Đường cong thu được có dạng giảm dần và thoải dần ở cuối — dấu hiệu điển hình của một quá trình huấn luyện hội tụ ổn định, khớp với các giá trị loss rời rạc đã quan sát ở Cell 10 (0.7753 → 0.4929).

1.6.13. Cell 13 — Dự đoán và tính các độ đo đánh giá trên tập kiểm thử

```

# Dự đoán trên test set
y_prob, _ = forward(X_test)
y_pred = (y_prob >= 0.5).astype(int)

# Confusion Matrix
TP = np.sum((y_pred == 1) & (y_test == 1))
TN = np.sum((y_pred == 0) & (y_test == 0))
FP = np.sum((y_pred == 1) & (y_test == 0))
FN = np.sum((y_pred == 0) & (y_test == 1))

# Metrics
accuracy = np.mean(y_pred == y_test)
precision = TP / (TP + FP + 1e-8)
recall = TP / (TP + FN + 1e-8)
f1 = 2 * precision * recall / (precision + recall + 1e-8)

print("=" * 50)
print("📊 KẾT QUẢ TRÊN TEST SET")
print("=" * 50)
print(f"✅ Accuracy: {accuracy:.4f}")
print(f"🎯 Precision: {precision:.4f}")
print(f"📈 Recall: {recall:.4f}")
print(f"🔄 F1-Score: {f1:.4f}")
print("-" * 50)
print("Confusion Matrix:")
print(f"TP: {TP:3d} FP: {FP:3d}")
print(f"FN: {FN:3d} TN: {TN:3d}")

```

Output:

```

=====
📊 KẾT QUẢ TRÊN TEST SET
=====

```

```

✅ Accuracy: 0.7273
🎯 Precision: 0.7500
📈 Recall:      0.4138
🔄 F1-Score: 0.5333

```

```

-----
Confusion Matrix:

```

```

TP: 24 FP: 8
FN: 34 TN: 88

```

`forward(X_test)` chạy lan truyền tiến trên 154 mẫu kiểm thử (dấu gạch dưới `_` dùng để bỏ qua giá trị cache trả về thứ hai vì ở bước dự đoán không cần tính gradient). `y_prob` là ma trận (154, 1) chứa xác suất dự đoán mắc tiểu đường. Lệnh `(y_prob >= 0.5).astype(int)` áp dụng ngưỡng quyết định 0.5: xác suất lớn hơn hoặc bằng 0.5 được gán nhãn 1 (có bệnh), ngược lại gán nhãn 0.

Bốn dòng tính TP, TN, FP, FN sử dụng toán tử logic `&` (AND theo từng phần tử) kết hợp với so sánh mảng, cho ra các mảng boolean rồi `np.sum()` đếm số lượng phần tử True. Từ bốn giá trị nền tảng này, `accuracy`, `precision`, `recall`, `f1` được tính đúng theo công thức đã nêu ở Chương 4 và Chương 8

của báo cáo. Hằng số 1e-8 ở mẫu số của precision, recall, f1 nhằm tránh lỗi chia cho 0 trong trường hợp mô hình không dự đoán dương tính nào hoặc không có ca dương tính thật nào.

1.6.14. Cell 14 — In ví dụ 10 dự đoán đầu tiên

```
# Show some predictions
print("\n📄 Ví dụ dự đoán (10 mẫu đầu tiên trong test set):")
print("-" * 70)
print(f'{"STT":<6} {"Probability":<15} {"Prediction":<15} {"Actual":<10}")
print("-" * 70)

for i in range(min(10, len(X_test))):
    prob = y_prob[i, 0]
    pred = y_pred[i, 0]
    actual = int(y_test[i, 0])
    print(f'{i+1:<6} {prob:.4f}          {pred:<15} {actual:<10}')
```

Cell cuối cùng minh họa trực quan kết quả dự đoán ở cấp độ từng bệnh nhân, giúp người đọc thấy được mối liên hệ giữa xác suất đầu ra (y_{prob}), nhãn dự đoán sau ngưỡng (y_{pred}) và nhãn thực tế (y_{test}). $\min(10, \text{len}(X_{\text{test}}))$ đảm bảo vòng lặp không bị lỗi nếu tập kiểm thử có ít hơn 10 mẫu. Định dạng chuỗi f-string với $\{i+1:<6\}$ và tương tự dùng để căn trái, tạo bảng in ra màn hình dễ đọc — kết quả cụ thể của 10 dòng này đã được trình bày dưới dạng bảng ở mục 8.4 của báo cáo.

1.7. GIẢI THÍCH CHI TIẾT CÁC HÀM TRONG CHƯƠNG TRÌNH

Chương này đi sâu giải thích ý nghĩa toán học, tham số đầu vào/đầu ra, và vai trò của từng hàm đã được định nghĩa trong notebook, giúp người đọc hiểu rõ "vì sao" mỗi dòng lệnh được viết như vậy chứ không chỉ dừng ở việc mô tả "nó làm gì".

1.7.1. Hàm $\text{relu}(x)$

```
def relu(x):
    return np.maximum(0, x)
```

Thành phần	Giải thích
Đầu vào	x — mảng NumPy bất kỳ chiều nào (ở đây là ma trận $Z1$ hoặc $Z2$, kích thước $N \times 16$ hoặc $N \times 8$)
Đầu ra	Mảng cùng kích thước với x , trong đó mọi giá trị âm được thay bằng 0
Công thức toán	$\text{ReLU}(x) = \max(0, x)$
$\text{np.maximum}(0, x)$	So sánh từng phần tử của x với 0 và giữ lại giá trị lớn hơn — thực hiện toàn bộ phép so sánh trên cả mảng cùng lúc (vectorized), nhanh hơn nhiều so với dùng vòng lặp for

Vai trò trong mạng	Đưa tính phi tuyến vào Lớp 1 và Lớp 2, cho phép mạng học được các quan hệ phức tạp hơn một phép biến đổi tuyến tính đơn thuần
--------------------	---

1.7.2. Hàm `relu_derivative(x)`

<pre>def relu_derivative(x): return (x > 0).astype(float)</pre>	
Thành phần	Giải thích
Đầu vào	x — chính là giá trị Z (trước khi qua ReLU) đã được lưu lại trong cache lúc forward
$x > 0$	Trả về mảng boolean (True/False) cùng kích thước, True tại vị trí Z dương
<code>.astype(float)</code>	Chuyển True/False thành 1.0/0.0 để có thể nhân trực tiếp với gradient phía sau
Công thức toán	$\text{ReLU}'(x) = 1$ nếu $x > 0$, ngược lại $= 0$
Vai trò trong mạng	Dùng trong <code>backward()</code> để "chặn" gradient tại các nơ-ron đã bị ReLU triệt tiêu ($Z \leq 0$) — nếu ở forward nơ-ron đó cho ra 0 thì ở backward gradient qua nó cũng bằng 0, đúng theo quy tắc đạo hàm

1.7.3. Hàm `sigmoid(x)`

<pre>def sigmoid(x): x = np.clip(x, -50, 50) # Tránh overflow return 1.0 / (1.0 + np.exp(-x))</pre>	
Thành phần	Giải thích
Đầu vào	x — chính là Z_3 , kết quả tuyến tính của lớp đầu ra, kích thước $(N, 1)$
<code>np.clip(x, -50, 50)</code>	Giới hạn giá trị x trong đoạn $[-50, 50]$ trước khi tính e^{-x} , nhằm tránh lỗi tràn số (overflow) khi x có trị tuyệt đối quá lớn — vì với $ x > 50$, <code>np.exp()</code> có thể gây ra giá trị vô cực hoặc lỗi số học, trong khi kết quả sigmoid thực tế đã xấp xỉ 0 hoặc 1 nên việc cắt giá trị này không ảnh hưởng đến độ chính xác
Công thức toán	$\sigma(x) = 1 / (1 + e^{-x})$
Miền giá trị đầu ra	$(0, 1)$ — có thể diễn giải như một xác suất
Vai trò trong mạng	Chuyển giá trị Z_3 (có thể là số thực bất kỳ) thành xác suất dự đoán mắc tiểu đường ý, phù hợp với bài toán phân loại nhị phân

1.7.4. Hàm sigmoid_derivative(a)

```
def sigmoid_derivative(a):  
    return a * (1.0 - a)
```

Hàm này nhận vào a — chính là giá trị sigmoid đã tính sẵn (không phải x gốc) — và trả về đạo hàm $\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x)) = a \cdot (1 - a)$. Việc truyền vào giá trị a (đã qua sigmoid) thay vì x giúp tránh phải tính lại hàm mũ e^{-x} một lần nữa, tiết kiệm chi phí tính toán. Đáng chú ý, trong notebook thực tế hàm này được định nghĩa nhưng không được gọi trực tiếp trong `backward()`, vì khi kết hợp sigmoid với hàm mất mát binary cross-entropy, đạo hàm tại lớp ra rút gọn trực tiếp thành $dz3 = (yhat - y) / n$ (xem giải thích ở mục 7.7), nên không cần tính `sigmoid_derivative` một cách tường minh.

1.7.5. Hàm forward(X)

```
def forward(X):  
    z1 = X @ W1 + b1  
    h1 = relu(z1)  
    z2 = h1 @ W2 + b2  
    h2 = relu(z2)  
    z3 = h2 @ W3 + b3  
    y_hat = sigmoid(z3)  
    cache = {  
        "x": X, "z1": z1, "h1": h1,  
        "z2": z2, "h2": h2,  
        "z3": z3, "y_hat": y_hat  
    }  
    return y_hat, cache
```

Hàm `forward` nhận vào ma trận đặc trưng X (kích thước $N \times 8$, với N là số mẫu) và trả về hai giá trị: y_hat là ma trận xác suất dự đoán ($N \times 1$), và `cache` là một dictionary lưu lại toàn bộ giá trị trung gian đã tính được trong lượt lan truyền tiến này.

Dòng lệnh	Ý nghĩa
$z1 = X @ W1 + b1$	Phép biến đổi tuyến tính đầu tiên: nhân ma trận X ($N \times 8$) với $W1$ (8×16) rồi cộng bias $b1$ (broadcast trên N hàng), cho ra $z1$ ($N \times 16$)
$h1 = \text{relu}(z1)$	Áp dụng phi tuyến ReLU lên $z1$, cho ra biểu diễn ẩn thứ nhất $h1$
$z2 = h1 @ W2 + b2$	Biến đổi tuyến tính thứ hai, nhận đầu vào là $h1$ (không phải X gốc) — đây chính là ý tưởng "học từ biểu diễn đã học"
$h2 = \text{relu}(z2)$	Biểu diễn ẩn thứ hai, kích thước $N \times 8$
$z3 = h2 @ W3 + b3$	Biến đổi tuyến tính cuối cùng, cho ra 1 giá trị mỗi mẫu ($N \times 1$)
$y_hat = \text{sigmoid}(z3)$	Chuyển $z3$ thành xác suất dự đoán trong khoảng (0, 1)
$cache = \{...\}$	Lưu lại tất cả $z1, h1, z2, h2, z3, y_hat$ và cả X gốc — các giá trị này bắt buộc phải có khi tính đạo hàm ở hàm <code>backward()</code> , vì đạo hàm của ReLU cần biết dấu của z , và đạo hàm của trọng số cần nhân với giá trị đầu vào tương ứng của lớp đó

1.7.6. Hàm `binary_cross_entropy(y, y_hat)`

```
def binary_cross_entropy(y, y_hat):
    eps = 1e-8
    y_hat = np.clip(y_hat, eps, 1 - eps)
    loss = -np.mean(y * np.log(y_hat) + (1 - y) * np.log(1 - y_hat))
    return loss
```

Dòng lệnh	Ý nghĩa
<code>eps = 1e-8</code>	Một số rất nhỏ dùng làm "vùng đệm an toàn"
<code>np.clip(y_hat, eps, 1 - eps)</code>	Ép mọi giá trị <code>y_hat</code> nằm trong khoảng $[1e-8, 1 - 1e-8]$, tránh trường hợp <code>y_hat</code> đúng bằng 0 hoặc 1 khiến <code>np.log(0)</code> trả về $-\infty$ (lỗi số học), gây NaN lan truyền vào toàn bộ quá trình huấn luyện
<code>y * np.log(y_hat)</code>	Thành phần loss tương ứng với các mẫu có nhãn thật <code>y = 1</code> : chỉ có giá trị khi <code>y = 1</code> , phạt mô hình nếu <code>y_hat</code> gần 0
<code>(1 - y) * np.log(1 - y_hat)</code>	Thành phần loss tương ứng với các mẫu có nhãn thật <code>y = 0</code> : chỉ có giá trị khi <code>y = 0</code> , phạt mô hình nếu <code>y_hat</code> gần 1
<code>np.mean(...)</code>	Tính trung bình cộng loss trên toàn bộ <code>N</code> mẫu, cho ra một số vô hướng (scalar) duy nhất đại diện cho "mức độ sai" của cả mô hình ở thời điểm hiện tại
Dấu trừ (-)	Vì log của một số trong (0,1) luôn âm, dấu trừ đảm bảo loss luôn là một số dương, giá trị càng nhỏ càng tốt

1.7.7. Hàm `backward(y, cache)`

```
def backward(y, cache):
    X = cache["x"]; z1, h1 = cache["z1"], cache["h1"]
    z2, h2 = cache["z2"], cache["h2"]; yhat = cache["y_hat"]
    n = len(X)

    dz3 = (yhat - y) / n
    dW3 = h2.T @ dz3
    db3 = np.sum(dz3, axis=0, keepdims=True)

    dh2 = dz3 @ W3.T
    dz2 = dh2 * relu_derivative(z2)
    dW2 = h1.T @ dz2
    db2 = np.sum(dz2, axis=0, keepdims=True)

    dh1 = dz2 @ W2.T
    dz1 = dh1 * relu_derivative(z1)
    dW1 = X.T @ dz1
```

```
db1 = np.sum(dz1, axis=0, keepdims=True)
...
```

Đây là hàm phức tạp nhất trong notebook. Hàm nhận vào nhãn thật y và cache đã lưu từ forward(), trả về một dictionary gradients chứa đạo hàm của loss theo cả 6 tham số. Bảng dưới đây giải thích từng nhóm dòng lệnh theo đúng thứ tự lan truyền ngược, từ Lớp 3 về Lớp 1.

Dòng lệnh	Công thức toán tương ứng	Giải thích
$n = \text{len}(X)$	N (số mẫu)	Số lượng mẫu trong batch, dùng để chia trung bình gradient — giữ cho độ lớn gradient không phụ thuộc vào kích thước batch
$dz3 = (\hat{y} - y) / n$	$\partial L / \partial Z_3 = (\hat{y} - y) / N$	Nhờ tính chất đặc biệt khi kết hợp sigmoid với binary cross-entropy, đạo hàm của loss theo Z_3 rút gọn cực kỳ đơn giản thành hiệu giữa dự đoán và nhãn thật, chia cho N (đây chính là lý do sigmoid_derivative() không cần dùng trực tiếp ở đây)
$dW3 = h2.T @ dz3$	$dW_3 = H_2^T \cdot dZ_3$	$H2.T$ chuyển vị ma trận $H2$ từ $(N \times 8)$ thành $(8 \times N)$, nhân với $dz3$ $(N \times 1)$ cho ra $dW3$ đúng kích thước (8×1) như $W3$
$db3 = \text{np.sum}(dz3, \text{axis}=0, \text{keepdims}=\text{True})$	$db_3 = \sum_i dZ_3$	Cộng gradient theo tất cả N mẫu vì bias được dùng chung (broadcast) cho mọi mẫu; keepdims=True giữ kích thước $(1,1)$ thay vì bị rút gọn thành số vô hướng
$dh2 = dz3 @ W3.T$	$dH_2 = dZ_3 \cdot W_3^T$	Lan truyền gradient ngược từ Z_3 về $H2$ thông qua trọng số $W3$
$dz2 = dh2 * \text{relu_derivative}(z2)$	$dZ_2 = dH_2 \odot \text{ReLU}'(Z_2)$	Nhân từng phần tử $(\odot)$ với đạo hàm ReLU tại $z2$ đã lưu trong cache — "khóa" gradient tại các nơ-ron đã bị ReLU triệt tiêu ở lượt forward
$dW2 = h1.T @ dz2$; $db2 = \text{np.sum}(dz2, \text{axis}=0, \text{keepdims}=\text{True})$	$dW_2 = H_1^T \cdot dZ_2$; $db_2 = \sum_i dZ_2$	Lặp lại đúng logic của Lớp 3 nhưng áp dụng cho Lớp 2, dùng $h1$ (đầu vào của lớp 2) thay vì $h2$
$dh1 = dz2 @ W2.T$; $dz1 = dh1 * \text{relu_derivative}(z1)$	$dH_1 = dZ_2 \cdot W_2^T$; $dZ_1 = dH_1 \odot \text{ReLU}'(Z_1)$	Tiếp tục lan truyền ngược từ Lớp 2 về Lớp 1
$dW1 = X.T @ dz1$; $db1 = \text{np.sum}(dz1, \text{axis}=0, \text{keepdims}=\text{True})$	$dW_1 = X^T \cdot dZ_1$; $db_1 = \sum_i dZ_1$	Ở lớp cuối cùng của quá trình lan truyền ngược, gradient được nhân với chính dữ liệu đầu vào gốc X (đã lưu trong cache) thay vì một biểu diễn ẩn

Có thể nhận thấy cấu trúc lặp lại rất rõ ràng giữa ba lớp: mỗi lớp đều gồm ba bước — (1) tính dW và db của lớp hiện tại dựa trên dZ của lớp đó và đầu vào của lớp đó, (2) lan truyền gradient dH sang lớp trước thông qua ma trận trọng số W hiện tại, (3) nhân với đạo hàm hàm kích hoạt để có dZ của lớp trước. Đây chính là minh chứng thực nghiệm cho quy tắc chuỗi (chain rule) đã trình bày ở mục 4.3: gradient của lớp trước luôn được tính dựa trên gradient đã có của lớp sau.

1.8. KẾT QUẢ THỰC NGHIỆM

1.8.1. Quá trình giảm Loss trong huấn luyện

Giá trị hàm mất mát (Binary Cross-Entropy Loss) trên tập huấn luyện được ghi nhận sau mỗi 100 epoch như sau:

Epoch	0	100	200	300	400	500	600	700	800	900	1000 (cuối)
Loss	0.7753	0.6353	0.6002	0.5780	0.5599	0.5446	0.5308	0.5187	0.5085	0.5000	0.4929

Có thể quan sát rõ xu hướng loss giảm dần và ổn định qua các epoch, từ 0.7753 ở epoch đầu tiên xuống còn 0.4929 sau 1000 epoch, cho thấy mô hình đang học được các quy luật trong dữ liệu huấn luyện. Tuy nhiên tốc độ giảm loss chậm dần ở các epoch cuối, cho thấy mô hình có dấu hiệu tiệm cận điểm hội tụ với tốc độ học và số epoch hiện tại.

1.8.2. Đánh giá trên tập kiểm thử (Test set, 154 mẫu)

Độ đo	Giá trị	Ý nghĩa
Accuracy	0.7273 (72,73%)	Tỉ lệ dự đoán đúng trên tổng số mẫu kiểm thử
Precision	0.7500 (75,00%)	Trong số các ca được dự đoán mắc tiểu đường, 75% là đúng
Recall	0.4138 (41,38%)	Trong số các ca thực sự mắc tiểu đường, chỉ 41,38% được phát hiện
F1-score	0.5333	Trung bình điều hòa giữa Precision và Recall

1.8.3. Ma trận nhầm lẫn (Confusion Matrix)

	Dự đoán: Không tiểu đường (0)	Dự đoán: Tiểu đường (1)
Thực tế: Không tiểu đường (0)	TN = 88	FP = 8
Thực tế: Tiểu đường (1)	FN = 34	TP = 24

Ma trận nhầm lẫn cho thấy: trong 96 ca thực sự không mắc bệnh, mô hình dự đoán đúng 88 ca (chỉ nhầm 8 ca); nhưng trong 58 ca thực sự mắc bệnh, mô hình chỉ phát hiện đúng 24 ca và bỏ sót tới 34 ca (False Negative).

1.8.4. Ví dụ dự đoán trên 10 bệnh nhân đầu tiên của tập kiểm thử

STT	Xác suất dự đoán	Nhãn dự đoán	Nhãn thực tế	Nhận xét
1	0.0812	0	0	Đúng
2	0.1986	0	0	Đúng
3	0.2142	0	0	Đúng
4	0.6060	1	0	Sai (False Positive)
5	0.3441	0	1	Sai (False Negative)
6	0.3260	0	0	Đúng
7	0.2690	0	0	Đúng
8	0.1551	0	0	Đúng
9	0.3416	0	1	Sai (False Negative)
10	0.3074	0	0	Đúng

1.9. NHẬN XÉT VÀ THẢO LUẬN

- Về quá trình học: Loss giảm liên tục và Ổn định qua các epoch chứng tỏ thuật toán lan truyền ngược và gradient descent đã được cài đặt đúng. Tuy nhiên, loss cuối cùng (≈ 0.49) vẫn còn khá cao, cho thấy mô hình chưa hội tụ hoàn toàn — có thể do learning rate nhỏ (0.01), số epoch chưa đủ lớn, hoặc kiến trúc mạng còn đơn giản.
- Về độ chính xác tổng thể: Accuracy đạt 72,73% là chấp nhận được cho một mô hình xây dựng từ đầu, không qua tinh chỉnh siêu tham số (hyperparameter tuning), nhưng chưa cao nếu so với các mô hình đã được tối ưu kỹ trên cùng bộ dữ liệu này.
- Về sự mất cân bằng giữa Precision và Recall: Precision (75%) cao hơn đáng kể so với Recall (41,38%). Điều này có nghĩa mô hình có xu hướng "thận trọng" khi dự đoán dương tính (tiểu đường): khi đã dự đoán dương tính thì thường đúng, nhưng lại bỏ sót nhiều ca dương tính thật (34/58 ca, tương ứng gần 59%).
- Ý nghĩa thực tiễn: Trong bài toán y tế, False Negative (bỏ sót bệnh nhân thực sự mắc bệnh) thường nguy hiểm hơn False Positive, vì có thể khiến bệnh nhân không được chẩn đoán và điều trị kịp thời. Do đó, Recall thấp (41,38%) là một hạn chế quan trọng cần khắc phục, ví dụ bằng cách hạ ngưỡng quyết định (decision threshold) xuống dưới 0.5, hoặc cân bằng lại dữ liệu huấn luyện giữa hai lớp.
- Nguyên nhân có thể của hạn chế trên: (i) tập dữ liệu Pima Diabetes vốn mất cân bằng nhãn (số ca không mắc bệnh nhiều hơn số ca mắc bệnh); (ii) mô hình chỉ được huấn luyện 1000

epoch với learning rate cố định 0.01, chưa áp dụng các kỹ thuật nâng cao như điều chỉnh learning rate, regularization hay class weighting.

1.10. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1.10.1. Kết luận

Báo cáo đã trình bày đầy đủ quá trình xây dựng một mạng nơ-ron nhân tạo 3 lớp (kiến trúc $8 \rightarrow 16 \rightarrow 8 \rightarrow 1$) hoàn toàn từ đầu bằng NumPy, không sử dụng bất kỳ framework Deep Learning nào, để giải quyết bài toán dự đoán bệnh tiểu đường. Các thành phần cốt lõi của Deep Learning — lan truyền tiến, hàm kích hoạt ReLU/Sigmoid, hàm mất mát Binary Cross-Entropy, lan truyền ngược và gradient descent — đều được cài đặt thủ công và minh chứng bằng toán học tương ứng. Mô hình đạt Accuracy 72,73%, Precision 75%, Recall 41,38% và F1-score 0,5333 trên tập kiểm thử, cho thấy quá trình huấn luyện hoạt động đúng nhưng vẫn còn dư địa cải thiện, đặc biệt về khả năng phát hiện đúng các ca mắc bệnh (Recall).

Qua bài tập này, có thể khẳng định lại ý tưởng cốt lõi mà tài liệu học tập đã nêu: Deep Learning về bản chất là sự hợp thành của các hàm số được học (composition of learned functions), và các framework như PyTorch hay TensorFlow không thay đổi bản chất toán học này mà chỉ cung cấp công cụ hiệu quả hơn để hiện thực hóa nó, đặc biệt thông qua tính năng tính đạo hàm tự động (automatic differentiation).

1.10.2. Hướng phát triển / Đề xuất cải tiến

Dựa trên các bài tập đề xuất trong tài liệu học tập, một số hướng cải tiến có thể thực hiện tiếp theo bao gồm:

- Thay đổi kiến trúc mạng: thử nghiệm với $8 \rightarrow 32 \rightarrow 16 \rightarrow 1$ hoặc bỏ bớt/thêm lớp ẩn để so sánh hiệu năng.
- Điều chỉnh learning rate ($\eta = 0.001; 0.01; 0.1$) để tìm tốc độ học phù hợp, tránh hội tụ quá chậm hoặc dao động không ổn định.
- Tăng số epoch huấn luyện hoặc thêm điều kiện dừng sớm (early stopping) khi loss không còn cải thiện.
- Điều chỉnh ngưỡng quyết định (decision threshold) xuống dưới 0.5 (ví dụ 0.3) để tăng Recall, phù hợp hơn với bài toán y tế nơi bỏ sót ca bệnh có rủi ro cao.
- Xử lý mất cân bằng dữ liệu (class imbalance) bằng kỹ thuật oversampling, undersampling hoặc trọng số lớp (class weighting) trong hàm mất mát.
- So sánh với việc loại bỏ hàm kích hoạt ReLU để minh họa thực nghiệm rằng việc chèn các phép biến đổi tuyến tính không mang lại khả năng biểu diễn phi tuyến.
- Sau khi hiểu rõ bản chất cài đặt thủ công, chuyển sang triển khai lại mô hình bằng một framework Deep Learning (PyTorch/TensorFlow/Keras) để so sánh tốc độ phát triển và hiệu năng.

2. Dự đoán giá nhà Việt Nam

XÂY DỰNG MẠNG NƠ-RON SÂU TỪ ĐẦU BẰNG NUMPY

CHO BÀI TOÁN DỰ ĐOÁN GIÁ NHÀ Ở VIỆT NAM

2.1. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU

Định giá bất động sản là một bài toán thực tế quan trọng, ảnh hưởng trực tiếp đến quyết định mua bán, đầu tư và quy hoạch đô thị. Giá nhà phụ thuộc vào nhiều yếu tố như diện tích, vị trí, số tầng, số phòng ngủ, số phòng tắm... và mối quan hệ giữa các yếu tố này với giá bán thường không tuyến tính.

Báo cáo này trình bày quá trình xây dựng một mạng nơ-ron nhân tạo sâu (5 lớp) hoàn toàn từ đầu bằng NumPy, nhằm giải quyết bài toán dự đoán giá nhà tại Việt Nam dựa trên 4 đặc trưng số: diện tích, số tầng, số phòng ngủ và số phòng tắm. Khác với báo cáo trước (bài toán phân loại nhị phân dự đoán tiểu đường), đây là một bài toán hồi quy (regression) — đầu ra là một giá trị số thực liên tục (giá nhà tính bằng tỷ VND) thay vì một xác suất phân loại.

Mục tiêu cụ thể của báo cáo bao gồm:

10. Trình bày sự khác biệt giữa bài toán hồi quy và bài toán phân loại trong Deep Learning, đặc biệt là ở hàm kích hoạt đầu ra và hàm mất mát.
11. Cài đặt thủ công một mạng nơ-ron sâu 5 lớp ($4 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$) bằng NumPy, bao gồm lan truyền tiến, hàm mất mát MSE, lan truyền ngược và gradient descent.
12. Huấn luyện mô hình trên tập dữ liệu Vietnam Housing Dataset với 21.906 mẫu bất động sản thực tế.
13. Đánh giá mô hình bằng các độ đo MSE, RMSE, MAE, R^2 và phân tích trực quan qua các biểu đồ residuals, scatter plot.
14. Phân tích tầm quan trọng của từng đặc trưng (feature importance) đối với giá nhà dự đoán.

2.2. CƠ SỞ LÝ THUYẾT

2.2.1. Bài toán hồi quy (Regression) trong Deep Learning

Cũng giống như báo cáo trước, ý tưởng trung tâm của Deep Learning vẫn là: mô hình học một chuỗi các phép biến đổi (representation) ghép nối tiếp nhau, biến đầu vào X thành đầu ra dự đoán $\hat{y}$ thông qua hợp thành hàm số:

$$\hat{y} = f_5(f_4(f_3(f_2(f_1(X)))))$$

Điểm khác biệt căn bản của bài toán hồi quy so với bài toán phân loại nằm ở hai chỗ: (1) hàm kích hoạt ở lớp đầu ra, và (2) hàm mất mát sử dụng. Trong bài toán phân loại nhị phân (như dự đoán tiểu đường ở báo cáo trước), lớp cuối dùng hàm sigmoid để ép đầu ra về khoảng (0, 1) thể hiện xác suất, và dùng hàm mất mát binary cross-entropy. Ngược lại, trong bài toán hồi quy dự đoán giá nhà, giá trị cần dự đoán là một số thực bất kỳ (có thể rất lớn, tính bằng tỷ VND), do đó lớp đầu ra sử dụng hàm kích hoạt tuyến tính (linear activation) — tức là giữ nguyên giá trị Z , không ép về một khoảng giới hạn nào.

$$\hat{y} = Z_5 = H_4W_5 + b_5 \text{ (không qua sigmoid)}$$

2.2.2. Sự khác biệt so với bài toán phân loại nhị phân

Tiêu chí	Phân loại nhị phân (Diabetes)	Hồi quy (Giá nhà)
Kiểu đầu ra	Xác suất trong (0, 1)	Giá trị số thực bất kỳ (tỷ VND)
Hàm kích hoạt lớp ra	Sigmoid	Linear (không kích hoạt / identity)
Hàm mất mát	Binary Cross-Entropy	Mean Squared Error (MSE)
Đạo hàm tại lớp ra	$dZ = \hat{y} - y$	$dZ = 2(\hat{y} - y)/N$
Độ đo đánh giá	Accuracy, Precision, Recall, F1	MSE, RMSE, MAE, R^2
Tiền xử lý nhãn y	Không cần chuẩn hóa ($y \in \{0,1\}$)	Cần chuẩn hóa y vì giá trị dao động lớn (vài trăm triệu đến hàng chục tỷ VND)

Điểm đáng chú ý là việc chuẩn hóa nhãn y (giá nhà) trước khi huấn luyện. Vì giá nhà có thể dao động từ dưới 1 tỷ đến hàng chục, hàng trăm tỷ VND, nếu huấn luyện trực tiếp trên giá trị gốc, gradient sẽ có độ lớn rất khác nhau giữa các mẫu, khiến quá trình học không ổn định. Do đó y cũng được chuẩn hóa về phân phối chuẩn (mean 0, std 1) giống như các đặc trưng đầu vào, và kết quả dự đoán được chuyển ngược lại (denormalize) về đơn vị tỷ VND khi đánh giá.

2.2.3. Kiến trúc mạng nơ-ron sâu 5 lớp sử dụng

Khác với mạng 3 lớp ($8 \rightarrow 16 \rightarrow 8 \rightarrow 1$) ở bài toán tiểu đường, bài toán dự đoán giá nhà sử dụng một mạng sâu hơn với 5 lớp có tham số huấn luyện, theo kiến trúc:

$$4 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$$

Lớp	Phép biến đổi	Kích thước	Hàm kích hoạt
Lớp 1 (ẩn)	$Z_1 = XW_1 + b_1$	$4 \rightarrow 64$	ReLU
Lớp 2 (ẩn)	$Z_2 = H_1W_2 + b_2$	$64 \rightarrow 32$	ReLU
Lớp 3 (ẩn)	$Z_3 = H_2W_3 + b_3$	$32 \rightarrow 16$	ReLU
Lớp 4 (ẩn)	$Z_4 = H_3W_4 + b_4$	$16 \rightarrow 8$	ReLU

Lớp 5 (đầu ra)	$Z_5 = H_4W_5 + b_5$	$8 \rightarrow 1$	Linear (identity)
----------------	----------------------	-------------------	-------------------

Mạng có 4 đặc trưng đầu vào (Area, Floors, Bedrooms, Bathrooms) nhưng lớp ẩn đầu tiên lại mở rộng lên tới 64 nơ-ron — nhiều hơn số đặc trưng gốc gấp 16 lần. Việc mở rộng (expand) rồi thu hẹp dần qua các lớp ($64 \rightarrow 32 \rightarrow 16 \rightarrow 8$) cho phép mạng học được nhiều tổ hợp phi tuyến khác nhau của 4 đặc trưng ban đầu ở lớp đầu, sau đó dần dần chắt lọc và nén lại thành các biểu diễn cô đọng hơn ở các lớp sau, trước khi ánh xạ về 1 giá trị giá nhà dự đoán duy nhất.

2.3. DỮ LIỆU VÀ TIỀN XỬ LÝ

Bộ dữ liệu sử dụng là Vietnam Housing Dataset (vietnam_housing_dataset.csv), chứa thông tin về các bất động sản rao bán tại Việt Nam. Sau bước làm sạch dữ liệu (loại bỏ các dòng thiếu giá trị ở 4 cột đặc trưng hoặc cột giá), tập dữ liệu còn lại 21.906 mẫu.

STT	Đặc trưng	Ý nghĩa
1	Area	Diện tích căn nhà/căn hộ (m ²)
2	Floors	Số tầng
3	Bedrooms	Số phòng ngủ
4	Bathrooms	Số phòng tắm
—	Price (nhãn)	Giá bán bất động sản (đơn vị: tỷ VND) — biến mục tiêu cần dự đoán

2.3.1. Làm sạch dữ liệu

Các cột đặc trưng số (Area, Floors, Bedrooms, Bathrooms) được ép kiểu về dạng số bằng `pd.to_numeric(..., errors='coerce')` — giá trị nào không thể chuyển thành số sẽ bị thay bằng NaN. Sau đó, các dòng có giá trị NaN ở bất kỳ đặc trưng nào hoặc ở cột Price đều bị loại bỏ bằng `df.dropna()`. Kết quả cuối cùng thu được ma trận đặc trưng X có kích thước (21906, 4).

2.3.2. Chia tập dữ liệu

Tương tự báo cáo trước, dữ liệu được xáo trộn ngẫu nhiên (`np.random.seed(42)`) và chia theo tỉ lệ 80% huấn luyện – 20% kiểm thử, cho ra 17.524 mẫu huấn luyện và 4.382 mẫu kiểm thử.

2.3.3. Chuẩn hóa dữ liệu (cả đặc trưng lẫn nhãn)

Khác với bài toán phân loại, ở bài toán hồi quy này cả ma trận đặc trưng X và nhãn y đều được chuẩn hóa theo công thức chuẩn hóa Z-score:

$$X' = (X - \mu_X) / \sigma_X ; y' = (y - \mu_y) / \sigma_y$$

trong đó các giá trị trung bình và độ lệch chuẩn (mean, std, y_mean, y_std) đều được tính riêng từ tập huấn luyện rồi áp dụng cho cả tập huấn luyện và kiểm thử, nhằm tránh rò rỉ thông tin (data leakage). Việc chuẩn hóa y là bắt buộc trong bài toán hồi quy vì thang đo giá nhà (có thể từ dưới 1 tỷ đến hàng trăm tỷ VND) rất khác với thang đo chuẩn của các trọng số khởi tạo ngẫu nhiên nhỏ, nếu không chuẩn hóa, loss ban đầu sẽ rất lớn và quá trình học có thể phân kỳ (diverge).

2.4. PHƯƠNG PHÁP XÂY DỰNG MÔ HÌNH

2.4.1. Lan truyền tiến (Forward Propagation)

Với kiến trúc 5 lớp, lan truyền tiến được mở rộng thêm so với mạng 3 lớp trước đây, nhưng vẫn tuân theo đúng nguyên lý composition of functions:

$$Z_1 = XW_1 + b_1, H_1 = \text{ReLU}(Z_1)$$

$$Z_2 = H_1W_2 + b_2, H_2 = \text{ReLU}(Z_2)$$

$$Z_3 = H_2W_3 + b_3, H_3 = \text{ReLU}(Z_3)$$

$$Z_4 = H_3W_4 + b_4, H_4 = \text{ReLU}(Z_4)$$

$$Z_5 = H_4W_5 + b_5, \hat{y} = Z_5 \text{ (linear — không qua kích hoạt phi tuyến)}$$

Bốn lớp ẩn đều dùng ReLU để đưa tính phi tuyến vào mô hình; riêng lớp đầu ra dùng hàm identity (linear) vì giá nhà là một số thực không giới hạn, không cần ép về một khoảng giá trị cụ thể như xác suất.

2.4.2. Hàm mất mát MSE (Mean Squared Error)

Vì đây là bài toán hồi quy, hàm mất mát Binary Cross-Entropy không còn phù hợp (nó chỉ dùng cho phân loại). Thay vào đó, hàm mất mát Sai số Bình phương Trung bình (Mean Squared Error — MSE) được sử dụng:

$$L = (1/N) \sum_i (y_i - \hat{y}_i)^2$$

MSE đo độ lệch bình phương trung bình giữa giá trị dự đoán và giá trị thực tế. Việc bình phương sai số khiến các sai số lớn bị phạt nặng hơn nhiều so với sai số nhỏ, đồng thời đảm bảo giá trị loss luôn không âm.

2.4.3. Lan truyền ngược (Backpropagation)

Đạo hàm của MSE theo đầu ra dự đoán $\hat{y}$ (chính là Z_5 vì lớp ra là linear) được tính trực tiếp:

$$\partial L / \partial Z_5 = 2(\hat{y} - y) / N$$

So với bài toán phân loại (đạo hàm tại lớp ra là $\hat{y} - y$ khi kết hợp sigmoid với binary cross-entropy), công thức ở đây có thêm hệ số 2 do đạo hàm của bình phương, nhưng về bản chất vẫn là "hiệu giữa dự đoán và nhãn thật", chia trung bình cho N mẫu. Từ đó, gradient tiếp tục được lan truyền ngược qua lần lượt 4 lớp ẩn phía trước theo đúng quy tắc chuỗi (chain rule), mỗi lớp đều gồm ba

bước: tính dW , db của lớp hiện tại; lan truyền dH sang lớp trước qua ma trận trọng số; nhân với đạo hàm ReLU để có dZ của lớp trước:

$$dW_5 = H_4^T \cdot dZ_5, dH_4 = dZ_5 \cdot W_5^T, dZ_4 = dH_4 \odot \text{ReLU}'(Z_4)$$

$$dW_4 = H_3^T \cdot dZ_4, dH_3 = dZ_4 \cdot W_4^T, dZ_3 = dH_3 \odot \text{ReLU}'(Z_3)$$

$$dW_3 = H_2^T \cdot dZ_3, dH_2 = dZ_3 \cdot W_3^T, dZ_2 = dH_2 \odot \text{ReLU}'(Z_2)$$

$$dW_2 = H_1^T \cdot dZ_2, dH_1 = dZ_2 \cdot W_2^T, dZ_1 = dH_1 \odot \text{ReLU}'(Z_1)$$

$$dW_1 = X^T \cdot dZ_1$$

2.4.4. Cập nhật trọng số (Gradient Descent)

Toàn bộ 10 tham số (5 ma trận trọng số $W_1 \dots W_5$ và 5 vector bias $b_1 \dots b_5$) được cập nhật theo đúng công thức gradient descent, với learning rate $\eta = 0.005$:

$$W \leftarrow W - \eta \cdot (\partial L / \partial W) ; b \leftarrow b - \eta \cdot (\partial L / \partial b)$$

Chu trình Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update được lặp lại 3000 epoch trên toàn bộ 17.524 mẫu huấn luyện.

2.5. CÀI ĐẶT VÀ HUẤN LUYỆN MÔ HÌNH

Mô hình được cài đặt bằng Python với NumPy cho toàn bộ phần "lỗi" của mạng nơ-ron (khởi tạo, forward, loss, backward, gradient descent); Pandas chỉ dùng để đọc và làm sạch dữ liệu CSV; Matplotlib dùng để trực quan hóa; riêng scikit-learn chỉ được dùng ở bước cuối để tính nhanh các độ đo đánh giá MSE/MAE/ R^2 (không dùng để huấn luyện mô hình). Các thành phần chính đã cài đặt bao gồm:

- Khởi tạo 10 tham số ($W_1 \dots W_5$, $b_1 \dots b_5$) theo phương pháp He initialization, phù hợp với ReLU.
- Hàm kích hoạt $\text{relu}(x)$ và đạo hàm $\text{relu_derivative}(x)$; hàm $\text{linear}(x)$ cho lớp ra.
- Hàm $\text{mse_loss}(y_true, y_pred)$ và $\text{mse_derivative}(y_true, y_pred)$.
- Hàm $\text{forward}(X)$ lan truyền tiến qua 5 lớp, lưu lại cache đầy đủ 10 giá trị trung gian.
- Hàm $\text{backward}(y, \text{cache})$ tính gradient cho toàn bộ 10 tham số bằng chain rule.
- Vòng lặp huấn luyện với learning rate $\eta = 0.005$ và 3000 epoch.
- Hàm $\text{predict_new}(\text{area, floors, bedrooms, bathrooms})$ để dự đoán giá nhà mới, tự động chuẩn hóa đầu vào và chuyển ngược (denormalize) đầu ra về đơn vị tỷ VND.

So với mạng 3 lớp ở bài toán tiểu đường, learning rate ở đây được giảm xuống còn 0.005 (thay vì 0.01) — theo đúng ghi chú trong code — vì mạng sâu hơn (5 lớp thay vì 3 lớp) dễ gặp hiện tượng gradient hoặc cập nhật trọng số dao động mạnh nếu dùng learning rate quá lớn.

2.6. PHÂN TÍCH CHI TIẾT TỪNG CELL CODE TRONG JUPYTER NOTEBOOK

Chương này trình bày lại toàn bộ notebook DL_vietnam_house.ipynb theo đúng thứ tự các cell đã chạy, kèm giải thích ý nghĩa từng dòng lệnh và output thực tế thu được.

2.6.1. Cell 1 — Nạp thư viện và cố định seed

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

np.random.seed(42)

print("✅ Import thành công!")
```

Output:

```
✅ Import thành công!
```

Ba thư viện được nạp: NumPy cho toàn bộ tính toán mạng nơ-ron, Matplotlib để vẽ biểu đồ, và Pandas để đọc/ xử lý file CSV — khác với bài toán tiểu đường (chỉ dùng np.loadtxt), ở đây Pandas được dùng vì file dữ liệu nhà đất có thể chứa các cột không phải số hoặc giá trị thiếu, cần xử lý linh hoạt hơn. np.random.seed(42) được đặt ngay từ đầu để toàn bộ notebook (chia dữ liệu, khởi tạo trọng số) đều có thể tái lập kết quả.

2.6.2. Cell 2 — Đọc dữ liệu và chọn đặc trưng số

```
# Đọc dữ liệu
df = pd.read_csv("C:/Users/Admin/Downloads/Test/vietnam_housing_dataset.csv")

# Chọn features số
numeric_cols = ['Area', 'Floors', 'Bedrooms', 'Bathrooms']

for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')

df_clean = df.dropna(subset=numeric_cols + ['Price'])

X = df_clean[numeric_cols].values
y = df_clean['Price'].values.reshape(-1, 1)

print(f"📊 Dataset shape: {X.shape}")
```

Output:

```
📊 Dataset shape: (21906, 4)
```

pd.read_csv() đọc toàn bộ file CSV thành một DataFrame, cho phép truy cập dữ liệu theo tên cột thay vì chỉ số như NumPy thuần. Danh sách numeric_cols xác định 4 cột đặc trưng đầu vào sẽ dùng

cho mô hình (bỏ qua các cột khác như địa chỉ, loại hình nhà đất, hướng nhà... vốn là dữ liệu dạng chữ, không thể đưa trực tiếp vào mạng nơ-ron nếu không mã hóa).

Vòng lặp for col in numeric_cols dùng `pd.to_numeric(..., errors='coerce')` để ép từng cột về kiểu số; bất kỳ giá trị nào không thể chuyển đổi (ví dụ ô trống, ký tự lạ) sẽ tự động trở thành NaN thay vì gây lỗi chương trình. `df.dropna(subset=...)` sau đó loại bỏ toàn bộ các dòng có giá trị NaN ở 4 cột đặc trưng hoặc ở cột Price, đảm bảo dữ liệu đưa vào huấn luyện hoàn toàn sạch. Cuối cùng, `.values` chuyển DataFrame/Series thành mảng NumPy thuần, và `.reshape(-1, 1)` đưa nhãn y về đúng dạng ma trận cột (N, 1) như đã làm ở báo cáo trước.

2.6.3. Cell 3 — Chia tập huấn luyện và kiểm thử

```
np.random.seed(42)
indices = np.random.permutation(len(X))
split = int(0.8 * len(X))

X_train = X[indices[:split]]
y_train = y[indices[:split]]
X_test = X[indices[split:]]
y_test = y[indices[split:]]

print(f"📖 Training: {len(X_train)} samples")
print(f"🔍 Testing: {len(X_test)} samples")
```

Output:

```
📖 Training: 17524 samples
🔍 Testing: 4382 samples
```

Logic hoàn toàn tương tự báo cáo trước: xáo trộn ngẫu nhiên chỉ số của toàn bộ 21.906 mẫu rồi cắt theo tỉ lệ 80:20, cho ra 17.524 mẫu huấn luyện và 4.382 mẫu kiểm thử. Việc gọi lại `np.random.seed(42)` ngay trước khi `permutation` đảm bảo cách chia dữ liệu luôn giống nhau giữa các lần chạy, bất kể trước đó chương trình đã dùng bao nhiêu số ngẫu nhiên.

2.6.4. Cell 4 — Chuẩn hóa cả đặc trưng và nhãn

```
# Chuẩn hóa features
mean = X_train.mean(axis=0)
std = X_train.std(axis=0) + 1e-8

X_train = (X_train - mean) / std
X_test = (X_test - mean) / std

# Chuẩn hóa target
y_mean = y_train.mean()
y_std = y_train.std() + 1e-8

y_train = (y_train - y_mean) / y_std
y_test = (y_test - y_mean) / y_std
```

```
print("✅ Đã chuẩn hóa dữ liệu")
```

Output:

✅ Đã chuẩn hóa dữ liệu

Phần chuẩn hóa đặc trưng (mean, std theo từng cột, axis=0) giống hết báo cáo trước. Điểm khác biệt quan trọng nằm ở việc chuẩn hóa thêm nhãn `y_train`, `y_test` bằng `y_mean` và `y_std` — đây là bước bắt buộc riêng cho bài toán hồi quy, đã giải thích lý do ở mục 3 của báo cáo. Cần đặc biệt lưu ý: `y_mean` và `y_std` được lưu lại (không bị ghi đè) vì chúng sẽ cần dùng ở bước cuối để chuyển giá trị dự đoán đã chuẩn hóa trở về đơn vị tỷ VND thực tế.

2.6.5. Cell 5 — Định nghĩa các hàm toán học cần thiết

```
def relu(x):  
    return np.maximum(0, x)  
  
def relu_derivative(x):  
    return (x > 0).astype(float)  
  
def linear(x):  
    return x  
  
def mse_loss(y_true, y_pred):  
    return np.mean((y_true - y_pred) ** 2)  
  
def mse_derivative(y_true, y_pred):  
    return 2 * (y_pred - y_true) / len(y_true)
```

Bốn hàm `relu`, `relu_derivative` giống hết báo cáo trước. Hai hàm mới xuất hiện do đây là bài toán hồi quy: `linear(x)` — hàm kích hoạt đồng nhất (identity) cho lớp đầu ra, và `mse_loss/mse_derivative` thay thế cho `binary_cross_entropy`. Giải thích chi tiết ở Chương 7.

2.6.6. Cell 6 — Khởi tạo mạng nơ-ron sâu 5 lớp

```
# ARCHITECTURE: 4 → 64 → 32 → 16 → 8 → 1
input_size = 4
hidden1_size = 64
hidden2_size = 32
hidden3_size = 16
hidden4_size = 8
output_size = 1

# He initialization
W1 = np.random.randn(input_size, hidden1_size) * np.sqrt(2.0 / input_size)
b1 = np.zeros((1, hidden1_size))
W2 = np.random.randn(hidden1_size, hidden2_size) * np.sqrt(2.0 / hidden1_size)
b2 = np.zeros((1, hidden2_size))
W3 = np.random.randn(hidden2_size, hidden3_size) * np.sqrt(2.0 / hidden2_size)
b3 = np.zeros((1, hidden3_size))
W4 = np.random.randn(hidden3_size, hidden4_size) * np.sqrt(2.0 / hidden3_size)
b4 = np.zeros((1, hidden4_size))
W5 = np.random.randn(hidden4_size, output_size) * np.sqrt(2.0 / hidden4_size)
b5 = np.zeros((1, output_size))

print("✅ Khởi tạo 5-layer network thành công!")
print(f"📐 W1: {W1.shape}, b1: {b1.shape}")
print(f"📐 W2: {W2.shape}, b2: {b2.shape}")
print(f"📐 W3: {W3.shape}, b3: {b3.shape}")
print(f"📐 W4: {W4.shape}, b4: {b4.shape}")
print(f"📐 W5: {W5.shape}, b5: {b5.shape}")
```

Output:

```
✅ Khởi tạo 5-layer network thành công!
📐 W1: (4, 64), b1: (1, 64)
📐 W2: (64, 32), b2: (1, 32)
📐 W3: (32, 16), b3: (1, 16)
📐 W4: (16, 8), b4: (1, 8)
📐 W5: (8, 1), b5: (1, 1)
```

Cell này khai báo trước kích thước của cả 5 lớp bằng các biến `input_size`, `hidden1_size`, ..., `output_size`, giúp code dễ đọc và dễ điều chỉnh kiến trúc hơn là viết số trực tiếp. Công thức khởi tạo He initialization (nhân thêm $\sqrt{2/n_{in}}$) được áp dụng nhất quán cho cả 5 ma trận trọng số, với n_{in} là số nơ-ron đầu vào của từng lớp tương ứng (4, 64, 32, 16, 8). Việc in ra shape của từng cặp (W, b) ngay sau khi khởi tạo là một thói quen tốt giúp phát hiện sớm lỗi sai kích thước ma trận trước khi bắt đầu huấn luyện.

2.6.7. Cell 7 — Hàm forward(X) qua 5 lớp

```
def forward(X):
    z1 = X @ W1 + b1
    h1 = relu(z1)
    z2 = h1 @ W2 + b2
    h2 = relu(z2)
    z3 = h2 @ W3 + b3
    h3 = relu(z3)
    z4 = h3 @ W4 + b4
    h4 = relu(z4)
    z5 = h4 @ W5 + b5
    y_hat = linear(z5)
    cache = {
        "x": X, "z1": z1, "h1": h1, "z2": z2, "h2": h2,
        "z3": z3, "h3": h3, "z4": z4, "h4": h4,
        "z5": z5, "y_hat": y_hat
    }
    return y_hat, cache

# Test forward
y_hat_test, _ = forward(X_train[:5])
print(f"✅ Forward test: {y_hat_test.shape}")
```

Output:

```
✅ Forward test: (5, 1)
```

Hàm forward mở rộng trực tiếp từ mạng 3 lớp ở báo cáo trước, chỉ thêm 2 lớp ẩn nữa (z3/h3 và z4/h4) trước khi tới lớp ra z5. Điểm khác biệt quan trọng: $y_hat = \text{linear}(z5)$ chứ không phải $\text{sigmoid}(z5)$ — vì $\text{linear}(x)$ chỉ trả về chính x , dòng này về bản chất tương đương $y_hat = z5$, giữ nguyên giá trị số thực không giới hạn. Dòng kiểm tra `forward(X_train[:5])` chạy thử trên 5 mẫu đầu tiên để xác nhận hình dạng đầu ra đúng là (5, 1) trước khi đưa vào vòng lặp huấn luyện chính thức trên toàn bộ dữ liệu — đây là một bước kiểm thử đơn vị (unit test) đơn giản, giúp phát hiện lỗi sớm.

2.6.8. Cell 8 — Hàm backward(y, cache) qua 5 lớp

```

def backward(y, cache):
    X = cache["x"]
    z1, h1 = cache["z1"], cache["h1"]
    z2, h2 = cache["z2"], cache["h2"]
    z3, h3 = cache["z3"], cache["h3"]
    z4, h4 = cache["z4"], cache["h4"]
    yhat = cache["y_hat"]
    n = len(X)

    dz5 = mse_derivative(y, yhat)
    dW5 = h4.T @ dz5
    db5 = np.sum(dz5, axis=0, keepdims=True)

    dh4 = dz5 @ W5.T
    dz4 = dh4 * relu_derivative(z4)
    dW4 = h3.T @ dz4
    db4 = np.sum(dz4, axis=0, keepdims=True)

    dh3 = dz4 @ W4.T
    dz3 = dh3 * relu_derivative(z3)
    dW3 = h2.T @ dz3
    db3 = np.sum(dz3, axis=0, keepdims=True)

    dh2 = dz3 @ W3.T
    dz2 = dh2 * relu_derivative(z2)
    dW2 = h1.T @ dz2
    db2 = np.sum(dz2, axis=0, keepdims=True)

    dh1 = dz2 @ W2.T
    dz1 = dh1 * relu_derivative(z1)
    dW1 = X.T @ dz1
    db1 = np.sum(dz1, axis=0, keepdims=True)

    gradients = {"dW1": dW1, "db1": db1, "dW2": dW2, "db2": db2,
                  "dW3": dW3, "db3": db3, "dW4": dW4, "db4": db4,
                  "dW5": dW5, "db5": db5}
    return gradients

# Test backward
grads = backward(y_train[:5], forward(X_train[:5])[1])
print("✅ Backward test thành công!")
for key in grads:
    print(f"📐 {key}: {grads[key].shape}")

```

Output:

✅ Backward test thành công!

▴ dW1: (4, 64)
▴ db1: (1, 64)
▴ dW2: (64, 32)
▴ db2: (1, 32)
▴ dW3: (32, 16)
▴ db3: (1, 16)
▴ dW4: (16, 8)
▴ db4: (1, 8)
▴ dW5: (8, 1)
▴ db5: (1, 1)

Điểm khác biệt duy nhất so với backward() ở bài toán tiểu đường nằm ở dòng đầu tiên: $dz5 = \text{mse_derivative}(y, \hat{y})$ thay vì $dz3 = (\hat{y} - y)/n$. Toàn bộ phần còn lại — lan truyền gradient qua 4 lớp ẩn ReLU — có cấu trúc lặp lại giống hệt, chỉ khác số lượng lớp (4 lớp ẩn thay vì 2). Điều này một lần nữa minh chứng cho tính tổng quát của backpropagation: dù mạng có bao nhiêu lớp, thuật toán vẫn chỉ là việc lặp lại đúng ba bước (tính dW/db , lan truyền dH , nhân đạo hàm kích hoạt) cho từng lớp theo thứ tự ngược lại với forward. Dòng kiểm tra cuối cùng gọi backward() ngay sau forward() trên 5 mẫu để xác nhận shape của toàn bộ 10 gradient khớp đúng với shape của 10 tham số tương ứng.

2.6.9. Cell 9 — Vòng lặp huấn luyện (3000 epoch)

```
learning_rate = 0.005 # Có thể giảm learning rate cho model sâu hơn
epochs = 3000
loss_history = []

print("🚀 Bắt đầu training 5-layer network...")
print("-" * 70)

for epoch in range(epochs):
    y_hat, cache = forward(X_train)
    loss = mse_loss(y_train, y_hat)
    loss_history.append(loss)
    gradients = backward(y_train, cache)

    W1 -= learning_rate * gradients["dW1"]; b1 -= learning_rate * gradients["db1"]
    W2 -= learning_rate * gradients["dW2"]; b2 -= learning_rate * gradients["db2"]
    W3 -= learning_rate * gradients["dW3"]; b3 -= learning_rate * gradients["db3"]
    W4 -= learning_rate * gradients["dW4"]; b4 -= learning_rate * gradients["db4"]
    W5 -= learning_rate * gradients["dW5"]; b5 -= learning_rate * gradients["db5"]

    if epoch % 200 == 0:
        print(f"Epoch {epoch:5d} | Loss = {loss:.6f}")

print("-" * 70)
print(f"✅ Training hoàn tất! Final Loss: {loss_history[-1]:.6f}")
```

Output:

🚀 Bắt đầu training 5-layer network...

```
Epoch 0 | Loss = 1.742146
Epoch 200 | Loss = 0.736867
Epoch 400 | Loss = 0.712499
Epoch 600 | Loss = 0.693345
Epoch 800 | Loss = 0.685019
Epoch 1000 | Loss = 0.679703
Epoch 1200 | Loss = 0.676099
Epoch 1400 | Loss = 0.673694
Epoch 1600 | Loss = 0.671819
Epoch 1800 | Loss = 0.670305
Epoch 2000 | Loss = 0.669109
Epoch 2200 | Loss = 0.668068
Epoch 2400 | Loss = 0.667140
Epoch 2600 | Loss = 0.666284
Epoch 2800 | Loss = 0.665466
```

✅ Training hoàn tất! Final Loss: 0.664727

So với bài toán tiêu đường (`learning_rate = 0.01`, `epochs = 1000`), ở đây `learning_rate` được giảm còn 0.005 và số epoch tăng lên 3000 — đúng như chú thích trong code "Có thể giảm `learning_rate` cho model sâu hơn": mạng càng sâu, gradient lan truyền qua càng nhiều lớp nhân liên tiếp, dễ khiến bước cập nhật trọng số dao động mạnh nếu `learning_rate` quá lớn, nên cần chọn `learning_rate` nhỏ hơn và bù lại bằng cách huấn luyện nhiều epoch hơn.

Quan sát dãy giá trị loss (MSE trên dữ liệu đã chuẩn hóa), có thể thấy: loss giảm rất nhanh trong khoảng 200 epoch đầu (từ 1.74 xuống 0.74), sau đó giảm rất chậm và gần như đi ngang từ epoch 1000 trở đi (0.6797 → 0.6647), chỉ giảm thêm khoảng 0.015 trong suốt 2000 epoch cuối. Đây là dấu hiệu mô hình đã tiệm cận điểm hội tụ với cấu hình `learning_rate` và kiến trúc hiện tại, hoặc đã chạm giới hạn khả năng biểu diễn của 4 đặc trưng đầu vào đối với sự biến thiên phức tạp của giá nhà thực tế (phân tích sâu hơn ở Chương 9).

2.6.10. Cell 10 — Vẽ biểu đồ đường cong Loss

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(loss_history)
plt.xlabel("Epoch"); plt.ylabel("Loss (MSE)")
plt.title("Training Loss - 5 Layer Network"); plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(loss_history[-200:])
plt.xlabel("Epoch (last 200)"); plt.ylabel("Loss (MSE)")
plt.title("Training Loss (Zoom)"); plt.grid(True)

plt.tight_layout()
plt.show()
```

Cell này vẽ hai biểu đồ cạnh nhau: biểu đồ bên trái thể hiện toàn bộ 3000 epoch (cho thấy dạng cong giảm mạnh rồi thoải dần đặc trưng), biểu đồ bên phải "zoom" vào 200 epoch cuối cùng (loss_history[-200:]) để quan sát rõ hơn liệu mô hình có còn đang cải thiện hay đã thực sự bão hòa (plateau) ở giai đoạn cuối huấn luyện — một kỹ thuật trực quan hóa hữu ích khi trục tung của biểu đồ tổng thể quá lớn khiến các thay đổi nhỏ ở cuối khó nhìn thấy.

2.6.11. Cell 11 — Dự đoán và tính các độ đo trên tập kiểm thử

```
# Dự đoán
y_pred_normalized, _ = forward(X_test)
y_pred = y_pred_normalized * y_std + y_mean
y_test_original = y_test * y_std + y_mean

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

mse = mean_squared_error(y_test_original, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test_original, y_pred)
r2 = r2_score(y_test_original, y_pred)

print("=" * 60)
print("📊 KẾT QUẢ 5-LAYER NETWORK")
print("=" * 60)
print(f"📉 MSE: {mse:.4f}")
print(f"📉 RMSE: {rmse:.4f} (tỷ VND)")
print(f"📊 MAE: {mae:.4f} (tỷ VND)")
print(f"📈 R²: {r2:.4f}")
```

Output:

```
=====
📊 KẾT QUẢ 5-LAYER NETWORK
=====
📉 MSE: 3.3263
📉 RMSE: 1.8238 (tỷ VND)
📊 MAE: 1.4690 (tỷ VND)
📈 R²: 0.3139
```

Đây là bước quan trọng: $y_pred_normalized = forward(X_test)$ cho ra dự đoán trên thang đo đã chuẩn hóa (mean 0, std 1), hoàn toàn không có ý nghĩa trực tiếp để báo cáo. Dòng $y_pred = y_pred_normalized * y_std + y_mean$ thực hiện phép biến đổi ngược (denormalize/inverse transform) — đúng bằng phép nghịch đảo của công thức chuẩn hóa ở Cell 4 — để đưa giá trị dự đoán về đúng đơn vị gốc là tỷ VND. Tương tự, y_test cũng được chuyển ngược về $y_test_original$ để so sánh công bằng trên cùng thang đo thực tế.

Bốn độ đo được tính bằng scikit-learn (chỉ dùng để tính toán, không dùng để huấn luyện): MSE và RMSE đo sai số bình phương trung bình (RMSE có cùng đơn vị với giá nhà — tỷ VND — nên dễ diễn giải hơn MSE); MAE đo sai số tuyệt đối trung bình; R^2 (hệ số xác định) đo tỉ lệ phương sai của giá

nhà thực tế được mô hình giải thích được, với giá trị tối đa là 1 (dự đoán hoàn hảo) và có thể âm nếu mô hình tệ hơn cả việc luôn dự đoán bằng giá trị trung bình.

2.6.12. Cell 12 — Biểu đồ Predictions vs Actual

```
plt.figure(figsize=(10, 6))
plt.scatter(y_test_original, y_pred, alpha=0.6, label='Predictions')

min_val = min(y_test_original.min(), y_pred.min())
max_val = max(y_test_original.max(), y_pred.max())
plt.plot([min_val, max_val], [min_val, max_val], 'r--', label='Perfect Prediction')

plt.xlabel("Giá thực tế (tỷ VND)")
plt.ylabel("Giá dự đoán (tỷ VND)")
plt.title(f"5-Layer Network: Predictions vs Actual (R² = {r2:.4f})")
plt.legend(); plt.grid(True)
plt.show()
```

Biểu đồ scatter đối chiếu trực tiếp giá thực tế (trục hoành) với giá mô hình dự đoán (trục tung) cho từng mẫu trong tập kiểm thử. Đường chấm đỏ $y = x$ ("Perfect Prediction") biểu diễn trường hợp lý tưởng dự đoán trùng khớp hoàn toàn với thực tế: các điểm càng nằm gần đường này, dự đoán càng chính xác; các điểm phân tán rộng ra khỏi đường này thể hiện sai số lớn.

2.6.13. Cell 13 — Phân tích Residuals (sai số dự đoán)

```
residuals = y_test_original.flatten() - y_pred.flatten()

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.scatter(y_pred, residuals, alpha=0.6)
plt.axhline(y=0, color='r', linestyle='--', label='Zero Error')
plt.xlabel("Giá dự đoán (tỷ VND)"); plt.ylabel("Residuals (Sai số)")
plt.title("Residual Plot"); plt.legend(); plt.grid(True)

plt.subplot(1, 2, 2)
plt.hist(residuals, bins=30, edgecolor='black', alpha=0.7)
plt.xlabel("Sai số (tỷ VND)"); plt.ylabel("Tần suất")
plt.title("Distribution of Residuals"); plt.grid(True)

plt.tight_layout()
plt.show()

print("📊 Thống kê Residuals:")
print(f"Mean: {np.mean(residuals):.4f}")
print(f"Std: {np.std(residuals):.4f}")
print(f"Min: {np.min(residuals):.4f}")
print(f"Max: {np.max(residuals):.4f}")
```

Output:

 Thống kê Residuals:

Mean: -0.0641

Std: 1.8227

Min: -6.5620

Max: 6.1748

residual (phần dư/sai số) được định nghĩa là hiệu giữa giá trị thực tế và giá trị dự đoán: $\text{residual} = y_{\text{thực}} - \hat{y}$. Biểu đồ Residual Plot (bên trái) vẽ residual theo giá dự đoán: nếu mô hình tốt, các điểm nên phân tán ngẫu nhiên quanh đường 0 (không có xu hướng hệ thống); nếu các điểm tạo thành một hình dạng có quy luật (ví dụ hình phễu, hoặc xu hướng tăng/giảm dần), đó là dấu hiệu mô hình còn bỏ sót một mối quan hệ nào đó trong dữ liệu. Biểu đồ Histogram (bên phải) cho biết phân phối của sai số có gần với phân phối chuẩn (hình chuông) hay không.

Thống kê residuals cho thấy: giá trị trung bình của residual gần bằng 0 (-0.0641), nghĩa là mô hình không bị thiên lệch (bias) một cách hệ thống về hướng dự đoán cao hơn hoặc thấp hơn thực tế; tuy nhiên độ lệch chuẩn khá lớn (1.8227 tỷ VND) và sai số cực đại có thể lên tới hơn 6.5 tỷ VND — cho thấy với một số bất động sản, mô hình dự đoán sai lệch đáng kể.

2.6.14. Cell 14 — Phân tích chi tiết một mẫu cụ thể

```
sample_idx = 0
sample_features = X_test[sample_idx].reshape(1, -1)
sample_actual = y_test_original[sample_idx, 0]

sample_pred_norm, cache = forward(sample_features)
sample_pred = sample_pred_norm * y_std + y_mean

print("=" * 60)
print("📄 PHÂN TÍCH CHI TIẾT 1 SAMPLE")
print("=" * 60)
print(f"Features (đã chuẩn hóa):")
for i, col in enumerate(numeric_cols):
    print(f" {col}: {sample_features[0, i]:.2f}")

print(f"💰 Giá thực tế: {sample_actual:.2f} tỷ VND")
print(f"🏠 Giá dự đoán: {sample_pred[0, 0]:.2f} tỷ VND")
print(f"📊 Sai số: {abs(sample_actual - sample_pred[0, 0]):.2f} tỷ VND")

print("\n🧠 Hidden Representations:")
print(f" Layer 1 (64 neurons): {cache['h1'][0, :5]}...")
print(f" Layer 2 (32 neurons): {cache['h2'][0, :5]}...")
print(f" Layer 3 (16 neurons): {cache['h3'][0, :5]}...")
print(f" Layer 4 (8 neurons): {cache['h4'][0, :5]}...")
```

Output:

PHÂN TÍCH CHI TIẾT 1 SAMPLE

Features (đã chuẩn hóa):

Area: -0.83

Floors: 0.51

Bedrooms: -0.41

Bathrooms: -0.29

🎯 Giá thực tế: 4.75 tỷ VND

🏠 Giá dự đoán: 4.47 tỷ VND

📊 Sai số: 0.28 tỷ VND

🧠 Hidden Representations:

Layer 1 (64 neurons): [0. 0. 0.11952995 1.32416873 0.59707202]...

Layer 2 (32 neurons): [0. 0. 0.17059767 0.62177363 0.]...

Layer 3 (16 neurons): [0. 0.28756514 0. 0. 0.52006334]...

Layer 4 (8 neurons): [0. 1.1963996 0.72289621 0. 0.43369633]...

Cell này minh họa cụ thể trên 1 mẫu (căn nhà đầu tiên trong tập kiểm thử) để người đọc thấy rõ toàn bộ luồng dữ liệu đi qua mạng: từ 4 đặc trưng đã chuẩn hóa, qua forward() thu được cache chứa toàn bộ biểu diễn ẩn ở cả 4 lớp, và cuối cùng ra một giá dự đoán. Với mẫu này, mô hình dự đoán 4.47 tỷ VND so với giá thực tế 4.75 tỷ VND, sai số chỉ 0.28 tỷ — một kết quả khá tốt cho trường hợp cụ thể này.

Phần in "Hidden Representations" cho thấy trực tiếp 5 giá trị đầu tiên của mỗi lớp ẩn. Có thể quan sát rằng nhiều giá trị bằng 0 — đây chính là hệ quả của hàm ReLU: bất kỳ nơ-ron nào có $Z \leq 0$ sẽ bị "tắt" (đầu ra = 0) đối với mẫu cụ thể này. Hiện tượng nhiều nơ-ron bị tắt đồng thời gọi là sparsity (tính thưa) của biểu diễn ẩn, là một đặc điểm điển hình của mạng dùng ReLU.

2.6.15. Cell 15 & 16 — Phân tích tầm quan trọng đặc trưng (Feature Importance)

```

n_samples = 100
importance = []

for feat_idx in range(X_test.shape[1]):
    X_perturbed = X_test[:n_samples].copy()
    noise = np.random.randn(n_samples) * 0.1
    X_perturbed[:, feat_idx] += noise

    pred_original, _ = forward(X_test[:n_samples])
    pred_perturbed, _ = forward(X_perturbed)

    loss_original = mse_loss(y_test[:n_samples], pred_original)
    loss_perturbed = mse_loss(y_test[:n_samples], pred_perturbed)

    importance.append(loss_perturbed - loss_original)

plt.figure(figsize=(10, 6))
plt.bar(numeric_cols, importance)
plt.xlabel("Features"); plt.ylabel("Importance (Change in Loss)")
plt.title("Feature Importance Analysis"); plt.grid(True, axis='y')
plt.show()

print("📊 Feature Importance Ranking:")
sorted_idx = np.argsort(importance)[::-1]
for i, idx in enumerate(sorted_idx):
    print(f" {i+1}. {numeric_cols[idx]}: {importance[idx]:.4f}")

```

Output Cell 15:

```

📊 Feature Importance Ranking:
1. Bathrooms: 0.0142
2. Floors: -0.0029
3. Bedrooms: -0.0054
4. Area: -0.0094

```

Output Cell 16 (chạy lại cùng đoạn code y hệt):

```

📊 Feature Importance Ranking:
1. Floors: 0.0037
2. Bedrooms: -0.0005
3. Area: -0.0010
4. Bathrooms: -0.0094

```

Kỹ thuật được dùng ở đây là Permutation/Perturbation Importance: với mỗi đặc trưng, thêm một nhiễu ngẫu nhiên nhỏ ($\text{noise} = \text{np.random.randn}(n_samples) * 0.1$) chỉ vào riêng cột đặc trưng đó, giữ nguyên các cột còn lại, rồi so sánh loss của mô hình trước và sau khi nhiễu ($\text{loss_perturbed} - \text{loss_original}$). Ý tưởng cốt lõi: nếu một đặc trưng thực sự quan trọng đối với dự đoán, việc làm nhiễu nó sẽ khiến loss tăng lên đáng kể; ngược lại, nếu đặc trưng ít ảnh hưởng, loss gần như không đổi.

- Lưu ý quan trọng: Cell 15 và Cell 16 trong notebook chứa đúng cùng một đoạn code (được chạy lại y nguyên), nhưng cho ra hai bảng xếp hạng tầm quan trọng khác nhau — Cell 15 xếp Bathrooms đứng đầu (0.0142), trong khi Cell 16 lại xếp Floors đứng đầu (0.0037).
- Nguyên nhân: đoạn code tạo nhiễu bằng np.random.randn(...) nhưng không cố định lại seed ngay trước vòng lặp, nên mỗi lần chạy lại cell sẽ sinh ra một chuỗi nhiễu ngẫu nhiên khác nhau, dẫn đến kết quả đo lường tầm quan trọng không ổn định (non-deterministic) giữa các lần chạy.
- Đây là một hạn chế đáng lưu ý của phương pháp perturbation importance khi không kiểm soát seed — kết luận về "đặc trưng nào quan trọng nhất" cần được xem xét thận trọng và tốt nhất nên lặp lại nhiều lần rồi lấy trung bình thay vì chỉ dựa vào một lần chạy duy nhất. Phân tích thêm ở Chương 9.

2.6.16. Cell 17 — Lưu mô hình và dự đoán giá nhà mới

```
model_params = {
    'W1': W1, 'b1': b1, 'W2': W2, 'b2': b2, 'W3': W3, 'b3': b3,
    'W4': W4, 'b4': b4, 'W5': W5, 'b5': b5,
    'mean': mean, 'std': std, 'y_mean': y_mean, 'y_std': y_std,
    'numeric_cols': numeric_cols
}
np.savez('housing_model_5layer.npz', **model_params)
print("✅ Đã lưu model vào 'housing_model_5layer.npz'")

def predict_new(area, floors, bedrooms, bathrooms):
    """
    Dự đoán giá nhà với input:
    - area: Diện tích (m²)
    - floors: Số tầng
    - bedrooms: Số phòng ngủ
    - bathrooms: Số phòng tắm
    """
    X_new = np.array([[area, floors, bedrooms, bathrooms]])
    X_new_norm = (X_new - mean) / std
    y_pred_norm, _ = forward(X_new_norm)
    y_pred = y_pred_norm * y_std + y_mean
    return y_pred[0, 0]

print("\n🏠 DỰ ĐOÁN GIÁ NHÀ MỚI:")
print("-" * 50)

test_cases = [(100, 3, 4, 3), (80, 2, 3, 2), (150, 5, 6, 5)]

for area, floors, bedrooms, bathrooms in test_cases:
    price = predict_new(area, floors, bedrooms, bathrooms)
    print(f"📏 {area} m², {floors} tầng, {bedrooms} PN, {bathrooms} PT")
    print(f"→ Giá dự đoán: {price:.2f} tỷ VND\n")
```

Output:

✓ Đã lưu model vào 'housing_model_5layer.npz'

🏠 DỰ ĐOÁN GIÁ NHÀ MỚI:

📏 100m², 3 tầng, 4 PN, 3 PT
→ Giá dự đoán: 6.34 tỷ VND

📏 80m², 2 tầng, 3 PN, 2 PT
→ Giá dự đoán: 4.52 tỷ VND

📏 150m², 5 tầng, 6 PN, 5 PT
→ Giá dự đoán: 8.75 tỷ VND

`np.savez()` lưu toàn bộ 10 tham số đã huấn luyện (`W1..W5`, `b1..b5`) cùng với các thống kê chuẩn hóa (`mean`, `std`, `y_mean`, `y_std`) và tên cột đặc trưng vào một file nén `.npz` duy nhất — cho phép nạp lại mô hình đã huấn luyện sau này mà không cần huấn luyện lại từ đầu, tương tự khái niệm "lưu checkpoint" trong các framework Deep Learning chuyên nghiệp.

Hàm `predict_new()` đóng gói toàn bộ pipeline suy luận (inference) thành một hàm tiện dụng cho người dùng cuối: nhận vào 4 thông số thô (diện tích, số tầng, số phòng ngủ, số phòng tắm) theo đúng đơn vị thực tế, tự động chuẩn hóa bằng đúng `mean/std` đã lưu từ lúc huấn luyện, chạy qua `forward()`, rồi chuyển ngược kết quả về đơn vị tỷ VND — người dùng không cần quan tâm đến các bước chuẩn hóa nội bộ. Ba trường hợp thử nghiệm cho kết quả hợp lý về mặt trực giác: nhà càng lớn (diện tích, số tầng, số phòng nhiều hơn) thì giá dự đoán càng cao, tăng dần từ 4.52 → 6.34 → 8.75 tỷ VND.

2.7. GIẢI THÍCH CHI TIẾT CÁC HÀM TRONG CHƯƠNG TRÌNH

Chương này giải thích sâu về mặt toán học và vai trò của từng hàm trong notebook, tập trung vào các hàm mới xuất hiện so với bài toán phân loại (linear, `mse_loss`, `mse_derivative`) cũng như các hàm được mở rộng quy mô (`forward`, `backward`).

2.7.1. Hàm `relu(x)` và `relu_derivative(x)`

```
def relu(x):  
    return np.maximum(0, x)  
  
def relu_derivative(x):  
    return (x > 0).astype(float)
```

Hai hàm này giữ nguyên vai trò như ở báo cáo tiểu đường: $\text{relu}(x) = \max(0, x)$ đưa tính phi tuyến vào 4 lớp ẩn; `relu_derivative(x)` trả về 1 tại vị trí x dương và 0 tại vị trí $x \leq 0$, dùng để lan truyền ngược gradient qua ReLU. Điểm khác biệt duy nhất là ở đây các hàm này được áp dụng nhất quán cho cả 4 lớp ẩn (thay vì 2 lớp như trước), phản ánh đúng tính "tái sử dụng" của các khối xây dựng cơ bản (building blocks) khi mở rộng độ sâu mạng.

2.7.2. Hàm `linear(x)` — Hàm kích hoạt lớp đầu ra cho hồi quy

```
def linear(x):  
    return x
```

Thành phần	Giải thích
Đầu vào / Đầu ra	x — chính là Z_5 , kết quả tuyến tính của lớp cuối; hàm trả về nguyên vẹn giá trị x , không biến đổi gì
Công thức toán	$f(x) = x$ (hàm đồng nhất — identity function)
Đạo hàm	$f'(x) = 1$ với mọi x (không làm thay đổi độ lớn gradient khi lan truyền ngược qua lớp ra)
Tại sao cần dùng thay vì sigmoid?	Giá nhà là một số thực dương không giới hạn trên (có thể vài trăm triệu đến hàng trăm tỷ VND). Nếu dùng sigmoid, đầu ra sẽ bị ép cứng về khoảng $(0,1)$, hoàn toàn không phù hợp để biểu diễn giá trị tiền tệ thực tế — đây là lý do bắt buộc phải đổi hàm kích hoạt lớp ra khi chuyển từ bài toán phân loại sang bài toán hồi quy

2.7.3. Hàm `mse_loss(y_true, y_pred)`

```
def mse_loss(y_true, y_pred):  
    return np.mean((y_true - y_pred) ** 2)
```

Thành phần	Giải thích
Đầu vào	y_{true} — nhãn giá nhà thật (đã chuẩn hóa); y_{pred} — giá nhà mô hình dự đoán (đã chuẩn hóa)
$(y_{\text{true}} - y_{\text{pred}}) ** 2$	Bình phương sai số của từng mẫu — luôn không âm, và phạt nặng các sai số lớn hơn nhiều so với sai số nhỏ (sai số gấp đôi sẽ bị phạt gấp bốn lần)
<code>np.mean(...)</code>	Lấy trung bình cộng trên toàn bộ N mẫu, cho ra một số vô hướng duy nhất đại diện cho "mức độ sai" trung bình của cả batch
Công thức toán	$L = (1/N) \sum_i (y_i - \hat{y}_i)^2$
So sánh với Binary Cross-Entropy	Không cần <code>np.clip</code> hay hằng số <code>eps</code> như binary cross-entropy, vì phép trừ và bình phương không có nguy cơ gây lỗi $\log(0)$ — đây là một trong những lý do MSE là lựa chọn mặc định phổ biến và đơn giản cho bài toán hồi quy

2.7.4. Hàm `mse_derivative(y_true, y_pred)`

```
def mse_derivative(y_true, y_pred):  
    return 2 * (y_pred - y_true) / len(y_true)
```

Thành phần	Giải thích
Công thức toán	$\partial L / \partial \hat{y} = 2(\hat{y} - y) / N$
Nguồn gốc công thức	Đạo hàm trực tiếp của $L = (1/N)\sum (y - \hat{y})^2$ theo $\hat{y}$: đạo hàm của $(y - \hat{y})^2$ theo $\hat{y}$ là $-2(y - \hat{y}) = 2(\hat{y} - y)$, sau đó chia cho N do có trung bình cộng ở ngoài
Vì sao dùng được trực tiếp làm dz5	Vì lớp ra dùng hàm linear (đạo hàm = 1), nên theo chain rule: $\partial L / \partial z_5 = \partial L / \partial \hat{y} \cdot \partial \hat{y} / \partial z_5 = \partial L / \partial \hat{y} \cdot 1 = \partial L / \partial \hat{y}$ — do đó mse_derivative(y, y_hat) chính là dz5, không cần nhân thêm với đạo hàm hàm kích hoạt nào khác (khác với các lớp ẩn ReLU phía sau, luôn cần nhân với relu_derivative)
len(y_true)	Chính là N — số mẫu trong batch hiện tại (ở đây là toàn bộ 17.524 mẫu huấn luyện vì mô hình dùng Batch Gradient Descent, không chia mini-batch)

2.7.5. Hàm forward(X) — mở rộng cho 5 lớp

```
def forward(X):
    z1 = X @ W1 + b1; h1 = relu(z1)
    z2 = h1 @ W2 + b2; h2 = relu(z2)
    z3 = h2 @ W3 + b3; h3 = relu(z3)
    z4 = h3 @ W4 + b4; h4 = relu(z4)
    z5 = h4 @ W5 + b5; y_hat = linear(z5)
    cache = {...}
    return y_hat, cache
```

Lớp	Dòng lệnh	Kích thước kết quả (N mẫu)
1	$z1 = X @ W1 + b1$; $h1 = \text{relu}(z1)$	$(N,4) @ (4,64) \rightarrow (N,64)$
2	$z2 = h1 @ W2 + b2$; $h2 = \text{relu}(z2)$	$(N,64) @ (64,32) \rightarrow (N,32)$
3	$z3 = h2 @ W3 + b3$; $h3 = \text{relu}(z3)$	$(N,32) @ (32,16) \rightarrow (N,16)$
4	$z4 = h3 @ W4 + b4$; $h4 = \text{relu}(z4)$	$(N,16) @ (16,8) \rightarrow (N,8)$
5 (ra)	$z5 = h4 @ W5 + b5$; $y_hat = \text{linear}(z5)$	$(N,8) @ (8,1) \rightarrow (N,1)$

So với hàm forward() 3 lớp ở bài toán tiểu đường, hàm này chỉ đơn thuần lặp lại đúng một khuôn mẫu (linear transform $\rightarrow$ ReLU) thêm hai lần nữa trước khi tới lớp ra. Điều này cho thấy việc "làm sâu" một mạng nơ-ron về bản chất chỉ là việc nối tiếp thêm các khối biến đổi giống nhau — minh chứng trực tiếp cho nguyên lý composition of functions đã nêu ở Chương 2. Biến cache vẫn đóng vai trò lưu lại toàn bộ 10 giá trị trung gian ($z1..z5$, $h1..h4$, y_hat cùng X gốc), cần thiết cho hàm backward() phía sau.

2.7.6. Hàm backward(y, cache) — lan truyền ngược qua 5 lớp

Cấu trúc của hàm backward() cho mạng 5 lớp là sự mở rộng trực tiếp của mạng 3 lớp, với điểm khởi đầu khác biệt duy nhất là công thức tính dz5 (do đổi từ sigmoid+BCE sang linear+MSE), còn lại toàn bộ 4 lớp vẫn đều áp dụng đúng một khuôn mẫu ba bước lặp lại:

Bước	Công thức tổng quát cho lớp k (tính từ lớp ra về)	Ý nghĩa
1	$dW_k = H_{(k-1)}^T \cdot dZ_k$; $db_k = \sum_i dZ_k$	Gradient của trọng số lớp k được tính từ đầu vào của chính lớp đó ($H_{(k-1)}$) nhân với gradient tại đầu ra tuyến tính của lớp đó (dZ_k)
2	$dH_{(k-1)} = dZ_k \cdot W_k^T$	Lan truyền gradient ngược từ lớp k sang lớp (k-1) thông qua chính ma trận trọng số W_k đã dùng ở forward (nhưng chuyển vị)
3	$dZ_{(k-1)} = dH_{(k-1)} \odot \text{ReLU}'(Z_{(k-1)})$	"Khóa" gradient tại các nơ-ron đã bị ReLU triệt tiêu ở lượt forward của lớp (k-1)

Áp dụng lần lượt khuôn mẫu này cho $k = 5, 4, 3, 2$ (và bước cuối dùng X thay vì H_0 ở lớp 1), ta thu được đúng 10 dòng lệnh tính gradient trong hàm backward() của notebook. Việc nhận ra khuôn mẫu lặp lại này rất quan trọng: nó cho thấy dù mạng có 3 lớp hay 5 lớp hay nhiều hơn nữa, thuật toán backpropagation luôn có thể triển khai bằng một vòng lặp duy nhất chạy ngược qua danh sách các lớp — đây chính là cách các framework như PyTorch/TensorFlow tổng quát hóa việc tính đạo hàm tự động (automatic differentiation) cho mạng với số lớp bất kỳ.

2.7.7. Hàm predict_new(area, floors, bedrooms, bathrooms)

```
def predict_new(area, floors, bedrooms, bathrooms):
    X_new = np.array([[area, floors, bedrooms, bathrooms]])
    X_new_norm = (X_new - mean) / std
    y_pred_norm, _ = forward(X_new_norm)
    y_pred = y_pred_norm * y_std + y_mean
    return y_pred[0, 0]
```

Dòng lệnh	Ý nghĩa
<code>X_new = np.array([[area, floors, bedrooms, bathrooms]])</code>	Đóng gói 4 giá trị đầu vào thô thành một ma trận (1, 4) — đúng định dạng mà hàm forward() yêu cầu, dù chỉ dự đoán cho 1 căn nhà duy nhất
<code>X_new_norm = (X_new - mean) / std</code>	Áp dụng đúng công thức chuẩn hóa đã dùng khi huấn luyện, với mean và std được tính từ tập huấn luyện trước đó — bắt buộc phải dùng lại đúng hai giá trị này (không được tính mean/std mới từ <code>X_new</code>) để đầu vào mới nằm trên cùng thang đo với dữ liệu mô hình đã học

<code>y_pred_norm, _ = forward(X_new_norm)</code>	Chạy lan truyền tiến qua toàn bộ 5 lớp đã huấn luyện, kết quả vẫn ở thang đo chuẩn hóa
<code>y_pred = y_pred_norm * y_std + y_mean</code>	Phép biến đổi ngược (denormalize), đưa kết quả từ thang đo chuẩn hóa về đúng đơn vị tỷ VND thực tế
<code>return y_pred[0, 0]</code>	Trích xuất giá trị số vô hướng duy nhất từ ma trận kết quả (1,1) để trả về cho người dùng

Hàm này là một ví dụ điển hình của việc đóng gói toàn bộ pipeline suy luận (tiền xử lý → mô hình → hậu xử lý) thành một giao diện đơn giản, dễ sử dụng — đúng là mục tiêu cuối cùng của bất kỳ hệ thống Machine Learning/Deep Learning nào khi đưa vào ứng dụng thực tế: người dùng cuối chỉ cần cung cấp thông tin thô (diện tích, số tầng...) mà không cần biết bên trong có bước chuẩn hóa hay mạng nơ-ron 5 lớp nào đang chạy.

2.8. KẾT QUẢ THỰC NGHIỆM

2.8.1. Quá trình giảm Loss trong huấn luyện

Giá trị hàm mất mát (MSE trên dữ liệu đã chuẩn hóa) được ghi nhận sau mỗi 200 epoch như sau:

Epoch	0	200	400	600	800	1000	1400	1800	2200	2600	3000 (cuối)
Loss	1.742	0.737	0.712	0.693	0.685	0.680	0.674	0.670	0.668	0.666	0.6647

Có thể quan sát rõ hai giai đoạn khác biệt: (1) giai đoạn giảm nhanh trong 200-400 epoch đầu tiên, loss giảm từ 1.7421 xuống dưới 0.72 — mô hình nhanh chóng học được các mối quan hệ tuyến tính/phi tuyến cơ bản nhất; (2) giai đoạn giảm chậm và gần như bão hòa (plateau) từ epoch 1000 trở đi, loss chỉ giảm thêm khoảng 0.015 trong suốt 2000 epoch còn lại, cho thấy mô hình đã tiệm cận giới hạn khả năng học với kiến trúc và dữ liệu hiện tại.

2.8.2. Đánh giá trên tập kiểm thử (Test set, 4.382 mẫu)

Độ đo	Giá trị	Ý nghĩa
MSE	3.3263	Sai số bình phương trung bình trên thang đo giá nhà gốc (tỷ VND) ²
RMSE	1.8238 (tỷ VND)	Căn bậc hai của MSE — trung bình mô hình lệch khoảng 1.82 tỷ VND so với giá thực tế
MAE	1.4690 (tỷ VND)	Trung bình trị tuyệt đối sai số — mô hình sai lệch trung bình 1.47 tỷ VND mỗi căn nhà

R ²	0.3139 (31,39%)	Mô hình chỉ giải thích được khoảng 31% sự biến thiên của giá nhà thực tế; gần 69% còn lại chưa được giải thích bởi 4 đặc trưng đầu vào
----------------	-----------------	--

2.8.3. Biểu đồ Predictions vs Actual và Residuals

Biểu đồ scatter giữa giá dự đoán và giá thực tế cho thấy các điểm phân tán khá rộng quanh đường "Perfect Prediction" ($y = x$), phản ánh đúng giá trị R² ở mức trung bình-thấp (0.3139) — mô hình nắm được xu hướng chung (nhà lớn hơn thường có giá cao hơn) nhưng chưa dự đoán chính xác được giá trị cụ thể cho từng căn nhà riêng lẻ.

Thông kê Residuals	Giá trị (tỷ VND)
Trung bình (Mean)	-0.0641
Độ lệch chuẩn (Std)	1.8227
Nhỏ nhất (Min)	-6.5620
Lớn nhất (Max)	6.1748

Residual trung bình gần bằng 0 cho thấy mô hình không bị thiên lệch hệ thống theo một hướng cụ thể (không có xu hướng luôn định giá cao hoặc luôn định giá thấp). Tuy nhiên độ lệch chuẩn residual khá lớn (1.82 tỷ VND) và biên độ dao động rộng (từ -6.56 đến +6.17 tỷ VND), cho thấy với nhiều bất động sản cụ thể (có thể là các căn có giá trị đặc biệt cao hoặc đặc biệt thấp so với mặt bằng chung, hoặc chịu ảnh hưởng bởi các yếu tố không có trong 4 đặc trưng như vị trí, pháp lý...), mô hình dự đoán sai lệch đáng kể.

2.8.4. Ví dụ phân tích chi tiết 1 mẫu

Thông tin	Giá trị
Đặc trưng (đã chuẩn hóa)	Area: -0.83, Floors: 0.51, Bedrooms: -0.41, Bathrooms: -0.29
Giá thực tế	4.75 tỷ VND
Giá dự đoán	4.47 tỷ VND
Sai số tuyệt đối	0.28 tỷ VND

2.8.5. Phân tích tầm quan trọng đặc trưng (Feature Importance)

Lần chạy	Hạng 1	Hạng 2	Hạng 3	Hạng 4
Lần 1 (Cell 15)	Bathrooms (0.0142)	Floors (-0.0029)	Bedrooms (-0.0054)	Area (-0.0094)

Lần 2 (Cell 16)	Floors (0.0037)	Bedrooms (-0.0005)	Area (-0.0010)	Bathrooms (-0.0094)
-----------------	-----------------	-----------------------	----------------	------------------------

Bảng trên cho thấy rõ tính không ổn định của phương pháp perturbation importance khi dùng nhiều ngẫu nhiên không cố định seed: giữa hai lần chạy với cùng một đoạn code, đặc trưng được xếp hạng quan trọng nhất hoàn toàn đổi chỗ (Bathrooms ở lần 1 nhưng lại xếp cuối ở lần 2). Điểm đáng chú ý hơn là mọi giá trị importance đều rất nhỏ, và phần lớn mang giá trị âm — tức là việc thêm nhiều vào đặc trưng đó thậm chí còn làm loss giảm nhẹ (một cách ngẫu nhiên), chứ không tăng lên rõ rệt như kỳ vọng nếu đặc trưng đó thực sự quan trọng. Điều này cho thấy với mức nhiễu nhỏ ($\text{std} = 0.1$ trên thang đã chuẩn hóa) và chỉ 100 mẫu thử, phép đo chưa đủ mạnh để phân biệt rõ tầm quan trọng tương đối giữa 4 đặc trưng — cần thảo luận thêm ở Chương 9.

2.8.6. Ví dụ dự đoán giá nhà mới

Diện tích	Số tầng	Phòng ngủ	Phòng tắm	Giá dự đoán
100 m ²	3	4	3	6.34 tỷ VND
80 m ²	2	3	2	4.52 tỷ VND
150 m ²	5	6	5	8.75 tỷ VND

Ba trường hợp thử nghiệm cho kết quả hợp lý về mặt xu hướng chung: căn nhà có diện tích, số tầng và số phòng lớn hơn đều nhận giá dự đoán cao hơn một cách nhất quán, chứng tỏ mô hình đã học được đúng quan hệ đồng biến cơ bản giữa quy mô bất động sản và giá bán.

2.9. NHẬN XÉT VÀ THẢO LUẬN

15. Về quá trình học: Loss giảm nhanh trong giai đoạn đầu rồi bão hòa rõ rệt từ khoảng epoch 1000 trở đi. Điều này cho thấy learning rate 0.005 và 3000 epoch là đủ để mô hình hội tụ tới mức tối ưu cục bộ mà kiến trúc và dữ liệu hiện tại cho phép — huấn luyện thêm nhiều epoch hơn nữa với cấu hình này khó có thể cải thiện đáng kể thêm.
16. Về độ chính xác tổng thể: R^2 chỉ đạt 0.3139, nghĩa là mô hình mới giải thích được khoảng 31% sự biến thiên của giá nhà thực tế. Đây là một kết quả khiêm tốn so với các bài toán hồi quy giá nhà thường thấy, nơi các mô hình tốt (có đầy đủ đặc trưng và được tinh chỉnh) có thể đạt R^2 trên 0.7–0.8.
17. Nguyên nhân chính có thể đến từ số lượng đặc trưng quá ít: mô hình chỉ sử dụng 4 đặc trưng số (Area, Floors, Bedrooms, Bathrooms), trong khi giá bất động sản trên thực tế còn phụ thuộc rất nhiều vào các yếu tố khác không được đưa vào mô hình — đặc biệt là vị trí/địa chỉ (yếu tố thường được xem là quan trọng nhất trong định giá bất động sản), tình trạng pháp lý (sổ đỏ/sổ hồng), hướng nhà, mặt tiền, tình trạng xây dựng, năm xây dựng,

khu vực lân cận (gần trường học, chợ, đường lớn...). Việc thiếu các đặc trưng quan trọng này khiến "trần" khả năng dự đoán của mô hình bị giới hạn, bất kể mạng nơ-ron có sâu và phức tạp đến đâu.

18. Về việc dùng mạng sâu (5 lớp, tới 64 nơ-ron) cho chỉ 4 đặc trưng đầu vào: đây là một lựa chọn kiến trúc đáng cân nhắc lại. Với chỉ 4 đặc trưng số, một mạng quá sâu và quá rộng ($64 \rightarrow 32 \rightarrow 16 \rightarrow 8$) có nguy cơ dư thừa năng lực biểu diễn (over-parameterized) so với lượng thông tin thực sự có trong dữ liệu đầu vào, đồng thời có thể tăng nguy cơ overfitting nếu không có thêm kỹ thuật regularization (như L2 regularization, dropout — vốn chưa được cài đặt trong notebook này).
19. Về độ ổn định của Feature Importance: như đã phân tích ở mục 8.5, việc không cố định seed khi sinh nhiễu ngẫu nhiên khiến kết quả xếp hạng tầm quan trọng đặc trưng thay đổi hoàn toàn giữa hai lần chạy cùng một đoạn code. Đây là một hạn chế về mặt phương pháp luận (methodology) cần khắc phục: nên đặt `np.random.seed()` cố định trước bước tính importance, hoặc lặp lại phép đo nhiều lần (ví dụ 30-50 lần) rồi lấy trung bình và độ lệch chuẩn để có kết luận đáng tin cậy hơn.
20. Về phân tích Residuals: sai số trung bình gần 0 cho thấy mô hình không bị thiên lệch hệ thống, đây là một điểm tích cực. Tuy nhiên độ lệch chuẩn và biên độ sai số còn khá lớn (lên tới hơn 6 tỷ VND ở một số trường hợp), phản ánh đúng thực tế rằng những bất động sản có giá trị bất thường (rất cao hoặc rất thấp so với mặt bằng chung diện tích/số phòng) thường chịu ảnh hưởng lớn bởi các yếu tố ngoài 4 đặc trưng đã dùng, ví dụ vị trí đắc địa hoặc tình trạng pháp lý đặc biệt.
21. Ý nghĩa thực tiễn: Với hiệu năng hiện tại ($R^2 \approx 0.31$, sai số trung bình khoảng 1.47-1.82 tỷ VND), mô hình có thể tạm dùng để ước lượng nhanh, mang tính tham khảo (ballpark estimate) về khoảng giá dựa trên quy mô căn nhà, nhưng chưa đủ tin cậy để dùng làm công cụ định giá chính xác trong giao dịch thực tế — cần bổ sung thêm đặc trưng và cải tiến mô hình như đề xuất ở Chương 10.

2.10. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

2.10.1. Kết luận

Báo cáo đã trình bày quá trình xây dựng một mạng nơ-ron sâu 5 lớp (kiến trúc $4 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$) hoàn toàn từ đầu bằng NumPy để giải quyết bài toán hồi quy dự đoán giá nhà tại Việt Nam — một bài toán khác về bản chất so với bài toán phân loại nhị phân đã trình bày ở báo cáo trước. Các điểm khác biệt cốt lõi giữa hồi quy và phân loại đã được làm rõ: đổi hàm kích hoạt lớp ra từ sigmoid sang linear, đổi hàm mất mát từ binary cross-entropy sang MSE, và bổ sung bước chuẩn hóa nhãn y do thang đo giá trị mục tiêu rất lớn.

Mô hình đạt $RMSE \approx 1.82$ tỷ VND, $MAE \approx 1.47$ tỷ VND và $R^2 \approx 0.31$ trên tập kiểm thử gồm 4.382 bất động sản. Kết quả này cho thấy mô hình đã học được xu hướng đồng biến cơ bản giữa quy mô nhà (diện tích, số tầng, số phòng) và giá bán, nhưng còn hạn chế đáng kể về độ chính xác tuyệt đối, chủ yếu do giới hạn ở số lượng đặc trưng đầu vào (chỉ 4 đặc trưng số, thiếu các yếu tố quan trọng như vị trí, pháp lý). Bên cạnh đó, báo cáo cũng phát hiện và phân tích một hạn chế về phương pháp luận

trong phần phân tích feature importance của notebook gốc — thiếu việc cố định seed ngẫu nhiên khiến kết quả xếp hạng đặc trưng không ổn định giữa các lần chạy.

Qua bài tập này, có thể thấy rõ ràng: việc mở rộng một mạng nơ-ron từ 3 lớp lên 5 lớp, từ bài toán phân loại sang bài toán hồi quy, về bản chất chỉ đòi hỏi thay đổi rất ít mã nguồn (đổi hàm kích hoạt lớp ra, đổi hàm mất mát, thêm chuẩn hóa nhãn) trong khi toàn bộ khung sườn composition of functions, forward propagation, backpropagation và gradient descent vẫn được giữ nguyên — một lần nữa khẳng định tính tổng quát của các nguyên lý nền tảng Deep Learning đã được xây dựng xuyên suốt cả hai báo cáo.

2.10.2. Hướng phát triển / Đề xuất cải tiến

- Bổ sung thêm đặc trưng: đưa vào các đặc trưng quan trọng khác có trong bộ dữ liệu gốc (nếu có) như địa chỉ/quận huyện (mã hóa bằng one-hot hoặc target encoding), loại hình bất động sản, tình trạng pháp lý, hướng nhà, mặt tiền... để tăng khả năng giải thích biến thiên giá nhà.
- Cố định seed cho bước phân tích feature importance (`np.random.seed()` trước vòng lặp perturbation) và lặp lại phép đo nhiều lần (ví dụ 30-50 lần) rồi lấy giá trị trung bình để có kết luận đáng tin cậy hơn về tầm quan trọng tương đối giữa các đặc trưng.
- Thử nghiệm kiến trúc gọn hơn (ví dụ $4 \rightarrow 16 \rightarrow 8 \rightarrow 1$) khi số đặc trưng đầu vào còn ít, so sánh với kiến trúc sâu hiện tại ($4 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$) để đánh giá xem việc tăng độ sâu/độ rộng mạng có thực sự cải thiện R^2 hay chỉ làm tăng chi phí tính toán mà không mang lại lợi ích tương xứng.
- Thêm kỹ thuật regularization (L2 regularization / weight decay, dropout) để giảm nguy cơ overfitting khi dùng mạng có nhiều tham số so với số đặc trưng đầu vào.
- Thử nghiệm các tối ưu hoá nâng cao hơn gradient descent thuần túy, ví dụ Mini-batch Gradient Descent, Momentum, hoặc Adam, để tăng tốc độ hội tụ và có thể vượt qua trạng thái bão hòa loss đã quan sát được từ epoch 1000 trở đi.
- Log thêm loss trên tập kiểm thử (validation loss) song song với loss huấn luyện trong suốt quá trình training, để phát hiện sớm nếu mô hình bắt đầu overfitting (train loss tiếp tục giảm nhưng validation loss lại tăng).
- So sánh hiệu năng của mạng nơ-ron tự cài đặt này với các mô hình hồi quy truyền thống (Linear Regression, Random Forest, Gradient Boosting/XGBoost) trên cùng bộ dữ liệu, nhằm đánh giá khách quan liệu Deep Learning có thực sự là lựa chọn phù hợp nhất cho bài toán này khi số lượng đặc trưng còn hạn chế, hay các mô hình cây quyết định truyền thống có thể cho kết quả tốt hơn với chi phí tính toán thấp hơn.
- Sau khi hoàn thiện, đóng gói lại toàn bộ pipeline (tiền xử lý, huấn luyện, dự đoán) và triển khai thử dưới dạng một ứng dụng web hoặc API đơn giản, sử dụng hàm `predict_new()` đã có sẵn làm nền tảng cho tầng suy luận (inference layer).

3. Dự đoán chi tiêu khách hàng E-commerce

XÂY DỰNG MẠNG NƠ-RON SÂU TỪ ĐẦU BẰNG NUMPY

CHO BÀI TOÁN DỰ ĐOÁN CHI TIÊU KHÁCH HÀNG E-COMMERCE

3.1. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU

Trong lĩnh vực thương mại điện tử, việc dự đoán mức chi tiêu của khách hàng (Customer Total Spend) là một bài toán phân tích khách hàng (customer analytics) có giá trị thực tiễn cao: doanh nghiệp có thể dùng kết quả dự đoán để phân khúc khách hàng, cá nhân hóa chương trình khuyến mãi, dự báo doanh thu, hoặc nhận diện sớm các khách hàng tiềm năng có giá trị cao (high-value customer).

Báo cáo này trình bày quá trình xây dựng một mạng nơ-ron nhân tạo sâu (5 lớp) từ đầu bằng NumPy để dự đoán tổng chi tiêu (Total Spend) của khách hàng dựa trên 9 đặc trưng hành vi và nhân khẩu học, sử dụng tập dữ liệu E-commerce Customer Behavior. Điểm khác biệt so với hai báo cáo trước là ở khâu tiền xử lý dữ liệu: notebook lần này tận dụng các công cụ có sẵn của thư viện Scikit-learn (LabelEncoder, StandardScaler, train_test_split) để xử lý dữ liệu hỗn hợp gồm cả biến số và biến phân loại (categorical), trong khi phần "lõi" — bản thân mạng nơ-ron — vẫn được cài đặt hoàn toàn thủ công bằng NumPy, đúng theo tinh thần "from scratch" xuyên suốt cả ba báo cáo.

Mục tiêu cụ thể của báo cáo bao gồm:

22. Trình bày cách kết hợp Scikit-learn cho khâu tiền xử lý dữ liệu (mã hóa biến phân loại, chuẩn hóa, chia tập dữ liệu) với một mạng nơ-ron tự cài đặt hoàn toàn bằng NumPy.
23. Xử lý một tập dữ liệu thực tế có cấu trúc phức tạp hơn hai báo cáo trước: vừa có biến số (Age, Items Purchased...) vừa có biến phân loại (Gender, City, Membership Type...) và có giá trị thiếu (missing values).
24. Huấn luyện mạng nơ-ron sâu 5 lớp ($9 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$) để dự đoán Total Spend, đánh giá bằng MSE, RMSE, MAE, R^2 , MAPE.
25. So sánh hiệu năng của mạng nơ-ron tự cài đặt với một mô hình hồi quy tuyến tính (Linear Regression) của Scikit-learn trên cùng bộ dữ liệu.
26. Phân tích tầm quan trọng của từng đặc trưng và xây dựng hàm dự đoán cho khách hàng mới.

3.2. CƠ SỞ LÝ THUYẾT

3.2.1. Bài toán dự đoán chi tiêu khách hàng trong phân tích khách hàng

Đây tiếp tục là một bài toán hồi quy (regression) giống báo cáo dự đoán giá nhà: đầu ra cần dự đoán — Total Spend — là một giá trị số thực dương (đơn vị USD), không phải một nhãn phân loại.

Do đó toàn bộ nền tảng lý thuyết về hàm kích hoạt linear ở lớp ra và hàm mất mát MSE đã trình bày ở báo cáo trước tiếp tục được áp dụng nguyên vẹn. Điểm mới về mặt ứng dụng là bài toán này thuộc lĩnh vực phân tích khách hàng (customer analytics) — một nhánh phổ biến của Data Science trong thương mại điện tử, thường được dùng để ước tính giá trị vòng đời khách hàng (Customer Lifetime Value — CLV), từ đó hỗ trợ ra quyết định marketing và chăm sóc khách hàng.

3.2.2. Ranh giới giữa Scikit-learn (tiền xử lý) và NumPy tự cài đặt (mô hình)

Notebook lần này là notebook đầu tiên trong chuỗi ba báo cáo có sử dụng Scikit-learn không chỉ để tính độ đo đánh giá (như ở báo cáo giá nhà) mà còn để thực hiện toàn bộ khâu tiền xử lý dữ liệu. Điều quan trọng cần làm rõ là ranh giới giữa hai phần này:

Thành phần	Công cụ sử dụng	Vai trò
Đọc dữ liệu, xử lý missing values	Pandas	Đọc CSV, kiểm tra và điền giá trị thiếu
Mã hóa biến phân loại	Scikit-learn (LabelEncoder)	Chuyển các cột dạng chữ (Gender, City...) thành số nguyên
Chia tập dữ liệu	Scikit-learn (train_test_split)	Thay thế cho việc tự viết np.random.permutation như ở hai báo cáo trước
Chuẩn hóa dữ liệu	Scikit-learn (StandardScaler)	Thay thế cho việc tự tính (X-mean)/std thủ công
Mô hình mạng nơ-ron (forward, backward, loss, gradient descent)	NumPy tự cài đặt	Phần "lõi" Deep Learning — không dùng bất kỳ hàm huấn luyện có sẵn nào của Scikit-learn/TensorFlow/PyTorch
Tính độ đo đánh giá cuối cùng	Scikit-learn (mean_squared_error, r2_score...)	Chỉ dùng để tính toán nhanh, không ảnh hưởng đến việc mô hình học như thế nào

Có thể thấy rõ: dù công cụ tiền xử lý đã thay đổi (từ tự viết bằng NumPy sang dùng Scikit-learn có sẵn), bản chất mạng nơ-ron — phần quan trọng nhất về mặt học thuật của cả ba báo cáo — vẫn hoàn toàn do người học tự cài đặt, không hề dùng bất kỳ hàm .fit() huấn luyện có sẵn nào của các framework Machine Learning/Deep Learning khác.

3.2.3. Mã hóa biến phân loại (Label Encoding) và hạn chế của nó

Tập dữ liệu khách hàng có 5 cột dạng phân loại (categorical): Gender, City, Membership Type, Satisfaction Level, và Discount Applied (dạng boolean True/False). Vì mạng nơ-ron chỉ có thể xử lý số thực, các cột này cần được mã hóa (encode) thành số trước khi đưa vào mô hình. Notebook sử dụng LabelEncoder của Scikit-learn — kỹ thuật gán mỗi giá trị phân loại khác nhau thành một số nguyên liên tiếp (ví dụ: Bronze → 0, Gold → 1, Silver → 2).

Cần lưu ý một hạn chế quan trọng của Label Encoding: kỹ thuật này ngầm định một thứ tự (ordinal) giữa các giá trị, dù trong thực tế nhiều biến phân loại ở đây là biến danh định (nominal) — không có thứ tự tự nhiên. Ví dụ, City được mã hóa thành Chicago=0, Houston=1, Los Angeles=2..., nhưng về bản chất giữa các thành phố không tồn tại quan hệ "lớn hơn/nhỏ hơn" nào — việc gán số 0,1,2,3... có thể vô tình khiến mạng nơ-ron học nhầm một mối quan hệ tuyến tính giả tạo giữa các thành phố (chẳng hạn ngộ nhận Miami "lớn hơn" Chicago một cách có ý nghĩa). Với biến Membership Type (Bronze/Silver/Gold) thì việc mã hóa theo thứ tự lại hợp lý hơn vì bản chất đây là biến có thứ tự (ordinal). Vấn đề này sẽ được phân tích sâu hơn ở Chương 9 và đề xuất cải tiến ở Chương 10.

3.2.4. Kiến trúc mạng nơ-ron sâu 5 lớp sử dụng

Kiến trúc mạng vẫn giữ nguyên độ sâu 5 lớp giống báo cáo giá nhà, chỉ khác ở số lượng đặc trưng đầu vào (9 đặc trưng thay vì 4, do đã bao gồm cả các biến phân loại đã mã hóa):

$$9 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$$

Lớp	Phép biến đổi	Kích thước	Hàm kích hoạt
Lớp 1 (ẩn)	$Z_1 = XW_1 + b_1$	$9 \rightarrow 64$	ReLU
Lớp 2 (ẩn)	$Z_2 = H_1W_2 + b_2$	$64 \rightarrow 32$	ReLU
Lớp 3 (ẩn)	$Z_3 = H_2W_3 + b_3$	$32 \rightarrow 16$	ReLU
Lớp 4 (ẩn)	$Z_4 = H_3W_4 + b_4$	$16 \rightarrow 8$	ReLU
Lớp 5 (đầu ra)	$Z_5 = H_4W_5 + b_5$	$8 \rightarrow 1$	Linear (identity)

Đáng chú ý, tổng số tham số huấn luyện của mạng này (ước tính khoảng $9 \times 64 + 64 \times 32 + 32 \times 16 + 16 \times 8 + 8 \times 1 +$ tổng bias ≈ 3.300 tham số) là khá lớn so với chỉ 280 mẫu huấn luyện hiện có — tỉ lệ tham số/mẫu cao hơn hẳn so với hai báo cáo trước (báo cáo tiểu đường có 17.524 mẫu huấn luyện, báo cáo giá nhà có 614/17.524 mẫu). Đây là một điểm cần lưu ý và sẽ được phân tích kỹ ở Chương 9, vì nó tiềm ẩn nguy cơ overfitting.

3.3. DỮ LIỆU VÀ TIỀN XỬ LÝ

Bộ dữ liệu sử dụng là E-commerce Customer Behavior (E-commerce_Customer_Behavior.csv), gồm 350 khách hàng với 11 cột thông tin, mô tả hành vi mua sắm và mức độ hài lòng của từng khách hàng.

STT	Cột	Kiểu dữ liệu	Ý nghĩa
1	Customer ID	Số nguyên	Mã định danh khách hàng (không dùng làm đặc trưng)

2	Gender	Phân loại	Giới tính (Female/Male)
3	Age	Số nguyên	Tuổi khách hàng
4	City	Phân loại	Thành phố sinh sống (6 thành phố)
5	Membership Type	Phân loại (có thứ tự)	Hạng thành viên: Bronze/Silver/Gold
6	Total Spend	Số thực (nhân)	Tổng chi tiêu (USD) — biến mục tiêu cần dự đoán
7	Items Purchased	Số nguyên	Số lượng sản phẩm đã mua
8	Average Rating	Số thực	Điểm đánh giá trung bình của khách hàng
9	Discount Applied	Boolean	Có áp dụng giảm giá hay không (True/False)
10	Days Since Last Purchase	Số nguyên	Số ngày kể từ lần mua gần nhất
11	Satisfaction Level	Phân loại	Mức độ hài lòng: Satisfied/Neutral/Unsatisfied

3.3.1. Xử lý giá trị thiếu (Missing Values)

Kiểm tra bằng `df.isnull().sum()` cho thấy toàn bộ dữ liệu chỉ có 2 giá trị thiếu, đều nằm ở cột Satisfaction Level. Các giá trị thiếu này được điền (impute) bằng nhãn 'Neutral' — một lựa chọn hợp lý vì đây là giá trị trung tính, tránh thiên lệch mô hình về hướng "hài lòng" hay "không hài lòng" một cách chủ quan khi thiếu thông tin thực tế.

3.3.2. Mã hóa biến phân loại

Năm cột phân loại (Gender, City, Membership Type, Satisfaction Level, Discount Applied) được mã hóa thành số bằng 5 đối tượng LabelEncoder riêng biệt:

Biến	Các giá trị (theo thứ tự mã hóa 0,1,2,...)
Gender	Female, Male
City	Chicago, Houston, Los Angeles, Miami, New York, San Francisco
Membership Type	Bronze, Gold, Silver
Satisfaction Level	Neutral, Satisfied, Unsatisfied

3.3.3. Lựa chọn đặc trưng (Feature Selection)

9 đặc trưng đầu vào được chọn để huấn luyện mô hình gồm 4 biến số gốc (Age, Items Purchased, Average Rating, Days Since Last Purchase) và 5 biến phân loại đã mã hóa (Gender_Encoded, City_Encoded, Membership_Encoded, Satisfaction_Encoded, Discount_Encoded). Cột Customer ID bị loại bỏ vì đây chỉ là mã định danh, không mang ý nghĩa dự đoán.

3.3.4. Chia tập dữ liệu và chuẩn hóa

Khác với hai báo cáo trước (tự viết np.random.permutation), notebook này dùng hàm train_test_split() có sẵn của Scikit-learn để chia 350 mẫu theo tỉ lệ 80:20 (test_size=0.2, random_state=42), cho ra 280 mẫu huấn luyện và 70 mẫu kiểm thử. Việc chuẩn hóa cũng được thực hiện bằng StandardScaler của Scikit-learn thay vì tự tính (X-mean)/std, nhưng về bản chất toán học là hoàn toàn tương đương: StandardScaler.fit_transform() trên tập huấn luyện học ra mean và std rồi áp dụng, còn .transform() trên tập kiểm thử chỉ áp dụng lại đúng mean/std đã học (không tính lại) — nguyên tắc tránh rò rỉ thông tin (data leakage) vẫn được tuân thủ đúng như hai báo cáo trước, chỉ khác về công cụ thực hiện. Cả đặc trưng X và nhãn y (Total Spend) đều được chuẩn hóa bằng hai đối tượng StandardScaler riêng biệt (scaler_X và scaler_y).

3.4. PHƯƠNG PHÁP XÂY DỰNG MÔ HÌNH

Toàn bộ phần phương pháp toán học của mạng nơ-ron 5 lớp trong báo cáo này hoàn toàn giống với báo cáo dự đoán giá nhà (cùng kiến trúc 5 lớp, cùng hàm mất mát MSE, cùng hàm kích hoạt linear ở lớp ra), chỉ khác về số chiều đầu vào (9 thay vì 4). Phần này trình bày lại ngắn gọn để báo cáo có tính tự chứa (self-contained).

3.4.1. Lan truyền tiến (Forward Propagation)

$$Z_1 = XW_1 + b_1, H_1 = \text{ReLU}(Z_1)$$

$$Z_2 = H_1W_2 + b_2, H_2 = \text{ReLU}(Z_2)$$

$$Z_3 = H_2W_3 + b_3, H_3 = \text{ReLU}(Z_3)$$

$$Z_4 = H_3W_4 + b_4, H_4 = \text{ReLU}(Z_4)$$

$$Z_5 = H_4W_5 + b_5, \hat{y} = Z_5 \text{ (linear)}$$

Với $X \in \mathbb{R}^{N \times 9}$ (N mẫu, 9 đặc trưng), các ma trận trọng số có kích thước $W_1 \in \mathbb{R}^{9 \times 64}$, $W_2 \in \mathbb{R}^{64 \times 32}$, $W_3 \in \mathbb{R}^{32 \times 16}$, $W_4 \in \mathbb{R}^{16 \times 8}$, $W_5 \in \mathbb{R}^{8 \times 1}$.

3.4.2. Hàm mất mát MSE (Mean Squared Error)

$$L = (1/N) \sum_i (y_i - \hat{y}_i)^2$$

Vì Total Spend là một giá trị số thực dương liên tục (đơn vị USD), MSE tiếp tục là lựa chọn phù hợp, đo độ lệch bình phương trung bình giữa chỉ tiêu dự đoán và chỉ tiêu thực tế trên thang đo đã chuẩn hóa.

3.4.3. Lan truyền ngược (Backpropagation)

$$\partial L / \partial Z_5 = 2(\hat{y} - y) / N$$

$$dW_5 = H_4^T \cdot dZ_5, dH_4 = dZ_5 \cdot W_5^T, dZ_4 = dH_4 \odot \text{ReLU}'(Z_4)$$

$$dW_4 = H_3^T \cdot dZ_4, dH_3 = dZ_4 \cdot W_4^T, dZ_3 = dH_3 \odot \text{ReLU}'(Z_3)$$

$$dW_3 = H_2^T \cdot dZ_3, dH_2 = dZ_3 \cdot W_3^T, dZ_2 = dH_2 \odot \text{ReLU}'(Z_2)$$

$$dW_2 = H_1^T \cdot dZ_2, dH_1 = dZ_2 \cdot W_2^T, dZ_1 = dH_1 \odot \text{ReLU}'(Z_1)$$

$$dW_1 = X^T \cdot dZ_1$$

Cấu trúc lan truyền ngược hoàn toàn tương tự báo cáo giá nhà: mỗi lớp ẩn đều lặp lại đúng ba bước — tính dW/db của lớp hiện tại, lan truyền dH sang lớp trước, nhân với đạo hàm ReLU.

3.4.4. Cập nhật trọng số (Gradient Descent)

$$W \leftarrow W - \eta \cdot (\partial L / \partial W) ; b \leftarrow b - \eta \cdot (\partial L / \partial b)$$

Với learning rate $\eta = 0.005$ và 3000 epoch, giống hết cấu hình đã dùng ở báo cáo giá nhà — cho thấy người thực hiện đã tái sử dụng lại đúng một khung huấn luyện (training pipeline) đã được kiểm chứng, chỉ thay đổi dữ liệu đầu vào và số chiều đặc trưng.

3.5. CÀI ĐẶT VÀ HUẤN LUYỆN MÔ HÌNH

Mô hình được cài đặt với sự phân chia rõ ràng giữa hai phần: phần tiền xử lý dữ liệu tận dụng Scikit-learn (LabelEncoder, StandardScaler, train_test_split) để rút gọn code và giảm khả năng sai sót so với tự viết thủ công; phần mô hình (mạng nơ-ron) vẫn hoàn toàn tự cài đặt bằng NumPy, không dùng bất kỳ lớp mô hình có sẵn nào (như sklearn.neural_network.MLPRegressor hay các framework Deep Learning). Các thành phần chính bao gồm:

- Đọc dữ liệu bằng Pandas, kiểm tra và xử lý giá trị thiếu ở cột Satisfaction Level.
- Mã hóa 5 biến phân loại bằng 5 đối tượng LabelEncoder riêng biệt.
- Chọn 9 đặc trưng đầu vào, chia tập huấn luyện/kiểm thử bằng train_test_split (280/70 mẫu).
- Chuẩn hóa cả X và y bằng hai đối tượng StandardScaler riêng biệt (scaler_X, scaler_y).
- Khởi tạo 10 tham số ($W_1..W_5, b_1..b_5$) theo He initialization cho kiến trúc $9 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$.
- Hàm forward(X), backward(y , cache), mse_loss, mse_derivative — hoàn toàn giống cấu trúc đã dùng ở báo cáo giá nhà.
- Vòng lặp huấn luyện 3000 epoch, learning rate $\eta = 0.005$, có theo dõi thêm train loss riêng biệt tại mỗi mốc in log (điểm mới so với hai báo cáo trước).

- So sánh với mô hình Linear Regression của Scikit-learn để đánh giá khách quan mức độ cải thiện của mạng nơ-ron sâu.
- Hàm `predict_customer(customer_data)` nhận vào một dictionary thông tin khách hàng (bao gồm cả các trường dạng chữ), tự động mã hóa và chuẩn hóa, rồi trả về dự đoán Total Spend.

Một điểm mới đáng chú ý so với hai báo cáo trước: notebook này import thêm thư viện Seaborn (`import seaborn as sns`) ở Cell 1 nhưng trên thực tế không có cell nào sử dụng thư viện này để trực quan hóa — có thể đây là một import dự phòng cho việc phân tích khám phá dữ liệu (EDA) mà người thực hiện dự định làm nhưng chưa hoàn thiện trong phiên bản notebook hiện tại.

3.6. PHÂN TÍCH CHI TIẾT TỪNG CELL CODE TRONG JUPYTER NOTEBOOK

Chương này trình bày lại toàn bộ notebook `DL_customer.ipynb` theo đúng thứ tự các cell đã chạy, kèm giải thích ý nghĩa từng dòng lệnh và output thực tế thu được.

3.6.1. Cell 1 — Nạp thư viện

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Thiết lập seed
np.random.seed(42)

print("✅ Import thành công!")
```

Output:

```
✅ Import thành công!
```

Đây là notebook có danh sách import phong phú nhất trong ba báo cáo: ngoài NumPy, Pandas, Matplotlib quen thuộc, còn có Seaborn (thư viện trực quan hóa nâng cao hơn Matplotlib) và ba nhóm công cụ từ Scikit-learn: `preprocessing` (`LabelEncoder`, `StandardScaler`) để tiền xử lý; `model_selection` (`train_test_split`) để chia dữ liệu; và `metrics` (`mean_squared_error`, `mean_absolute_error`, `r2_score`) để đánh giá. Cần lưu ý: toàn bộ các công cụ Scikit-learn này chỉ hỗ trợ ở khâu tiền xử lý và đánh giá — bản thân mạng nơ-ron dự đoán vẫn được tự cài đặt bằng NumPy ở các cell sau.

3.6.2. Cell 2 — Đọc dữ liệu và khảo sát tổng quan

```
df = pd.read_csv("C:/Users/Admin/Downloads/Test/E-commerce_Customer_Behavior.csv")

print("📊 Thông tin dataset:")
print(f"Shape: {df.shape}")
print("\n📄 5 dòng đầu tiên:")
print(df.head())
print("\n📋 Thông tin các cột:")
print(df.info())
print("\n📊 Thống kê mô tả:")
print(df.describe())
```

Output (rút gọn):

```
Shape: (350, 11)
...
#   Column                Non-Null Count  Dtype
0   Customer ID           350 non-null    int64
1   Gender                350 non-null    str
2   Age                   350 non-null    int64
3   City                  350 non-null    str
4   Membership Type       350 non-null    str
5   Total Spend            350 non-null    float64
6   Items Purchased       350 non-null    int64
7   Average Rating        350 non-null    float64
8   Discount Applied      350 non-null    bool
9   Days Since Last Purchase 350 non-null    int64
10  Satisfaction Level    348 non-null    str
...
      Total Spend  ...
mean    845.38  ...
std     362.06  ...
min     410.80  ...
max     1520.10  ...
```

`df.head()` hiển thị 5 dòng đầu để kiểm tra nhanh định dạng dữ liệu; `df.info()` cho biết kiểu dữ liệu và số lượng giá trị không rỗng (non-null) của từng cột — đây chính là bước phát hiện ra cột Satisfaction Level chỉ có 348/350 giá trị (thiếu 2 giá trị). `df.describe()` tính các thống kê mô tả cơ bản (trung bình, độ lệch chuẩn, min, max, các phân vị) cho các cột số, giúp có cái nhìn tổng quan về thang đo và độ phân tán của dữ liệu trước khi xây dựng mô hình — ví dụ Total Spend dao động từ 410.8 đến 1520.1 USD, trung bình khoảng 845.4 USD.

3.6.3. Cell 3 — Xử lý missing values và mã hóa biến phân loại

```

print("🔍 Kiểm tra missing values:")
print(df.isnull().sum())

df['Satisfaction Level'] = df['Satisfaction Level'].fillna('Neutral')

le_gender = LabelEncoder()
le_city = LabelEncoder()
le_membership = LabelEncoder()
le_satisfaction = LabelEncoder()
le_discount = LabelEncoder()

df['Gender_Encoded'] = le_gender.fit_transform(df['Gender'])
df['City_Encoded'] = le_city.fit_transform(df['City'])
df['Membership_Encoded'] = le_membership.fit_transform(df['Membership Type'])
df['Satisfaction_Encoded'] = le_satisfaction.fit_transform(df['Satisfaction Level'])
df['Discount_Encoded'] = le_discount.fit_transform(df['Discount Applied'])

print("✅ Đã encode categorical variables")
print(f"Gender: {list(le_gender.classes_)}")
print(f"City: {list(le_city.classes_)}")
print(f"Membership: {list(le_membership.classes_)}")
print(f"Satisfaction: {list(le_satisfaction.classes_)}")

```

Output:

```

🔍 Kiểm tra missing values:
Satisfaction Level      2
(các cột khác đều bằng 0)

✅ Đã encode categorical variables
Gender: ['Female', 'Male']
City: ['Chicago', 'Houston', 'Los Angeles', 'Miami', 'New York', 'San Francisco']
Membership: ['Bronze', 'Gold', 'Silver']
Satisfaction: ['Neutral', 'Satisfied', 'Unsatisfied']

```

`df.isnull().sum()` đếm số giá trị thiếu (NaN) theo từng cột, xác nhận chỉ có 2 giá trị thiếu ở Satisfaction Level. Dòng `fillna('Neutral')` thay thế các giá trị thiếu này bằng nhãn 'Neutral' — một lựa chọn hợp lý về mặt nghiệp vụ vì đây là mức trung tính, không thiên về hài lòng hay không hài lòng.

Năm đối tượng `LabelEncoder()` riêng biệt được tạo cho 5 cột phân loại khác nhau — mỗi `LabelEncoder` chỉ nên dùng cho một cột duy nhất vì nó ghi nhớ (fit) một bảng ánh xạ (mapping) riêng cho tập giá trị của cột đó. Phương thức `.fit_transform()` thực hiện đồng thời hai việc: fit (học ra danh sách các giá trị duy nhất và gán số thứ tự) và transform (áp dụng ánh xạ để chuyển cột dữ liệu thành số nguyên). Thuộc tính `.classes_` cho biết chính xác thứ tự ánh xạ đã học được — ví dụ với `le_city`, thứ tự bằng chữ cái Chicago-Houston-Los Angeles-Miami-New York-San Francisco tương ứng với các mã 0-1-2-3-4-5.

3.6.4. Cell 4 — Chọn đặc trưng đầu vào

```
feature_cols = [
    'Age',
    'Items Purchased',
    'Average Rating',
    'Days Since Last Purchase',
    'Gender_Encoded',
    'City_Encoded',
    'Membership_Encoded',
    'Satisfaction_Encoded',
    'Discount_Encoded'
]

X = df[feature_cols].values
y = df['Total Spend'].values.reshape(-1, 1)

print(f" Target shape: {y.shape}")
print(f" Features: {feature_cols}")
```

Output:

```
 Features shape: (350, 9)
 Target shape: (350, 1)

 Features: ['Age', 'Items Purchased', 'Average Rating', 'Days Since Last Purchase', 'Gender_Encoded', 'City_Encoded', 'Membership_Encoded', 'Satisfaction_Encoded', 'Discount_Encoded']
```

Danh sách feature_cols chọn ra đúng 9 cột sẽ dùng làm đặc trưng đầu vào — kết hợp 4 cột số gốc và 5 cột phân loại đã mã hóa ở Cell 3. df[feature_cols].values trích các cột này theo đúng thứ tự trong danh sách rồi chuyển thành mảng NumPy (350, 9). Đây là bước quan trọng vì thứ tự các cột trong feature_cols sẽ quyết định thứ tự các cột trong X — thứ tự này phải được giữ nhất quán xuyên suốt notebook, đặc biệt khi dự đoán cho dữ liệu mới ở Cell 17.

3.6.5. Cell 5 — Chia tập huấn luyện/kiểm thử bằng Scikit-learn

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f" Training samples: {len(X_train)}")
print(f" Testing samples: {len(X_test)}")
```

Output:

```
 Training samples: 280
 Testing samples: 70
```

train_test_split() của Scikit-learn thực hiện đúng công việc mà hai báo cáo trước phải tự viết bằng np.random.permutation + slicing thủ công: xáo trộn ngẫu nhiên rồi chia dữ liệu theo tỉ lệ chỉ định. test_size=0.2 giữ lại 20% làm tập kiểm thử (70/350 mẫu), phần còn lại 80% (280 mẫu) làm tập huấn

luyện. Tham số `random_state=42` đóng vai trò tương đương `np.random.seed(42)` — cố định cách chia để có thể tái lập kết quả giữa các lần chạy.

3.6.6. Cell 6 — Chuẩn hóa dữ liệu bằng `StandardScaler`

```
scaler_X = StandardScaler()
X_train = scaler_X.fit_transform(X_train)
X_test = scaler_X.transform(X_test)

scaler_y = StandardScaler()
y_train = scaler_y.fit_transform(y_train)
y_test = scaler_y.transform(y_test)

print("✅ Đã chuẩn hóa dữ liệu")
print(f"Mean y_train: {scaler_y.mean_[0]:.2f}")
print(f"Std y_train: {scaler_y.scale_[0]:.2f}")
```

Output:

```
✅ Đã chuẩn hóa dữ liệu
Mean y_train: 830.64
Std y_train: 358.47
```

`StandardScaler().fit_transform(X_train)` học ra mean và độ lệch chuẩn của tập huấn luyện rồi áp dụng ngay để chuẩn hóa; `scaler_X.transform(X_test)` (không có `fit`) áp dụng lại đúng mean/std đã học đó cho tập kiểm thử — về bản chất đây chính là công thức $(X - \text{mean}) / \text{std}$ đã tự viết ở hai báo cáo trước, chỉ khác là được Scikit-learn đóng gói sẵn thành một đối tượng có thể tái sử dụng (`scaler_X` có thể lưu lại và dùng cho dữ liệu mới sau này, như sẽ thấy ở Cell 17).

Tương tự, `scaler_y` chuẩn hóa riêng cho nhãn `Total Spend`. Hai thuộc tính `scaler_y.mean_[0]` và `scaler_y.scale_[0]` cho biết chính xác giá trị trung bình (830.64 USD) và độ lệch chuẩn (358.47 USD) của `Total Spend` trên tập huấn luyện — hai con số này sẽ được dùng ngầm bên trong `scaler_y.inverse_transform()` ở các cell sau để chuyển kết quả dự đoán từ thang đo chuẩn hóa trở lại đơn vị USD.

3.6.7. Cell 7 — Định nghĩa các hàm toán học

```
def relu(x):
    return np.maximum(0, x)

def relu_derivative(x):
    return (x > 0).astype(float)

def linear(x):
    return x

def mse_loss(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)
```

```
def mse_derivative(y_true, y_pred):
    return 2 * (y_pred - y_true) / len(y_true)
```

Nhắm hàm này giống hệt về mặt code với báo cáo dự đoán giá nhà — một minh chứng nữa cho việc phần "lỗi toán học" của Deep Learning hoàn toàn độc lập với việc dữ liệu đến từ lĩnh vực nào (bất động sản hay hành vi khách hàng) hay được tiền xử lý bằng công cụ gì (tự viết NumPy hay Scikit-learn). Giải thích chi tiết từng hàm ở Chương 7.

3.6.8. Cell 8 — Khởi tạo mạng nơ-ron sâu 5 lớp (9 → 64 → 32 → 16 → 8 → 1)

```
input_size = X_train.shape[1] # 9 features
hidden1_size = 64
hidden2_size = 32
hidden3_size = 16
hidden4_size = 8
output_size = 1

W1 = np.random.randn(input_size, hidden1_size) * np.sqrt(2.0 / input_size)
b1 = np.zeros((1, hidden1_size))
W2 = np.random.randn(hidden1_size, hidden2_size) * np.sqrt(2.0 / hidden1_size)
b2 = np.zeros((1, hidden2_size))
W3 = np.random.randn(hidden2_size, hidden3_size) * np.sqrt(2.0 / hidden2_size)
b3 = np.zeros((1, hidden3_size))
W4 = np.random.randn(hidden3_size, hidden4_size) * np.sqrt(2.0 / hidden3_size)
b4 = np.zeros((1, hidden4_size))
W5 = np.random.randn(hidden4_size, output_size) * np.sqrt(2.0 / hidden4_size)
b5 = np.zeros((1, output_size))

print("✅ Khởi tạo 5-layer network thành công!")
print(f"📊 Input: {input_size} → {hidden1_size} → {hidden2_size} → {hidden3_size} → {hidden4_size} → {output_size}")
print(f"📊 W1: {W1.shape}, b1: {b1.shape}")
print(f"📊 W2: {W2.shape}, b2: {b2.shape}")
print(f"📊 W3: {W3.shape}, b3: {b3.shape}")
print(f"📊 W4: {W4.shape}, b4: {b4.shape}")
print(f"📊 W5: {W5.shape}, b5: {b5.shape}")
```

Output:

```
✅ Khởi tạo 5-layer network thành công!
📊 Input: 9 → 64 → 32 → 16 → 8 → 1

📊 W1: (9, 64), b1: (1, 64)
📊 W2: (64, 32), b2: (1, 32)
📊 W3: (32, 16), b3: (1, 16)
📊 W4: (16, 8), b4: (1, 8)
📊 W5: (8, 1), b5: (1, 1)
```

Điểm khác biệt nhỏ nhưng đáng chú ý so với báo cáo giá nhà: `input_size = X_train.shape[1]` được lấy động từ dữ liệu thực tế thay vì gán cứng bằng số 4 — đây là một thói quen viết code tốt hơn, giúp code tự động thích ứng nếu số lượng đặc trưng thay đổi (ví dụ nếu sau này thêm/bớt cột

trong `feature_cols`) mà không cần sửa lại dòng khởi tạo `W1`. Toàn bộ công thức He initialization còn lại giữ nguyên như hai báo cáo trước.

3.6.9. Cell 9 — Hàm `forward(X)`

```
def forward(X):
    z1 = X @ W1 + b1; h1 = relu(z1)
    z2 = h1 @ W2 + b2; h2 = relu(z2)
    z3 = h2 @ W3 + b3; h3 = relu(z3)
    z4 = h3 @ W4 + b4; h4 = relu(z4)
    z5 = h4 @ W5 + b5; y_hat = linear(z5)
    cache = {"x": X, "z1": z1, "h1": h1, "z2": z2, "h2": h2,
            "z3": z3, "h3": h3, "z4": z4, "h4": h4,
            "z5": z5, "y_hat": y_hat}
    return y_hat, cache

y_hat_test, _ = forward(X_train[:5])
print(f"✅ Forward test: {y_hat_test.shape}")
```

Output:

```
✅ Forward test: (5, 1)
```

Hàm `forward()` giống hệt cấu trúc ở báo cáo giá nhà — chỉ khác kích thước ma trận đầu vào (`X` có 9 cột thay vì 4). Đây là minh chứng rõ ràng cho tính tổng quát hóa của kiến trúc: cùng một đoạn code `forward()`, khi thay `input_size`, hoàn toàn có thể xử lý bất kỳ bài toán hồi quy nào có số lượng đặc trưng khác nhau mà không cần viết lại logic.

3.6.10. Cell 10 — Hàm `backward(y, cache)`

```

def backward(y, cache):
    X = cache["x"]
    z1, h1 = cache["z1"], cache["h1"]
    z2, h2 = cache["z2"], cache["h2"]
    z3, h3 = cache["z3"], cache["h3"]
    z4, h4 = cache["z4"], cache["h4"]
    yhat = cache["y_hat"]
    n = len(X)

    dz5 = mse_derivative(y, yhat)
    dW5 = h4.T @ dz5; db5 = np.sum(dz5, axis=0, keepdims=True)

    dh4 = dz5 @ W5.T
    dz4 = dh4 * relu_derivative(z4)
    dW4 = h3.T @ dz4; db4 = np.sum(dz4, axis=0, keepdims=True)

    dh3 = dz4 @ W4.T
    dz3 = dh3 * relu_derivative(z3)
    dW3 = h2.T @ dz3; db3 = np.sum(dz3, axis=0, keepdims=True)

    dh2 = dz3 @ W3.T
    dz2 = dh2 * relu_derivative(z2)
    dW2 = h1.T @ dz2; db2 = np.sum(dz2, axis=0, keepdims=True)

    dh1 = dz2 @ W2.T
    dz1 = dh1 * relu_derivative(z1)
    dW1 = X.T @ dz1; db1 = np.sum(dz1, axis=0, keepdims=True)

    gradients = {"dW1": dW1, "db1": db1, "dW2": dW2, "db2": db2,
                  "dW3": dW3, "db3": db3, "dW4": dW4, "db4": db4,
                  "dW5": dW5, "db5": db5}
    return gradients

grads = backward(y_train[:5], forward(X_train[:5])[1])
print("✅ Backward test thành công!")
for key in grads:
    print(f"📐 {key}: {grads[key].shape}")

```

Output:

```

✅ Backward test thành công!
📐 dW1: (9, 64)
📐 db1: (1, 64)
📐 dW2: (64, 32)
📐 db2: (1, 32)
📐 dW3: (32, 16)
📐 db3: (1, 16)
📐 dW4: (16, 8)
📐 db4: (1, 8)
📐 dW5: (8, 1)
📐 db5: (1, 1)

```

Hàm backward() cũng giữ nguyên cấu trúc như báo cáo giá nhà, chỉ khác dW1 có kích thước (9, 64) thay vì (4, 64) do X.T giờ có 9 hàng thay vì 4. Giải thích chi tiết công thức ở Chương 7.

3.6.11. Cell 11 — Vòng lặp huấn luyện (có theo dõi thêm Train Loss)

```
learning_rate = 0.005
epochs = 3000
loss_history = []
train_loss_history = []

print("🚀 Bắt đầu training 5-layer network...")
print("-" * 70)

for epoch in range(epochs):
    y_hat, cache = forward(X_train)
    loss = mse_loss(y_train, y_hat)
    loss_history.append(loss)
    gradients = backward(y_train, cache)

    W1 -= learning_rate * gradients["dW1"]; b1 -= learning_rate * gradients["db1"]
    W2 -= learning_rate * gradients["dW2"]; b2 -= learning_rate * gradients["db2"]
    W3 -= learning_rate * gradients["dW3"]; b3 -= learning_rate * gradients["db3"]
    W4 -= learning_rate * gradients["dW4"]; b4 -= learning_rate * gradients["db4"]
    W5 -= learning_rate * gradients["dW5"]; b5 -= learning_rate * gradients["db5"]

    if epoch % 200 == 0:
        train_loss = mse_loss(y_train, forward(X_train)[0])
        print(f"Epoch {epoch:5d} | Loss = {loss:.6f} | Train Loss = {train_loss:.6f}")

print("-" * 70)
print(f"✅ Training hoàn tất! Final Loss: {loss_history[-1]:.6f}")
```

Output:

```
🚀 Bắt đầu training 5-layer network...
-----
Epoch   0 | Loss = 0.770169 | Train Loss = 0.411297
Epoch 200 | Loss = 0.012599 | Train Loss = 0.012570
Epoch 400 | Loss = 0.008621 | Train Loss = 0.008607
Epoch 600 | Loss = 0.006446 | Train Loss = 0.006438
Epoch 800 | Loss = 0.005132 | Train Loss = 0.005126
Epoch 1000 | Loss = 0.004261 | Train Loss = 0.004257
Epoch 1200 | Loss = 0.003656 | Train Loss = 0.003654
Epoch 1400 | Loss = 0.003214 | Train Loss = 0.003212
Epoch 1600 | Loss = 0.002824 | Train Loss = 0.002822
Epoch 1800 | Loss = 0.002554 | Train Loss = 0.002553
Epoch 2000 | Loss = 0.002344 | Train Loss = 0.002343
Epoch 2200 | Loss = 0.002166 | Train Loss = 0.002165
Epoch 2400 | Loss = 0.002021 | Train Loss = 0.002020
Epoch 2600 | Loss = 0.001895 | Train Loss = 0.001894
Epoch 2800 | Loss = 0.001788 | Train Loss = 0.001787
-----
```

✓ Training hoàn tất! Final Loss: 0.001696

Cấu hình huấn luyện (`learning_rate = 0.005`, `epochs = 3000`) giống hệt báo cáo giá nhà. Điểm mới ở cell này là việc tính thêm `train_loss = mse_loss(y_train, forward(X_train)[0])` ngay bên trong điều kiện `in log` — về bản chất `train_loss` ở đây tính trên chính tập huấn luyện giống hệt biến `loss` đã tính ở trên, nên hai giá trị luôn xấp xỉ bằng nhau (ví dụ epoch 200: `Loss = 0.012599` $\approx$ `Train Loss = 0.012570`, chênh lệch cực nhỏ do gradient descent đã cập nhật trọng số giữa hai lần tính). Đây thực chất là một phép tính dư thừa (`loss` và `train_loss` đang đo cùng một đại lượng) — một thiết kế thường thấy khi lẽ ra nên tính thêm validation loss trên tập kiểm thử `X_test` để theo dõi overfitting, nhưng ở đây lại vô tình tính lại loss trên chính tập huấn luyện.

Về giá trị `loss`: khác biệt rất lớn so với hai báo cáo trước, `loss` giảm cực nhanh và cực sâu — chỉ sau 200 epoch đã giảm từ 0.7702 xuống còn 0.0126 (giảm hơn 60 lần), và sau 3000 epoch chỉ còn 0.0017. Đây là một con số `loss` cực thấp trên thang đo đã chuẩn hóa, báo hiệu mô hình khớp (fit) dữ liệu huấn luyện gần như hoàn hảo — cần được phân tích cẩn thận ở Chương 9 vì đây có thể vừa là tín hiệu tốt (mô hình học tốt) vừa là dấu hiệu cảnh báo (nguy cơ overfitting nghiêm trọng, đặc biệt khi tập huấn luyện chỉ có 280 mẫu nhưng mạng có tới hàng nghìn tham số).

3.6.12. Cell 12 — Vẽ biểu đồ đường cong Loss

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(loss_history)
plt.xlabel("Epoch"); plt.ylabel("Loss (MSE)")
plt.title("Training Loss - 5 Layer Network"); plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(loss_history[-200:])
plt.xlabel("Epoch (last 200)"); plt.ylabel("Loss (MSE)")
plt.title("Training Loss (Zoom)"); plt.grid(True)

plt.tight_layout()
plt.show()
```

Cấu trúc vẽ biểu đồ giống hệt báo cáo giá nhà: một biểu đồ tổng thể toàn bộ 3000 epoch và một biểu đồ zoom vào 200 epoch cuối. Với dữ liệu khách hàng, do `loss` giảm rất sâu ngay từ đầu, biểu đồ tổng thể sẽ cho thấy đường cong giảm gần như thẳng đứng ngay ở những epoch đầu tiên rồi đi ngang gần trục hoành trong phần lớn thời gian còn lại — biểu đồ zoom giúp quan sát rõ hơn xu hướng giảm rất nhỏ vẫn đang tiếp diễn ở giai đoạn cuối.

3.6.13. Cell 13 — Dự đoán và tính các độ đo (bao gồm cả MAPE)

```

y_pred_normalized, _ = forward(X_test)
y_pred = scaler_y.inverse_transform(y_pred_normalized)
y_test_original = scaler_y.inverse_transform(y_test)

mse = mean_squared_error(y_test_original, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test_original, y_pred)
r2 = r2_score(y_test_original, y_pred)

print("=" * 60)
print("📊 KẾT QUẢ TRÊN TEST SET")
print("=" * 60)
print(f"📉 MSE: {mse:.4f}")
print(f"📉 RMSE: {rmse:.4f} (USD)")
print(f"📊 MAE: {mae:.4f} (USD)")
print(f"📈 R²: {r2:.4f}")

mape = np.mean(np.abs((y_test_original - y_pred) / y_test_original)) * 100
print(f"📊 MAPE: {mape:.2f}%")

```

Output:

```

=====
📊 KẾT QUẢ TRÊN TEST SET
=====

```

```

📉 MSE: 291.2805
📉 RMSE: 17.0669 (USD)
📊 MAE: 11.3758 (USD)
📈 R²: 0.9978
📊 MAPE: 1.46%

```

Điểm khác biệt về công cụ: `scaler_y.inverse_transform()` được dùng thay cho phép tính thủ công ($y * y_std + y_mean$) đã làm ở báo cáo giá nhà — về bản chất là cùng một phép biến đổi ngược, chỉ do `StandardScaler` tự động ghi nhớ `mean/std` nên không cần truyền tham số thủ công.

Điểm mới thứ hai là độ đo MAPE (Mean Absolute Percentage Error) — sai số phần trăm tuyệt đối trung bình, tính bằng trung bình của $|thực\ tế - dự\ đoán| / thực\ tế$, nhân 100 để ra phần trăm. MAPE hữu ích hơn RMSE/MAE trong nhiều tình huống kinh doanh vì nó biểu diễn sai số dưới dạng phần trăm (dễ diễn giải: "mô hình sai lệch trung bình 1.46% so với chi tiêu thực tế") thay vì đơn vị tuyệt đối (USD) vốn khó so sánh giữa các khách hàng có mức chi tiêu rất khác nhau.

Các chỉ số ở đây cực kỳ ấn tượng: $R^2 = 0.9978$ (gần như tuyệt đối), RMSE chỉ 17.07 USD trên nền chi tiêu trung bình hơn 845 USD, MAPE chỉ 1.46%. So với báo cáo giá nhà (R^2 chỉ 0.31), đây là một bước nhảy vọt về độ chính xác — cần được phân tích kỹ ở Chương 9 để hiểu rõ nguyên nhân đằng sau kết quả cao bất thường này.

3.6.14. Cell 14 — Biểu đồ Predictions vs Actual

```
plt.figure(figsize=(10, 6))
plt.scatter(y_test_original, y_pred, alpha=0.6, label='Predictions')

min_val = min(y_test_original.min(), y_pred.min())
max_val = max(y_test_original.max(), y_pred.max())
plt.plot([min_val, max_val], [min_val, max_val], 'r--', label='Perfect Prediction')

plt.xlabel("Total Spend Thực tế (USD)")
plt.ylabel("Total Spend Dự đoán (USD)")
plt.title(f"5-Layer Network: Predictions vs Actual (R² = {r2:.4f})")
plt.legend(); plt.grid(True)
plt.show()
```

Với R^2 gần bằng 1, biểu đồ scatter này được kỳ vọng sẽ cho thấy hầu hết các điểm bám rất sát vào đường "Perfect Prediction" ($y = x$), khác biệt rõ rệt so với biểu đồ tương ứng ở báo cáo giá nhà (nơi các điểm phân tán rộng do R^2 chỉ đạt 0.31).

3.6.15. Cell 15 — Phân tích Residuals

```
residuals = y_test_original.flatten() - y_pred.flatten()

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.scatter(y_pred, residuals, alpha=0.6)
plt.axhline(y=0, color='r', linestyle='--', label='Zero Error')
plt.xlabel("Giá dự đoán (USD)"); plt.ylabel("Residuals (Sai số)")
plt.title("Residual Plot"); plt.legend(); plt.grid(True)

plt.subplot(1, 2, 2)
plt.hist(residuals, bins=30, edgecolor='black', alpha=0.7)
plt.xlabel("Sai số (USD)"); plt.ylabel("Tần suất")
plt.title("Distribution of Residuals"); plt.grid(True)

plt.tight_layout()
plt.show()

print("📊 Thống kê Residuals:")
print(f"Mean: {np.mean(residuals):.4f}")
print(f"Std: {np.std(residuals):.4f}")
print(f"Min: {np.min(residuals):.4f}")
print(f"Max: {np.max(residuals):.4f}")
```

Output:

```
📊 Thống kê Residuals:
Mean: 1.9811
Std: 16.9516
Min: -28.4431
Max: 89.7568
```

Thống kê residuals cho thấy: sai số trung bình gần 0 (1.98 USD, rất nhỏ so với thang chi tiêu hàng trăm USD) và độ lệch chuẩn residual chỉ 16.95 USD — rất nhỏ so với độ lệch chuẩn tổng thể của

Total Spend (358.47 USD, đã tính ở Cell 6). Tuy nhiên giá trị Max lên tới 89.76 USD, lớn hơn hẳn phần còn lại của phân phối (Std chỉ 16.95), cho thấy tồn tại ít nhất một vài khách hàng ngoại lai (outlier) mà mô hình dự đoán sai lệch nhiều hơn đáng kể so với đa số các khách hàng còn lại — đáng để kiểm tra riêng những trường hợp này trong thực tế triển khai.

3.6.16. Cell 16 — Phân tích tầm quan trọng đặc trưng (Feature Importance)

```
n_samples = min(100, X_test.shape[0])
importance = []

print(f" Feature Importance (%):")
for i, idx in enumerate(sorted_idx):
    pct = (importance[idx] / total_importance) * 100
    print(f" {i+1}. {feature_cols[idx]}: {pct:.2f}%")
```

Output:

 Phân tích feature importance với 70 samples...

 Feature Importance Ranking:

1. Items Purchased: 0.0030
2. City_Encoded: 0.0023
3. Average Rating: 0.0021
4. Gender_Encoded: 0.0009
5. Days Since Last Purchase: 0.0007
6. Membership_Encoded: 0.0007
7. Satisfaction_Encoded: 0.0006
8. Age: 0.0004
9. Discount_Encoded: 0.0004

 Feature Importance (%):

1. Items Purchased: 27.03%
2. City_Encoded: 20.58%
3. Average Rating: 18.84%
4. Gender_Encoded: 8.15%
5. Days Since Last Purchase: 6.62%
6. Membership_Encoded: 6.26%
7. Satisfaction_Encoded: 5.48%
8. Age: 3.70%
9. Discount_Encoded: 3.35%

Kỹ thuật permutation/perturbation importance được áp dụng giống hết báo cáo giá nhà: thêm nhiễu nhỏ vào từng đặc trưng và đo mức tăng loss tương ứng. Điểm khác biệt tích cực so với báo cáo giá nhà là ở đây $n_samples = \min(100, X_test.shape[0])$ — dòng lệnh này tự động giới hạn số mẫu thử theo đúng kích thước thực tế của tập kiểm thử (70 mẫu, nhỏ hơn 100), tránh lỗi truy cập chỉ số ngoài phạm vi mảng nếu tập kiểm thử nhỏ hơn 100 mẫu — một cải tiến nhỏ nhưng thể hiện sự cẩn thận hơn trong việc viết code so với báo cáo trước (nơi $n_samples = 100$ được gán cứng).

Kết quả xếp hạng cho thấy Items Purchased (số lượng sản phẩm đã mua) là đặc trưng quan trọng nhất (27.03%), tiếp theo là City_Encoded (20.58%) và Average Rating (18.84%) — ba đặc trưng này chiếm hơn 66% tổng tầm quan trọng. Về mặt trực giác kinh doanh, việc Items Purchased quan trọng nhất là hợp lý (mua nhiều sản phẩm hơn thường đi kèm chi tiêu nhiều hơn); tuy nhiên việc City_Encoded đứng thứ 2 cần được nhìn nhận thận trọng, vì đây là biến danh định được mã hóa bằng số thứ tự nhân tạo (xem lại mục 2.3) — mối liên hệ mà mô hình "học" được có thể chỉ là quan hệ giả tạo do cách mã hóa, không phản ánh đúng bản chất ảnh hưởng của yếu tố địa lý.

- Lưu ý về khả năng tái lập: khác với báo cáo giá nhà (chạy 2 lần cho 2 kết quả khác nhau), notebook này chỉ chạy phân tích feature importance đúng một lần nên không thể so sánh trực tiếp độ ổn định. Tuy nhiên rủi ro tương tự vẫn tồn tại: đoạn code sinh nhiễu bằng `np.random.randn()` ngay tại đây mà không cố định lại seed ngay trước vòng lặp — nếu người dùng chạy lại đúng cell này nhiều lần (không chạy lại từ đầu notebook), thứ tự xếp hạng có thể thay đổi, đúng như đã quan sát thấy ở báo cáo giá nhà.

3.6.17. Cell 17 — So sánh với mô hình Linear Regression

```

from sklearn.linear_model import LinearRegression

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

y_pred_lr_original = scaler_y.inverse_transform(y_pred_lr.reshape(-1, 1))

mse_lr = mean_squared_error(y_test_original, y_pred_lr_original)
rmse_lr = np.sqrt(mse_lr)
r2_lr = r2_score(y_test_original, y_pred_lr_original)

print("=" * 60)
print("📊 SO SÁNH: 5-Layer Network vs Linear Regression")
print("=" * 60)
print(f'{"Metric":<15} {"5-Layer Network":<20} {"Linear Regression":<20}')
```

```

print("-" * 60)
print(f'{"RMSE":<15} {"rmse":.4f} {"rmse_lr:.4f}')
```

```

print(f'{"R²":<15} {"r2":.4f} {"r2_lr:.4f}')
```

Output:

```

=====
📊 SO SÁNH: 5-Layer Network vs Linear Regression
=====
```

Metric	5-Layer Network	Linear Regression
RMSE	17.0669	30.6751
R²	0.9978	0.9930

Đây là cell duy nhất trong cả ba báo cáo có so sánh trực tiếp mạng nơ-ron tự cài đặt với một mô hình Machine Learning truyền thống (Linear Regression) trên cùng bộ dữ liệu — một thực hành rất tốt trong nghiên cứu/báo cáo Machine Learning, giúp trả lời câu hỏi "liệu mô hình phức tạp hơn có thực sự cần thiết?". `LinearRegression().fit(X_train, y_train)` huấn luyện một mô hình hồi quy tuyến tính đơn giản (chỉ tìm 1 vector trọng số duy nhất, không có lớp ẩn hay phi tuyến) trên cùng dữ liệu đã chuẩn hóa.

Kết quả so sánh rất đáng chú ý: Linear Regression (mô hình đơn giản, gần như không có "độ sâu") vẫn đạt $R^2 = 0.9930$ — chỉ thấp hơn mạng nơ-ron 5 lớp phức tạp một chút (0.9978). Điều này gợi ý mạnh mẽ rằng mối quan hệ giữa 9 đặc trưng đầu vào và Total Spend trong bộ dữ liệu này có bản chất gần như tuyến tính (hoặc dữ liệu được tạo ra theo công thức có yếu tố tuyến tính chiếm ưu thế) — một phát hiện quan trọng cần thảo luận sâu ở Chương 9.

3.6.18. Cell 18 — Hàm dự đoán cho khách hàng mới

```

def predict_customer(customer_data):
    """
    Dự đoán Total Spend cho khách hàng mới
    Parameters: customer_data: dict với các keys Age, Items Purchased,
    Average Rating, Days Since Last Purchase, Gender, City,
    Membership Type, Satisfaction Level, Discount Applied
    """
    gender_enc = le_gender.transform([customer_data['Gender']])[0]
    city_enc = le_city.transform([customer_data['City']])[0]
    membership_enc = le_membership.transform([customer_data['Membership Type']])[0]
    satisfaction_enc = le_satisfaction.transform([customer_data['Satisfaction Level']])[0]
    discount_enc = le_discount.transform([customer_data['Discount Applied']])[0]

    features = np.array([[
        customer_data['Age'], customer_data['Items Purchased'],
        customer_data['Average Rating'], customer_data['Days Since Last Purchase'],
        gender_enc, city_enc, membership_enc, satisfaction_enc, discount_enc
    ]])

    features_scaled = scaler_X.transform(features)
    pred_norm, _ = forward(features_scaled)
    pred = scaler_y.inverse_transform(pred_norm)
    return pred[0, 0]

test_customer = {
    'Age': 30, 'Items Purchased': 15, 'Average Rating': 4.5,
    'Days Since Last Purchase': 20, 'Gender': 'Female', 'City': 'New York',
    'Membership Type': 'Gold', 'Satisfaction Level': 'Satisfied',
    'Discount Applied': True
}

predicted_spend = predict_customer(test_customer)
print("=" * 60)
print("🏠 DỰ ĐOÁN CHO KHÁCH HÀNG MỚI")
print("=" * 60)
print(f"📋 Thông tin khách hàng:")
for key, value in test_customer.items():
    print(f" {key}: {value}")
print(f"💰 Total Spend dự đoán: ${predicted_spend:.2f}")

```

Output:

DỰ ĐOÁN CHO KHÁCH HÀNG MỚI

Thông tin khách hàng:

Age: 30
Items Purchased: 15
Average Rating: 4.5
Days Since Last Purchase: 20
Gender: Female
City: New York
Membership Type: Gold
Satisfaction Level: Satisfied
Discount Applied: True

 Total Spend dự đoán: \$1175.84

Đây là hàm dự đoán phức tạp nhất trong cả ba báo cáo vì phải xử lý dữ liệu đầu vào hỗn hợp (vừa số vừa chữ). Năm dòng `.transform()` đầu tiên dùng lại đúng 5 đối tượng `LabelEncoder` đã fit từ lúc huấn luyện (`le_gender`, `le_city`...) để mã hóa các trường dạng chữ trong `customer_data` thành số — bắt buộc phải dùng lại đúng các encoder này (không tạo `LabelEncoder` mới) để đảm bảo ánh xạ nhất quán (ví dụ 'Gold' phải luôn được mã hóa thành đúng số đã dùng lúc huấn luyện).

Sau khi có đủ 9 giá trị số, `np.array([[...]])` đóng gói thành ma trận (1, 9), `scaler_X.transform()` chuẩn hóa theo đúng thống kê đã học, `forward()` chạy qua mạng 5 lớp, và `scaler_y.inverse_transform()` chuyển kết quả về đơn vị USD gốc. Với khách hàng mẫu (30 tuổi, mua 15 sản phẩm, đánh giá trung bình 4.5, hạng Gold, hài lòng, có giảm giá), mô hình dự đoán mức chi tiêu 1175.84 USD — hợp lý vì các chỉ số của khách hàng này (nhiều sản phẩm, đánh giá cao, hạng thành viên cao) đều là các đặc điểm của nhóm khách hàng chi tiêu nhiều.

3.6.19. Cell 19 — Lưu toàn bộ mô hình và các bộ mã hóa/chuẩn hóa

```
model_params = {
    'W1': W1, 'b1': b1, 'W2': W2, 'b2': b2, 'W3': W3, 'b3': b3,
    'W4': W4, 'b4': b4, 'W5': W5, 'b5': b5,
    'scaler_X': scaler_X, 'scaler_y': scaler_y,
    'le_gender': le_gender, 'le_city': le_city,
    'le_membership': le_membership, 'le_satisfaction': le_satisfaction,
    'le_discount': le_discount, 'feature_cols': feature_cols
}

np.savez('customer_behavior_model.npz', **model_params)
print("✅ Đã lưu model vào 'customer_behavior_model.npz'")
```

Output:

✅ Đã lưu model vào 'customer_behavior_model.npz'

Điểm khác biệt quan trọng so với báo cáo giá nhà: ngoài 10 tham số mạng nơ-ron, `model_params` ở đây còn lưu thêm toàn bộ 2 đối tượng `StandardScaler` (`scaler_X`, `scaler_y`) và 5 đối tượng `LabelEncoder` — đây là các đối tượng Python phức tạp (không chỉ là mảng số như `W`, `b`), `np.savez()`

vẫn có thể lưu được nhờ NumPy hỗ trợ lưu "pickled objects" bên trong file .npz. Việc lưu đầy đủ cả mô hình lẫn toàn bộ pipeline tiền xử lý là điều bắt buộc trong thực tế triển khai, vì nếu chỉ lưu 10 tham số mạng nơ-ron mà thiếu các scaler/encoder, sẽ không thể tái tạo lại chính xác bước tiền xử lý cho dữ liệu mới, dẫn đến dự đoán sai.

3.7. GIẢI THÍCH CHI TIẾT CÁC HÀM VÀ CÔNG CỤ TIỀN XỬ LÝ

Chương này giải thích sâu cả các hàm tự cài đặt bằng NumPy (giống cấu trúc báo cáo giá nhà) lẫn các công cụ tiền xử lý mới của Scikit-learn xuất hiện lần đầu trong báo cáo này.

3.7.1. LabelEncoder — Mã hóa biến phân loại

```
le_city = LabelEncoder()
df['City_Encoded'] = le_city.fit_transform(df['City'])
```

Thành phần	Giải thích
LabelEncoder()	Khởi tạo một đối tượng mã hóa rỗng, chưa biết bất kỳ giá trị phân loại nào
.fit_transform(cột)	Kết hợp 2 bước: fit (quét toàn bộ cột, liệt kê các giá trị duy nhất theo thứ tự bảng chữ cái, gán mỗi giá trị một số nguyên bắt đầu từ 0) và transform (thay từng giá trị chữ trong cột bằng số nguyên tương ứng)
.classes_	Thuộc tính lưu lại đúng danh sách giá trị gốc theo thứ tự đã gán số — dùng để tra cứu ngược (số nào ứng với giá trị chữ nào) và bắt buộc khi cần .transform() dữ liệu mới sau này
.transform() (không .fit) trên dữ liệu mới	Áp dụng lại đúng ánh xạ đã học, không học ánh xạ mới — nếu gặp giá trị chưa từng thấy lúc fit sẽ báo lỗi (ValueError)
Hạn chế	Ngầm định thứ tự giữa các giá trị dù dữ liệu gốc có thể không có thứ tự (biến danh định) — xem phân tích ở mục 2.3 và Chương 9

3.7.2. StandardScaler — Chuẩn hóa dữ liệu

```
scaler_X = StandardScaler()
X_train = scaler_X.fit_transform(X_train)
X_test = scaler_X.transform(X_test)
```

Thành phần	Giải thích
StandardScaler()	Khởi tạo một đối tượng chuẩn hóa, chưa có thống kê nào được học
.fit_transform(X_train)	Học mean và độ lệch chuẩn của từng cột trong X_train (lưu vào .mean_ và .scale_) rồi áp dụng ngay công thức $(x - \text{mean}) / \text{std}$

<code>.transform(X_test)</code>	Chỉ áp dụng lại đúng mean/std đã học từ <code>X_train</code> (không tính lại) — đảm bảo nguyên tắc chống rò rỉ thông tin (data leakage) giống hệ thống tự viết thủ công ở hai báo cáo trước
<code>.inverse_transform(y_pred)</code>	Phép biến đổi ngược: nhân lại với std rồi cộng lại mean, dùng để chuyển kết quả dự đoán đã chuẩn hóa về đơn vị gốc (USD)
So sánh với cách tự viết ở báo cáo giá nhà	Về mặt toán học hoàn toàn tương đương công thức $x' = (x - \mu) / \sigma$ tự viết trước đây; <code>StandardScaler</code> chỉ đóng gói tiện lợi hơn (tự động lưu mean/std bên trong đối tượng, không cần quản lý biến mean, std riêng lẻ)

3.7.3. `train_test_split` — Chia tập dữ liệu

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Hàm này thực hiện đúng hai việc mà hai báo cáo trước phải tự viết bằng tay: (1) xáo trộn ngẫu nhiên thứ tự các mẫu, và (2) cắt theo tỉ lệ chỉ định (`test_size=0.2` nghĩa là giữ lại 20% làm tập kiểm thử). Hàm trả về 4 giá trị theo đúng thứ tự cố định `X_train`, `X_test`, `y_train`, `y_test` — cần gán biến đúng thứ tự này để tránh nhầm lẫn dữ liệu. Tham số `random_state=42` giữ vai trò tương đương `np.random.seed(42)`, đảm bảo kết quả chia tách có thể tái lập giữa các lần chạy khác nhau.

3.7.4. Các hàm `relu`, `linear`, `mse_loss`, `mse_derivative`

Bốn hàm này có công thức toán học và vai trò hoàn toàn giống hệt đã giải thích chi tiết ở báo cáo dự đoán giá nhà (Chương 7 của báo cáo đó): `relu(x) = max(0, x)` và `relu_derivative(x)` đưa phi tuyến vào các lớp ẩn; `linear(x) = x` giữ nguyên giá trị ở lớp ra vì Total Spend là số thực dương không giới hạn; `mse_loss` đo sai số bình phương trung bình; `mse_derivative = 2(y - y_hat) / N` là đạo hàm trực tiếp dùng làm điểm khởi đầu cho backpropagation. Việc các hàm này được tái sử dụng y nguyên giữa hai bài toán hồi quy khác nhau (giá nhà và chi tiêu khách hàng) một lần nữa khẳng định: một khi đã xây dựng đúng bộ "khối lắp ráp" toán học cơ bản cho một bài toán hồi quy, ta có thể áp dụng lại cho bất kỳ bài toán hồi quy nào khác chỉ bằng cách thay đổi dữ liệu và số chiều đầu vào.

3.7.5. Hàm `forward(X)` và `backward(y, cache)`

Cấu trúc hai hàm này giống hệt phiên bản trong báo cáo giá nhà (đã giải thích chi tiết dòng-theo-dòng ở Chương 7 của báo cáo đó, bao gồm bảng đối chiếu công thức toán cho từng dòng lệnh trong `backward()`). Khác biệt duy nhất về mặt số học là kích thước ma trận đầu vào `X` thay đổi từ (N, 4) thành (N, 9), kéo theo `W1` có kích thước (9, 64) thay vì (4, 64) và `dW1 = X.T @ dz1` có kích thước (9, 64) tương ứng. Toàn bộ logic lan truyền tiến/ngược qua 4 lớp ẩn ReLU và 1 lớp ra `linear` không thay đổi.

3.7.6. Hàm `predict_customer(customer_data)` — Pipeline suy luận hoàn chỉnh

So với hàm `predict_new()` ở báo cáo giá nhà (chỉ nhận 4 số thực), hàm `predict_customer()` phức tạp hơn đáng kể vì phải xử lý đầu vào hỗn hợp gồm cả số và chữ. Bảng dưới đây tóm tắt luồng xử lý:

Bước	Thao tác	Công cụ sử dụng
1	Nhận dictionary <code>customer_data</code> với các trường số (Age, Items Purchased...) và trường chữ (Gender, City...)	Python dict
2	Mã hóa 5 trường chữ thành số bằng <code>.transform()</code> của đúng 5 LabelEncoder đã fit lúc huấn luyện	<code>le_gender</code> , <code>le_city</code> , <code>le_membership</code> , <code>le_satisfaction</code> , <code>le_discount</code>
3	Ghép toàn bộ 9 giá trị (4 số gốc + 5 số vừa mã hóa) thành ma trận (1, 9) theo đúng thứ tự đã dùng lúc huấn luyện	<code>np.array([[...]])</code>
4	Chuẩn hóa ma trận (1, 9) bằng đúng thống kê đã học từ tập huấn luyện	<code>scaler_X.transform()</code>
5	Chạy lan truyền tiến qua 5 lớp mạng nơ-ron đã huấn luyện	<code>forward()</code>
6	Chuyển kết quả dự đoán từ thang chuẩn hóa về đơn vị USD gốc	<code>scaler_y.inverse_transform()</code>
7	Trích xuất giá trị số vô hướng từ ma trận (1,1) kết quả	<code>pred[0, 0]</code>

Điểm mấu chốt cần lưu ý: cả 5 LabelEncoder lẫn 2 StandardScaler dùng trong hàm này đều là những đối tượng đã được `.fit()` từ trước trên dữ liệu huấn luyện gốc (từ Cell 3 và Cell 6) — hàm `predict_customer()` chỉ gọi lại `.transform()`, không bao giờ gọi `.fit()` hay `.fit_transform()` một lần nữa. Đây là nguyên tắc bắt buộc khi triển khai mô hình Machine Learning/Deep Learning vào thực tế: mọi phép biến đổi áp dụng cho dữ liệu mới phải dùng lại chính xác các tham số/thống kê đã học từ dữ liệu huấn luyện, tuyệt đối không học lại từ dữ liệu mới (dù chỉ là 1 mẫu duy nhất).

3.8. KẾT QUẢ THỰC NGHIỆM

3.8.1. Quá trình giảm Loss trong huấn luyện

Epoch	0	200	400	600	800	1000	1400	1800	2200	2600	3000 (cuối)
Loss	0.770	0.013	0.009	0.006	0.005	0.004	0.003	0.003	0.002	0.002	0.0017

Loss giảm cực nhanh và cực sâu ngay từ 200 epoch đầu tiên (từ 0.77 xuống 0.0126, giảm hơn 60 lần), sau đó tiếp tục giảm chậm dần đều tới 0.0017 ở epoch 3000. Đây là mức loss thấp hơn rất

nhiều so với hai báo cáo trước (0.4929 ở bài toán tiểu đường, 0.6647 ở bài toán giá nhà) — cho thấy mạng đã khớp (fit) tập huấn luyện gần như hoàn hảo trên thang đo chuẩn hóa.

3.8.2. Đánh giá trên tập kiểm thử (Test set, 70 mẫu)

Độ đo	Giá trị	Ý nghĩa
MSE	291.2805	Sai số bình phương trung bình trên thang đo USD ²
RMSE	17.0669 (USD)	Trung bình mô hình lệch khoảng 17.07 USD so với chi tiêu thực tế
MAE	11.3758 (USD)	Trung bình trị tuyệt đối sai số khoảng 11.38 USD mỗi khách hàng
R ²	0.9978 (99,78%)	Mô hình giải thích được 99,78% sự biến thiên của Total Spend thực tế — gần như hoàn hảo
MAPE	1.46%	Sai số phần trăm trung bình chỉ 1.46% so với giá trị thực tế

3.8.3. So sánh với Linear Regression

Mô hình	RMSE (USD)	R ²
Mạng nơ-ron 5 lớp (NumPy tự cài đặt)	17.0669	0.9978
Linear Regression (Scikit-learn)	30.6751	0.9930

Mạng nơ-ron 5 lớp cho RMSE thấp hơn khoảng 44% so với Linear Regression (17.07 so với 30.68 USD), và R² cao hơn một chút (0.9978 so với 0.9930). Tuy nhiên điều đáng chú ý nhất ở đây là việc Linear Regression — một mô hình cực kỳ đơn giản không có lớp ẩn hay phi tuyến — cũng đã đạt R² tới 0.9930, cho thấy phần lớn (hơn 99%) sự biến thiên của Total Spend có thể được giải thích chỉ bằng một tổ hợp tuyến tính đơn giản của 9 đặc trưng đầu vào.

3.8.4. Phân tích Residuals

Thống kê Residuals	Giá trị (USD)
Trung bình (Mean)	1.9811
Độ lệch chuẩn (Std)	16.9516
Nhỏ nhất (Min)	-28.4431
Lớn nhất (Max)	89.7568

Residual trung bình rất nhỏ (1.98 USD) cho thấy mô hình không thiên lệch hệ thống. Tuy nhiên giá trị Max lên tới 89.76 USD — cao gấp hơn 5 lần độ lệch chuẩn (16.95 USD) — là dấu hiệu của ít nhất

một khách hàng ngoại lai (outlier) mà mô hình dự đoán sai lệch đáng kể so với phần lớn các trường hợp còn lại.

3.8.5. Phân tích tầm quan trọng đặc trưng (Feature Importance)

Hạng	Đặc trưng	Importance (Δ Loss)	Tỉ lệ (%)
1	Items Purchased	0.0030	27.03%
2	City_Encoded	0.0023	20.58%
3	Average Rating	0.0021	18.84%
4	Gender_Encoded	0.0009	8.15%
5	Days Since Last Purchase	0.0007	6.62%
6	Membership_Encoded	0.0007	6.26%
7	Satisfaction_Encoded	0.0006	5.48%
8	Age	0.0004	3.70%
9	Discount_Encoded	0.0004	3.35%

Ba đặc trưng quan trọng nhất — Items Purchased, City_Encoded, Average Rating — chiếm hơn 66% tổng tầm quan trọng, trong khi Age và Discount_Encoded có ảnh hưởng thấp nhất (dưới 4% mỗi đặc trưng). Cần lưu ý thận trọng với vị trí thứ 2 của City_Encoded do hạn chế của Label Encoding đã phân tích ở mục 2.3 — phân tích sâu hơn ở Chương 9.

3.8.6. Ví dụ dự đoán khách hàng mới

Thông tin khách hàng	Giá trị
Tuổi / Số sản phẩm đã mua	30 tuổi / 15 sản phẩm
Đánh giá trung bình / Số ngày từ lần mua gần nhất	4.5 / 20 ngày
Giới tính / Thành phố	Female / New York
Hạng thành viên / Mức hài lòng / Có giảm giá	Gold / Satisfied / True
 Total Spend dự đoán	1175.84 USD

3.9. NHẬN XÉT VÀ THẢO LUẬN

27. Về độ chính xác cao bất thường: $R^2 = 0.9978$ và MAPE chỉ 1.46% là những con số rất ấn tượng, nhưng cũng cần được nhìn nhận với sự thận trọng cần thiết thay vì chỉ ăn mừng đơn thuần. Kết hợp với việc mô hình Linear Regression đơn giản cũng đạt R^2 tới 0.9930 (mục 8.3), có cơ sở để nghi ngờ rằng bộ dữ liệu E-commerce Customer Behavior này có thể là dữ liệu tổng hợp (synthetic/mô phỏng) được sinh ra theo một công thức toán học có yếu tố tuyến tính chiếm ưu thế, thay vì dữ liệu thu thập từ hành vi khách hàng thực tế vốn thường nhiễu và phức tạp hơn nhiều. Nếu đúng như vậy, R^2 gần tuyệt đối không phản ánh năng lực "học sâu" đặc biệt của mạng nơ-ron, mà chỉ đơn giản là mô hình đã khớp đúng công thức sinh dữ liệu.
28. Về nguy cơ overfitting do mất cân đối tham số/dữ liệu: như đã nêu ở mục 2.4, mạng nơ-ron có khoảng 3.300 tham số huấn luyện trong khi tập huấn luyện chỉ có 280 mẫu — tỉ lệ tham số/mẫu rất cao (hơn 10 tham số cho mỗi mẫu dữ liệu). Đây là điều kiện lý tưởng để một mạng nơ-ron ghi nhớ (memorize) gần như nguyên vẹn tập huấn luyện thay vì học ra quy luật tổng quát. Loss huấn luyện giảm xuống mức cực thấp (0.0017) càng củng cố nghi ngờ này. Việc mô hình vẫn đạt R^2 cao trên tập kiểm thử (dù chỉ 70 mẫu) một phần có thể do bản chất đơn giản/tuyến tính của dữ liệu (như nhận xét 1), nhưng với một tập kiểm thử nhỏ như vậy, kết quả đánh giá cũng có độ tin cậy thống kê hạn chế — chỉ cần vài mẫu kiểm thử khác đi (do cách chia dữ liệu khác) có thể cho ra R^2 dao động đáng kể.
29. Về việc dùng mạng quá sâu/quá rộng cho bài toán này: xét về hiệu năng thực tế, chênh lệch R^2 giữa Linear Regression (0.9930) và mạng nơ-ron 5 lớp phức tạp (0.9978) là rất nhỏ (0.0048), trong khi độ phức tạp tính toán và thời gian huấn luyện của mạng nơ-ron lớn hơn rất nhiều. Trong một dự án thực tế, sự đánh đổi này cần được cân nhắc: nếu Linear Regression đã đạt hiệu năng gần tương đương với chi phí triển khai, bảo trì và diễn giải thấp hơn nhiều, đây có thể là lựa chọn thực dụng hơn cho bài toán cụ thể này.
30. Về hạn chế của Label Encoding cho biến danh định: như đã phân tích ở mục 2.3, việc mã hóa City (biến danh định, không có thứ tự tự nhiên) bằng số nguyên liên tiếp có thể khiến mạng học một mối quan hệ tuyến tính giả tạo giữa các thành phố. Việc City_Encoded được xếp hạng 2 về tầm quan trọng (mục 8.5) cần được diễn giải thận trọng: rất có thể đây một phần là ảnh hưởng thực của yếu tố địa lý (ví dụ khách hàng ở các thành phố lớn có xu hướng chi tiêu khác nhau), nhưng cũng có thể một phần là nhiễu do cách mã hóa gây ra, khó tách bạch rõ ràng nếu không thử nghiệm thêm với One-Hot Encoding để đối chiếu.
31. Về Cell 11 (Loss vs Train Loss trùng lặp): như đã chỉ ra ở Chương 6, biến train_loss trong vòng lặp huấn luyện tính trên cùng tập X_train giống hệt biến loss, khiến việc in cả hai giá trị song song trở nên dư thừa và có thể gây hiểu lầm rằng đây là validation loss (tính trên tập kiểm thử). Đây là một điểm cần sửa nếu muốn theo dõi đúng hiện tượng overfitting trong các lần thực hiện sau.
32. Về outlier trong residuals: giá trị residual lớn nhất lên tới gần 90 USD (mục 8.4), cao bất thường so với độ lệch chuẩn chung (~17 USD). Trong bối cảnh phân tích khách hàng thực tế, việc xác định và tìm hiểu riêng những khách hàng bị dự đoán sai lệch lớn có thể mang lại giá trị kinh doanh (ví dụ phát hiện các khách hàng có hành vi mua sắm đặc biệt, không tuân theo quy luật chung của phần lớn khách hàng còn lại).
33. Về thư viện Seaborn chưa sử dụng: notebook import seaborn nhưng không có cell nào tận dụng thư viện này để phân tích khám phá dữ liệu (EDA) sâu hơn, ví dụ vẽ ma trận tương

quan (correlation heatmap) giữa các đặc trưng và Total Spend — một bước phân tích có thể giúp xác nhận sớm ngay từ đầu liệu dữ liệu có quan hệ tuyến tính mạnh hay không, trước khi mất công xây dựng một mạng nơ-ron sâu phức tạp.

34. Ý nghĩa thực tiễn: với các con số hiệu năng hiện tại và những nghi vấn đã nêu về bản chất dữ liệu, mô hình này phù hợp làm bài tập minh họa kỹ thuật xây dựng mạng nơ-ron sâu kết hợp tiền xử lý Scikit-learn, nhưng trước khi áp dụng vào một hệ thống phân tích khách hàng thực tế, cần kiểm chứng thêm trên dữ liệu thực tế thu thập từ hệ thống thương mại điện tử thật (vốn thường nhiễu và phi tuyến hơn nhiều), đồng thời xử lý các hạn chế về mã hóa và kiến trúc đã nêu.

3.10. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

3.10.1. Kết luận

Báo cáo đã trình bày quá trình xây dựng một mạng nơ-ron sâu 5 lớp (kiến trúc $9 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$) để dự đoán tổng chi tiêu khách hàng thương mại điện tử, kết hợp Scikit-learn cho khâu tiền xử lý (mã hóa biến phân loại bằng LabelEncoder, chuẩn hóa bằng StandardScaler, chia dữ liệu bằng train_test_split) với một mạng nơ-ron hoàn toàn tự cài đặt bằng NumPy cho phần mô hình hóa cốt lõi. Đây là báo cáo đầu tiên trong chuỗi ba báo cáo xử lý một bộ dữ liệu có cấu trúc hỗn hợp thực sự (vừa số vừa phân loại, có giá trị thiếu), đòi hỏi một quy trình tiền xử lý đầy đủ hơn hẳn hai bài toán trước.

Mô hình đạt kết quả rất cao trên tập kiểm thử: $RMSE \approx 17.07$ USD, $MAPE \approx 1.46\%$, $R^2 \approx 0.9978$. Tuy nhiên, qua phân tích đối chiếu với mô hình Linear Regression đơn giản (cũng đạt $R^2 \approx 0.9930$) và xem xét tỉ lệ tham số/dữ liệu huấn luyện rất cao (~ 3.300 tham số cho 280 mẫu), báo cáo đã chỉ ra rằng độ chính xác cao này nhiều khả năng phản ánh bản chất gần-tuyến tính của bộ dữ liệu (có thể là dữ liệu tổng hợp) hơn là minh chứng cho việc kiến trúc sâu 5 lớp thực sự cần thiết hay vượt trội — một bài học quan trọng về việc luôn cần có mô hình baseline đơn giản để đối chiếu trước khi kết luận về hiệu năng của một mô hình phức tạp.

Báo cáo cũng đã phân tích các hạn chế về phương pháp luận đáng lưu ý trong notebook gốc: việc dùng Label Encoding cho biến danh định (City) có thể tạo ra quan hệ thứ tự giả tạo; việc tính train_loss trùng lặp với loss trong vòng lặp huấn luyện; và rủi ro về khả năng tái lập của phân tích feature importance khi không cố định seed ngẫu nhiên — những vấn đề tương tự cũng đã được ghi nhận ở báo cáo dự đoán giá nhà, cho thấy đây là các lỗi phổ biến cần đặc biệt chú ý khi xây dựng pipeline Machine Learning/Deep Learning trong thực tế.

3.10.2. Hướng phát triển / Đề xuất cải tiến

- Thử nghiệm One-Hot Encoding thay cho Label Encoding đối với các biến danh định thực sự không có thứ tự (Gender, City) — chỉ giữ Label Encoding cho biến có thứ tự tự nhiên như Membership Type (Bronze < Silver < Gold) — rồi so sánh lại hiệu năng và bảng feature importance để xem liệu thứ hạng của City có thay đổi hay không.
- Bổ sung bước phân tích khám phá dữ liệu (EDA) bằng chính thư viện Seaborn đã import nhưng chưa sử dụng — đặc biệt là vẽ ma trận tương quan (correlation heatmap) giữa các

đặc trưng và Total Spend, giúp xác nhận sớm mức độ tuyến tính của dữ liệu trước khi lựa chọn kiến trúc mô hình.

- Sửa lại cách theo dõi loss trong vòng lặp huấn luyện: tính validation loss thực sự trên X_{test} (thay vì tính lại train_loss trùng lặp trên X_{train}), để có thể phát hiện sớm hiện tượng overfitting nếu xảy ra.
- Áp dụng K-Fold Cross-Validation thay vì chỉ chia 1 lần train/test (280/70 mẫu), vì với bộ dữ liệu nhỏ (350 mẫu), một phép chia dữ liệu duy nhất có thể cho kết quả đánh giá kém ổn định — Cross-Validation sẽ cho ước lượng R^2 /RMSE đáng tin cậy hơn.
- Thử nghiệm kiến trúc mạng gọn nhẹ hơn nhiều (ví dụ $9 \rightarrow 16 \rightarrow 8 \rightarrow 1$, giảm số tham số xuống còn vài trăm) phù hợp hơn với quy mô 280 mẫu huấn luyện, so sánh với kiến trúc hiện tại ($9 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$) để kiểm chứng liệu độ phức tạp cao có thực sự cải thiện hiệu năng tương xứng hay chỉ tăng nguy cơ overfitting.
- Bổ sung kỹ thuật regularization (L2, Dropout) và/hoặc early stopping dựa trên validation loss, đặc biệt quan trọng khi tỉ lệ tham số/mẫu dữ liệu cao như trong bài toán này.
- Thu thập thêm dữ liệu thực tế (nhiều hơn 350 khách hàng) hoặc kiểm chứng lại trên một bộ dữ liệu thương mại điện tử thực tế khác, để xác nhận liệu kết quả R^2 gần tuyệt đối có lặp lại được hay chỉ là đặc thù của bộ dữ liệu tổng hợp này.
- Cố định seed ngẫu nhiên trước bước phân tích feature importance (như đã đề xuất ở báo cáo giá nhà) và lặp lại phép đo nhiều lần để có kết luận đáng tin cậy hơn về tầm quan trọng tương đối giữa các đặc trưng.
- Đóng gói toàn bộ pipeline (tiền xử lý bằng Scikit-learn + mô hình NumPy tự cài đặt) đã lưu trong file customer_behavior_model.npz thành một API hoặc dashboard đơn giản, phục vụ đội ngũ marketing/chăm sóc khách hàng tra cứu dự đoán Total Spend cho khách hàng mới trong thực tế.